Parallel Transport of Tangent Vectors on Triangle Meshes: from the Vector Heat Equation to Code, with Finite Elements

Mauricio Cele Lopez Belon

Abstract. We derive, step by step, a program that spreads a tangent vector given at one point of a triangle mesh to every other point, so that the result at each point is the parallel transport of the given vector. The method is the vector heat method: diffuse the vector by the heat equation for tangent vector fields, whose operator is the connection Laplacian, and keep only the direction of the result. We start from the partial differential equation and go all the way down to the linear system that the code solves: the covariant derivative and the Levi-Civita connection of a surface, Green's identity for tangent fields, the backward Euler step, the weak form, the finite element space of piecewise linear tangent fields, the discrete parallel transport that makes that space possible, the gradients of the shape functions on a triangle embedded in space, the element stiffness and mass matrices, their assembly, the boundary condition, and the solution. Every step is motivated, every formula is derived, and every formula that the code evaluates is matched with the function that evaluates it, so that the text can be read side by side with the implementation [13]. A second part derives the same method in geometric algebra, where the tangent plane is a bivector, coordinates are complex numbers of that plane, parallel transport is a rolling rotor and the connection Laplacian becomes a complex Hermitian matrix, and it clarifies which parts of the computation are intrinsic to the curved surface. A third part removes the last extrinsic ingredient, the tangent plane of a vertex, and builds the whole method from the edge lengths of the mesh alone: the metric of a face is its Gram matrix, the plane of a vertex is the cone of its angles rescaled to a full turn, and the connection matches the bisectors of the corners; the result is exact on bent and folded meshes. A fourth part moves the curvature from the vertices into the faces, with the connection as a discrete 1-form on the edges and a finite element whose connection is curved inside each face, and a fifth part builds discrete exterior calculus from the beginning and shows that its connection Laplacian is that finite element with the curvature terms

dropped. A sixth part is a map of the whole: one pipeline shared by all the methods, the three choices that distinguish them, where each keeps the curvature of the mesh, an exact formula relating the corner transport of the third part to the edge transport of the fourth, the connection form and vector-valued forms in every part, Cartan's moving frames as the common language of the methods with a sketch of a curved element of degree two built on them, the Weitzenböck identity that separates the connection Laplacian from the Hodge Laplacian, the extrinsic finite element method of numerical analysis, a concept map, a master table, and a guide to choosing and extending a method.

Mathematics Subject Classification (2010). 65N30 Finite element methods; 53A05 Surfaces in Euclidean space; 65F05 Direct methods for linear systems.

Keywords. Finite Element Method, Connection Laplacian, Levi-Civita Connection, Parallel Transport, Vector Heat Method.

1. Introduction

Suppose we are given one tangent vector at one point of a curved surface, and we want to know what that vector "is" at every other point: the vector we would obtain by carrying it along the surface without turning it. On a plane the answer is trivial, the same vector everywhere. On a curved surface it is not, for two reasons. First, the tangent planes at different points are different planes, so the vector cannot simply be copied; it has to be carried, and the rule that says how is the *parallel transport* of the surface, given by its *Levi-Civita connection*. Second, the result depends on the path: carrying a vector around a closed loop on a sphere brings it back rotated, by an angle equal to the curvature enclosed. A sensible global answer is to carry the vector along the shortest path from the source, the geodesic.

The *vector heat method* of Sharp, Soliman and Crane [17] computes that answer without computing a single geodesic. Its idea is simple. Heat spreading from a point source travels, for a short time, mostly along shortest paths. If what spreads is not a temperature but a tangent vector, which diffuses by the heat equation for tangent vector fields, then the direction of the diffused vector at each point is the direction of the source vector carried along the shortest path; only its length decays with distance. So: diffuse the vector for a short time, and normalize.

This text derives a finite element implementation of that idea [13], following the classical presentations of the finite element method [11, 15] and extending them from scalar functions on flat domains to tangent vector fields on surfaces. The derivation has a fixed structure, which recurs in every finite element method and which we follow strictly:

1. the continuous problem: the operator, the equation, the boundary condition (Sections 2–4);

2. the weak form, which needs only first derivatives (Section 5);
3. a finite dimensional space of candidate solutions, the *trial space*, and of *test functions* (Section 6);
4. the element matrices, computed triangle by triangle (Sections 7–8);
5. the assembly of the global matrices, the boundary condition, and the solution of the linear system (Sections 9–10).

The ingredient that is not in the scalar method is the connection: tangent vectors at different vertices live in different planes and cannot be added before they are brought into a common plane. Section 3 shows how the code does that, and Section 6 shows that this is exactly where the Levi-Civita connection enters the finite element method. Table 1 at the end of Part I lists every formula with the function of `FEM.cpp` that evaluates it. Part II repeats the derivation in the language of geometric algebra, where frames become a single vector, coordinates become complex numbers of the tangent plane, transports become rotors, and the gradients of the shape functions become a reciprocal frame; it also separates what is intrinsic to the curved surface from what depends on its embedding in space. Part III builds the same method from the edge lengths alone, slowly and with every derivation, and compares the two connections on a folded sheet, a sphere and the cactus. Part IV transports along the edges instead, puts the curvature into the faces and derives the finite element that goes with it, and Part V explains discrete exterior calculus slowly and places it among the others. Part VI is a map of all five, meant to be read on its own: a reader who returns to this text after a long time may want to start there.

Part I. The Derivation with Matrices

2. Tangent Vectors on a Surface

2.1. Tangent planes and frames

Let $\Omega \subset \mathbb{R}^3$ be a smooth surface with unit normal $n(x)$ at each point x. The *tangent plane* $T_x\Omega$ is the plane through the origin orthogonal to $n(x)$, and a *tangent vector* at x is a vector of $\mathbb{R}^3$ in that plane. A *tangent vector field* U assigns to every point x a tangent vector $U(x) \in T_x\Omega$.

A vector of $\mathbb{R}^3$ needs three numbers, but a tangent vector needs only two, once we choose a basis of the tangent plane. We choose at each point an orthonormal, positively oriented *frame* $(t(x), b(x))$: two unit tangent vectors, orthogonal to each other, with $b = n \times t$. Collecting them as the columns of a 3×2 matrix,

$$T(x) = \big[\, t(x) \;\; b(x) \,\big], \qquad T^T T = I_2, \qquad TT^T = I_3 - nn^T, \tag{2.1}$$

a tangent vector $U(x)$ has the coordinates $u = T^T U \in \mathbb{R}^2$, and is recovered as $U = Tu$. The two identities say that the columns are orthonormal, and that TT^T is the orthogonal projection of $\mathbb{R}^3$ onto the tangent plane. Since the frame is orthonormal, coordinates preserve lengths and angles: $U \cdot V = u \cdot v$ for tangent vectors at the same point.

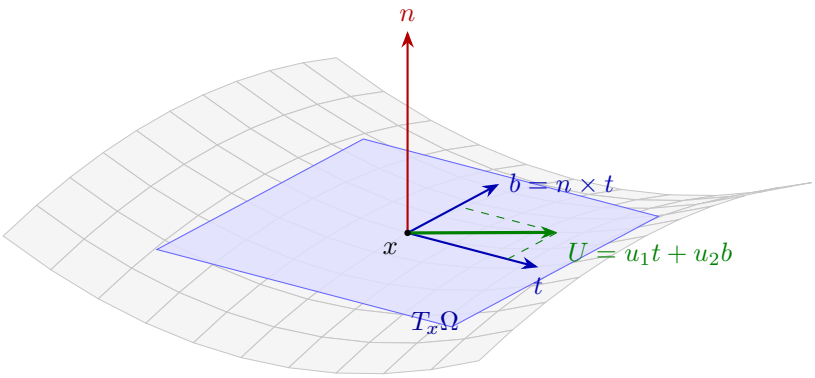


FIGURE 1. A point x of a surface, its normal n, its tangent plane $T_x\Omega$ (blue), an orthonormal frame (t, b) of the tangent plane, and a tangent vector U with its two coordinates u_1, u_2 in that frame.

The choice of t is arbitrary: any rotation of the frame in the tangent plane is as good. The physics does not depend on that choice, and we will see that the method does not either, because every quantity that compares vectors at different points will carry the change of frames with it.

2.2. The discrete surface

The surface is a triangle mesh with vertices $x_1, \dots, x_n$ and triangular faces. For a face f with vertices $x_{f_1}, x_{f_2}, x_{f_3}$ in counterclockwise order, its unit normal and area are

$$n_f = \frac{(x_{f_2} - x_{f_1}) \times (x_{f_3} - x_{f_1})}{|(x_{f_2} - x_{f_1}) \times (x_{f_3} - x_{f_1})|}, \qquad A_f = \tfrac{1}{2}|(x_{f_2} - x_{f_1}) \times (x_{f_3} - x_{f_1})|, \tag{2.2}$$

which is `computeFaceNormals`. A face is flat: all its points share the tangent plane orthogonal to n_f. A vertex has no tangent plane of its own, since it is a corner shared by several faces, so we must choose one. The code takes the plane orthogonal to the *vertex normal*, the normalized sum of the unnormalized normals of the incident faces (`Mesh::computeNormals`), which weighs each face by its area:

$$n_i = \frac{\sum_{f \ni i} 2A_f\, n_f}{\left|\sum_{f \ni i} 2A_f\, n_f\right|}. \tag{2.3}$$

Frames. Each vertex and each face gets an orthonormal frame:

- *vertex* i (`computeVertexTangentSpace`): t_i is the projection, onto the tangent plane of i, of the edge from x_i to one of its neighbors x_j, normalized; and $b_i = n_i \times t_i$:

$$t_i = \frac{(x_j - x_i) - \big((x_j - x_i) \cdot n_i\big)\, n_i}{\big|(x_j - x_i) - \big((x_j - x_i) \cdot n_i\big)\, n_i\big|}, \qquad T_i = \big[\, t_i \;\; n_i \times t_i \,\big]; \tag{2.4}$$

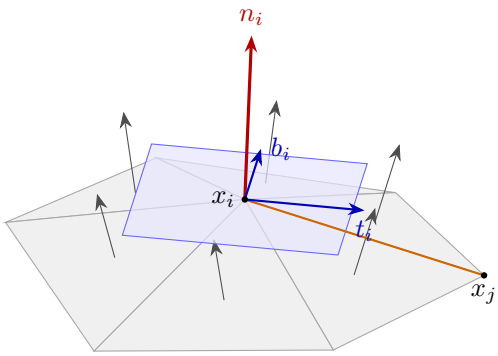


FIGURE 2. The one-ring of a vertex x_i. Each face has its own normal (gray arrows); the vertex normal n_i (red) is their area-weighted average (2.3). The vertex frame vector t_i is the edge to a neighbor x_j (orange), projected onto the vertex tangent plane (blue) and normalized, and $b_i = n_i \times t_i$.

- *face* f (`computeFaceTangentSpace`): t_f is the direction of its first edge, and $T_f = [\, t_f \;\; n_f \times t_f \,]$.

Unknowns. The unknown field is represented by one tangent vector per vertex, stored by its two coordinates in the vertex frame: $u_i \in \mathbb{R}^2$, with the vector $T_i u_i \in \mathbb{R}^3$. All the unknowns form one vector of $2n$ numbers,

$$\mathrm{u} = (u_{1,1}, u_{1,2}, u_{2,1}, u_{2,2}, \ldots, u_{n,1}, u_{n,2}), \tag{2.5}$$

so that the coordinate $k \in \{1, 2\}$ of vertex i is the entry $2(i-1)+k$, or $2i+k$ with the zero-based indices of the code (`v->ID * 2 + row`). It is often convenient to read u_i as the complex number $u_{i,1} + \mathrm{i}\, u_{i,2}$: a rotation of the tangent plane by an angle α is then multiplication by $e^{\mathrm{i}\alpha}$. The code uses this reading to set the source vector and to draw the solution (Section 10).

3. Parallel Transport and the Levi-Civita Connection

3.1. The covariant derivative

To differentiate a tangent field we would like to compare $U(x + \epsilon X)$ with $U(x)$, but they lie in different tangent planes, and their difference is not a tangent vector. Differentiating in $\mathbb{R}^3$ is still possible: the *directional derivative* $D_X U$ of the three components of U in a tangent direction X is a vector of $\mathbb{R}^3$. It has a tangent part, which measures how U turns within the surface, and a normal part, which measures only how the surface itself bends under U. The *covariant derivative* keeps the tangent part:

$$\nabla_X U = \left(I - nn^T\right) D_X U = TT^T D_X U. \tag{3.1}$$

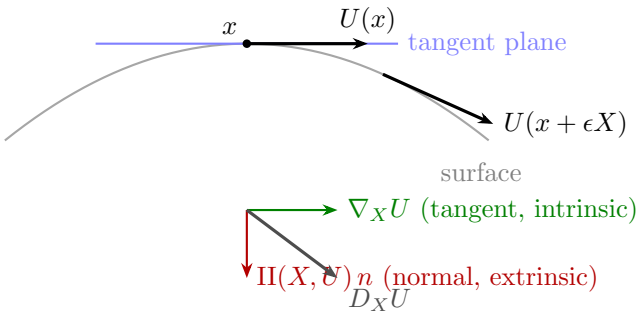


FIGURE 3. The covariant derivative. U at two nearby points lies in two different tangent planes; its ambient derivative $D_X U$ has a tangent part $\nabla_X U$, the covariant derivative, and a normal part, which depends only on how the surface bends (Section 14).

Figure 3 shows the split in a section of the surface: the tangent part measures the turning of U within the surface, the normal part only the bending of the surface.

This is the *Levi-Civita connection* of a surface in $\mathbb{R}^3$ [4]. Two of its properties will be used repeatedly. It is *compatible with the metric*: for tangent fields U, V,

$$X(U \cdot V) = \nabla_X U \cdot V + U \cdot \nabla_X V, \tag{3.2}$$

because $X(U \cdot V) = D_X U \cdot V + U \cdot D_X V$, and the normal parts of $D_X U$ and $D_X V$ are orthogonal to the tangent vectors V and U. And it is linear in X, so at each point it defines a linear map $\nabla U : X \mapsto \nabla_X U$ from the tangent plane to itself, the *total covariant derivative*, a 2×2 matrix in a frame.

In a frame. Write $U = Tu$ with a frame field T. Then $D_X U = (D_X T)\, u + T\,(X \cdot \nabla u)$, and projecting with TT^T,

$$T^T \nabla_X U = X \cdot \nabla u + \omega(X)\, u, \qquad \omega(X) = T^T D_X T. \tag{3.3}$$

The first term is the ordinary derivative of the coordinates; the second corrects for the turning of the frame. The 2×2 matrix $\omega(X)$ is skew-symmetric, because differentiating $T^T T = I$ gives $\omega + \omega^T = 0$: it is $\begin{pmatrix} 0 & -\theta(X) \\ \theta(X) & 0 \end{pmatrix}$, an infinitesimal rotation by the angle $\theta(X)$, the *connection form* of the frame [12]. On a flat piece of surface with a constant frame, $\omega = 0$, and the covariant derivative is the ordinary derivative of the coordinates. This is the situation inside one face of a mesh, and it is what will make the finite element computation simple. Computing with a frame field that is carried along the surface, and recording the turning of the frame in ω, is *Cartan's method of moving frames* [2,5]; Section 42 shows that every part of this text is a choice of frames and of their connection form.

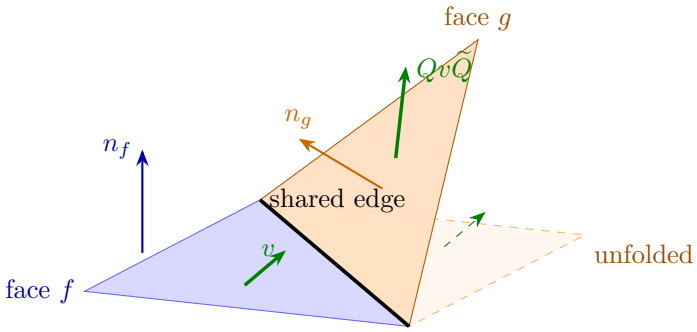


FIGURE 4. Transport across an edge. Two faces sharing an edge meet at an angle in space; unfolding face g into the plane of face f (dashed) is a rotation about the shared edge, and that rotation is exactly the smallest rotation taking n_f to n_g. The vector v of face f and its transport $Qv\widetilde{Q}$ into face g are the same vector of the unfolded plane.

3.2. Parallel transport

A field U is *parallel* along a curve γ if it does not change along it in the sense of the connection, $\nabla_{\gamma'} U = 0$. Given U at the start of the curve, this linear differential equation determines it along the whole curve: that is the parallel transport of the vector along γ. By (3.2), $\frac{d}{ds}(U \cdot V) = 0$ for two parallel fields, so parallel transport preserves lengths and angles: it is a rotation from one tangent plane onto another.

On a mesh we need it only between a vertex and an incident face, two points whose tangent planes, with normals n_i and n_f, are close. Of all the rotations of $\mathbb{R}^3$ that take n_i to n_f, and therefore the tangent plane of i onto that of f, the natural one is the *smallest*: the rotation about the axis $n_i \times n_f$ by the angle between the normals. It is the discrete analogue of the transport along the short path from the vertex into the face. When the two planes are those of two faces sharing an edge, the axis $n_f \times n_g$ is the direction of the shared edge, and this rotation is exactly the *unfolding* of one face onto the plane of the other about their common edge, which is how a vector is transported across an edge of a polyhedral surface. (Figure 4.) The smallest rotation is given by the Rodrigues formula.

Proposition 3.1. *Let a and c be unit vectors with $a \cdot c \neq -1$, and let $v = a \times c$ and $[v]_\times$ the matrix of $w \mapsto v \times w$. Then*

$$Q = I + [v]_\times + \frac{1}{1 + a \cdot c} [v]_\times^2 \tag{3.4}$$

is the rotation about the axis v that takes a to c.

Proof. Let ϕ be the angle between a and c, so that $a \cdot c = \cos\phi$ and $|v| = \sin\phi$, and let $k = v/|v|$ be the unit axis. The Rodrigues rotation by ϕ about k is

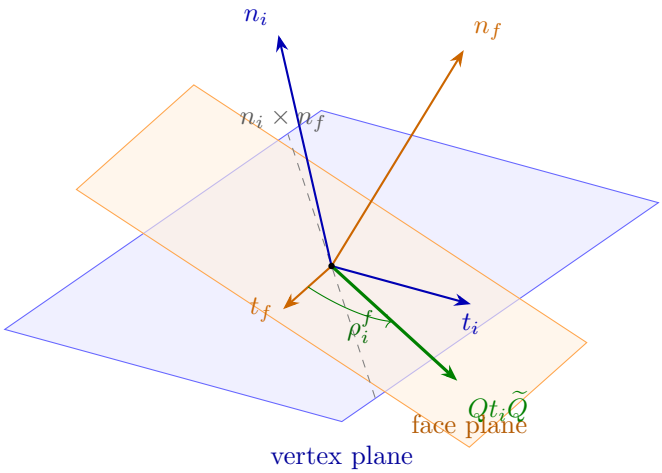


FIGURE 5. The discrete transport from vertex i into face f. The smallest rotation Q, about the axis $n_i \times n_f$ (dashed), takes the vertex plane (blue) onto the face plane (orange); the frame vector t_i becomes $Qt_i\widetilde{Q}$ (green, written as a rotor as in Part II), which makes the angle ρ_i^f with the face frame vector t_f. The matrix R_i^f is the rotation by ρ_i^f.

$I + \sin\phi\,[k]_\times + (1-\cos\phi)\,[k]_\times^2$. Since $[v]_\times = \sin\phi\,[k]_\times$ and $[v]_\times^2 = \sin^2\phi\,[k]_\times^2$, and $\frac{\sin^2\phi}{1+\cos\phi} = \frac{1-\cos^2\phi}{1+\cos\phi} = 1-\cos\phi$, this is (3.4). It takes a to c because a is orthogonal to the axis k and the angle from a to c about k is ϕ. □

This is `alignVectors`(n_i, n_f), with $[v]_\times$ computed by `bracket`. Figure 5 shows the rotation and the planes it connects. The formula fails when the normals are opposite, $a \cdot c = -1$, where the axis is undetermined; the code then returns $-I$, which only occurs for degenerate meshes (a face folded over onto its vertex) and is not a rotation.

The discrete connection. The rotation $Q_{i\to f}$ takes tangent vectors of vertex i to tangent vectors of face f. In coordinates, a tangent vector $T_i u$ of i becomes $Q_{i\to f} T_i u$, whose coordinates in the face frame are $T_f^T Q_{i\to f} T_i u$. So the transport from vertex i to face f, in coordinates, is the 2×2 matrix

$$R_i^f = T_f^T\, Q_{i\to f}\, T_i, \tag{3.5}$$

which is `Rvf`(v_i, f).

Proposition 3.2. *R_i^f is a rotation of $\mathbb{R}^2$: $(R_i^f)^T R_i^f = I_2$ and $\det R_i^f = 1$.*

Proof. The columns of T_i are orthonormal and orthogonal to n_i, so those of $Q_{i\to f}T_i$ are orthonormal and orthogonal to $Q_{i\to f}\, n_i = n_f$: they are an orthonormal basis of the tangent plane of f. The columns of T_f are another one, so T_f^T, restricted to that plane, is an isometry onto $\mathbb{R}^2$, and R_i^f, the

matrix of a change between two orthonormal bases of the same plane, is orthogonal. Both bases are positively oriented with respect to n_f: T_f by construction, and $Q_{i\to f}T_i$ because the rotation Q preserves orientation and takes $n_i \times t_i = b_i$ to $n_f \times Qt_i$. So the determinant is $+1$. □

Worked example 3.3. *Let the vertex have the normal $n_i = (0,0,1)$ and the frame vector $t_i = (0,1,0)$, so $b_i = n_i \times t_i = (-1,0,0)$, and let the face have the normal $n_f = (0,-1,1)/\sqrt{2}$ and the frame vector $t_f = (1,0,0)$, so $b_f = n_f \times t_f = (0,1,1)/\sqrt{2}$ (this is the face of Example 7.2). Then $n_i \cdot n_f = 1/\sqrt{2}$ and $v = n_i \times n_f = (1/\sqrt{2},0,0)$: Q is the rotation by $45°$ about the x axis, and it takes $t_i = (0,1,0)$ to $(0,1,1)/\sqrt{2} = b_f$ and b_i to itself. So the transported t_i is the second face frame vector, and*

$$R_i^f = T_f^T Q T_i = \begin{pmatrix} t_f \cdot Qt_i & t_f \cdot Qb_i \\ b_f \cdot Qt_i & b_f \cdot Qb_i \end{pmatrix} = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix},$$

the rotation by $\rho_i^f = 90°$. The program `ga_fem` prints these numbers (Section 17).

A rotation of the plane is multiplication by a unit complex number, $R_i^f \leftrightarrow e^{\mathrm{i}\rho_i^f}$: the angle ρ_i^f is the discrete connection between the frame of vertex i and that of face f. The angle has two contributions, the turning of the tangent plane, from Q, and the arbitrary choice of the two frames, which is why only combinations in which the frame choices cancel have a geometric meaning.

Transport between two vertices. To bring a vector from vertex j to vertex i through a face f that contains both, go from j into the face and from the face back to i:

$$R_{j\to i}^f = (R_i^f)^{-1} R_j^f = (R_i^f)^T R_j^f. \tag{3.6}$$

Here the choice of the face frame cancels: replacing T_f by $T_f S$ for a rotation S of the plane changes R_i^f to $S^T R_i^f$ and R_j^f to $S^T R_j^f$, and leaves $(R_i^f)^T R_j^f$ unchanged. The choices of the vertex frames do not cancel, and should not: they define the coordinates of the unknowns. This product is the matrix that will appear in the stiffness matrix. The first version of the code, `AssembleStiffnessElementEmbedded`, uses instead the smallest rotation from n_j to n_i directly, $\mathtt{Rvv}(v_j, v_i) = T_i^T Q_{j\to i} T_j$, which is the same to first order in the angles between the normals; the version used, `AssembleStiffnessElementEmbedded2`, goes through the face as in (3.6), which is what the finite element derivation of Section 6 produces.

Remark 3.4 (Curvature). *Going around a vertex from face to face, the transports (3.6) through consecutive faces compose to a rotation that is not the identity in general. For the intrinsic transport of the polyhedral surface (unfolding the faces one after another) its angle is the* angle defect $2\pi - \sum_f \alpha_f$ *of the vertex, the discrete Gaussian curvature; for the transport through vertex normals used here, it approximates the angle defect on fine meshes (Section 14). This is the path dependence of parallel transport mentioned in the*

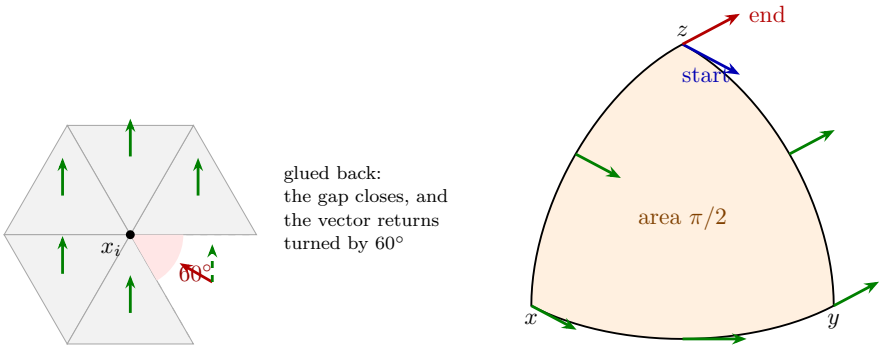


FIGURE 6. Holonomy. Left: the one-ring of a vertex of five equilateral triangles, unfolded into the plane; its angles add up to 300°, and a vector transported around the vertex (constant in the unfolded plane) comes back turned by the angle defect 60° when the gap is closed. Right: on the unit sphere, a vector carried around the octant bounded by three quarter great circles comes back turned by 90°, the area $\pi/2$ of the octant.

introduction, and the reason why the transported field must be defined globally by an equation, rather than by composing transports along one particular path.

Worked example 3.5. *Five equilateral triangles around a vertex (Figure 6, left) form a cone with total angle* $5 \cdot 60° = 300°$*, a pentagonal pyramid. Cutting it along an edge and laying it flat leaves a gap of* $60°$*. A vector carried around the vertex keeps its direction in the flat layout, and when the gap is glued shut it meets its starting copy turned by* $60° = 2\pi - 300°$*, the angle defect. On the unit sphere (right), start at the north pole with a vector, go down a meridian to the equator, along the equator by a quarter turn, and up another meridian back to the pole. Along each great circle the vector keeps its angle with the direction of motion, and it returns turned by* $90°$*, which is the area* $4\pi/8 = \pi/2$ *of the octant: on a sphere of curvature* $K = 1$*, holonomy is enclosed area.*

4. The Connection Laplacian and the Vector Heat Equation

4.1. The Dirichlet energy and the connection Laplacian

How much does a field turn? The natural measure is the *Dirichlet energy*

$$E(U) = \frac{1}{2}\int_{\Omega} |\nabla U|^2 \, dA, \qquad |\nabla U|^2 = \nabla U : \nabla U = \sum_{k=1}^{2} |\nabla_{e_k} U|^2, \tag{4.1}$$

where e_1, e_2 is any orthonormal basis of the tangent plane and $A : B = \operatorname{tr}(A^T B)$ is the inner product of 2×2 matrices. $E(U) = 0$ exactly when $\nabla U = 0$ everywhere, that is, when U is parallel along every curve; on a curved surface such fields do not exist, and the fields of smallest energy are the "most parallel" ones.

The operator associated with this energy is found, as always, by differentiating it. For a variation $U + \epsilon V$, the derivative of E at $\epsilon = 0$ is $\int \nabla U : \nabla V \, dA$, and we want to write it as $\int (\cdots) \cdot V \, dA$, with no derivative on V. The tool is Green's identity for tangent fields.

Proposition 4.1 (Green's identity). *For tangent fields U and V on a surface Ω with boundary $\partial\Omega$ and outward unit conormal ν,*

$$\int_\Omega \nabla U : \nabla V \, dA = \int_\Omega (\nabla^* \nabla U) \cdot V \, dA + \oint_{\partial\Omega} (\nabla_\nu U) \cdot V \, ds, \tag{4.2}$$

where the connection Laplacian *is* $\nabla^* \nabla U = -\sum_k \left(\nabla_{e_k} \nabla_{e_k} U - \nabla_{\nabla_{e_k} e_k} U \right)$.

Proof. Consider the tangent field W defined by $W \cdot X = (\nabla_X U) \cdot V$ for every tangent X. Its divergence, computed with the connection, is $\operatorname{div} W = \sum_k (\nabla_{e_k} W) \cdot e_k$, and by the metric compatibility (3.2) applied twice,

$$\begin{aligned} \operatorname{div} W &= \sum_k \Big(\left(\nabla_{e_k} \nabla_{e_k} U - \nabla_{\nabla_{e_k} e_k} U \right) \cdot V + \nabla_{e_k} U \cdot \nabla_{e_k} V \Big) \\ &= -(\nabla^* \nabla U) \cdot V + \nabla U : \nabla V. \end{aligned}$$

Integrating over Ω and using the divergence theorem, $\int_\Omega \operatorname{div} W \, dA = \oint_{\partial\Omega} W \cdot \nu \, ds = \oint (\nabla_\nu U) \cdot V \, ds$, gives (4.2). □

With V vanishing on the boundary, (4.2) reads $\int \nabla U : \nabla V = \int (\nabla^* \nabla U) \cdot V$: $\nabla^* \nabla$ is the *adjoint* of ∇, hence the notation. Taking $V = U$ shows that it is *positive*: $\int (\nabla^* \nabla U) \cdot U \, dA = \int |\nabla U|^2 \, dA \geq 0$. Its negative, $\Delta U = -\nabla^* \nabla U$, is the *rough* or *Bochner* Laplacian of tangent fields; on scalar functions the same construction gives the familiar Laplace–Beltrami operator. The sign matters: the energy decreases along $-\nabla^* \nabla U$, which is the direction in which the heat equation must move.

4.2. The vector heat equation

The heat equation for tangent fields lets a field flow downhill in the Dirichlet energy:

$$\frac{\partial U}{\partial t} = -\nabla^* \nabla U = \Delta U. \tag{4.3}$$

By Green's identity, $\frac{d}{dt} \frac{1}{2} \int |U|^2 = -\int |\nabla U|^2 \leq 0$ (with $U = 0$ or $\nabla_\nu U = 0$ on the boundary): the flow smooths the field and decays it. Starting from a vector concentrated at a source point s, Sharp, Soliman and Crane [17] show that for short times the solution at a point x points in the direction of the source vector transported along the shortest geodesic from s to x, and that only its magnitude depends on the distance. Normalizing the solution at every point therefore gives the parallel transport of the source vector. (Their

method also diffuses the magnitude separately, with a scalar heat equation, to recover a field of prescribed length; here we need only the direction.)

4.3. Backward Euler

We take one time step of length Δt with the *backward* Euler method, which evaluates the right-hand side of (4.3) at the new time: with U^0 at time 0 and U at time Δt,

$$\frac{U - U^0}{\Delta t} = -\nabla^*\nabla U \qquad \Longleftrightarrow \qquad U + \Delta t\, \nabla^*\nabla U = U^0. \tag{4.4}$$

The operator $I + \Delta t\, \nabla^*\nabla$ is positive definite, because $\nabla^*\nabla$ is positive, so the step is always solvable and stable for any Δt, which is why the implicit method is used: a single large step can be taken. The vector heat method takes exactly one step, with Δt of the order of the square of the mesh spacing h, the time it takes heat to cross a few elements; the code uses $\Delta t = \bar{h}^2$ with $\bar{h}$ the mean edge length (`computeEdgeLengths`).

The boundary condition used by the code. The source can be imposed in two ways. Sharp et al. put it in the initial condition, $U^0 = \delta_s\, U_s$, a vector concentrated at the source vertex. The code instead prescribes the value at the source vertex and starts from zero everywhere else: it solves

$$U + \Delta t\, \nabla^*\nabla U = 0 \quad \text{away from } s, \qquad U(s) = U_s, \tag{4.5}$$

a *Dirichlet* condition at one point. Both produce a field that decays away from the source and whose direction is transported along the paths of steepest heat flow; the direction, which is all we keep, is the same to the accuracy of the method. Equation (4.5) is a *screened* Poisson equation for the connection Laplacian: the term U makes the influence of the source decay over a distance of order $\sqrt{\Delta t} \approx \bar{h}$.

5. The Weak Form

The equation (4.4) involves second derivatives of U, through $\nabla^*\nabla$, and a piecewise linear field on a mesh has no second derivatives: its first derivatives jump across edges. The weak form removes the second derivatives by moving one of them onto a test field.

Test fields. Multiply (4.4) by an arbitrary tangent field V, a *test field*, take the inner product, and integrate:

$$\int_\Omega U \cdot V\, dA + \Delta t \int_\Omega (\nabla^*\nabla U) \cdot V\, dA = \int_\Omega U^0 \cdot V\, dA.$$

If this holds for every test field V, the equation holds at every point (take V concentrated near any point). Conversely it suffices to ask it for every V in a large enough family; this is the idea that the finite element method will exploit with a finite family.

Integration by parts. By Green's identity (4.2), the second term equals $\int \nabla U : \nabla V$ minus a boundary term. Our surfaces are closed, or we choose test fields that vanish where U is prescribed, so the boundary term disappears, and the *weak form* of the backward Euler step is: find U such that

$$\boxed{\int_\Omega U \cdot V \, dA + \Delta t \int_\Omega \nabla U : \nabla V \, dA = \int_\Omega U^0 \cdot V \, dA} \tag{5.1}$$

for every test field V. Only first derivatives appear, of U and of V symmetrically. The equation now makes sense for fields whose derivatives are merely square integrable, including continuous, piecewise linear ones, which is the meaning of "weak". And it is the condition that the energy

$$J(U) = \frac{1}{2} \int_\Omega |U|^2 \, dA + \frac{\Delta t}{2} \int_\Omega |\nabla U|^2 \, dA - \int_\Omega U^0 \cdot U \, dA \tag{5.2}$$

be stationary, since its derivative in the direction V is the left-hand side of (5.1) minus its right-hand side. Being a sum of squares plus a linear term, J is strictly convex, and its unique minimizer is the solution: the backward Euler step looks for the field closest to U^0 that is also as parallel as possible, with Δt trading one against the other.

6. The Galerkin Method: Piecewise Linear Tangent Fields

The Galerkin method replaces "every test field" by a finite family, and the unknown field by a combination of the same family. We need, then, a finite dimensional space of tangent fields on the mesh. For scalar functions the standard choice is the piecewise linear functions; this section builds their analogue for tangent fields, and shows that the connection is what makes it possible.

6.1. The scalar hat functions

For each vertex i, the *hat function* ϕ_i is the continuous function on the mesh that is linear on every face, equals 1 at x_i and 0 at every other vertex. It is nonzero only on the faces around i (its *support*), it is a tent over the one-ring of i, and on each face f the three hat functions of its vertices add up to 1, since $\phi_{f_1} + \phi_{f_2} + \phi_{f_3}$ is linear and equals 1 at the three vertices. The scalar piecewise linear functions are the combinations $\sum_i c_i \phi_i$, and c_i is the value at vertex i.

6.2. Why the vector case needs a connection

The obvious vector analogue would be $U = \sum_i \phi_i \, (T_i u_i)$: interpolate the three vertex vectors of each face with the hat functions. But $T_i u_i$ lies in the tangent plane of vertex i, and the three vertices of a face have three different tangent planes: the interpolated vector would not be tangent to the face, and its derivative would contain the bending of the surface rather than the turning of the field. Before interpolating, the three vectors must be brought into one plane, the plane of the face, and the rule that brings a vector from

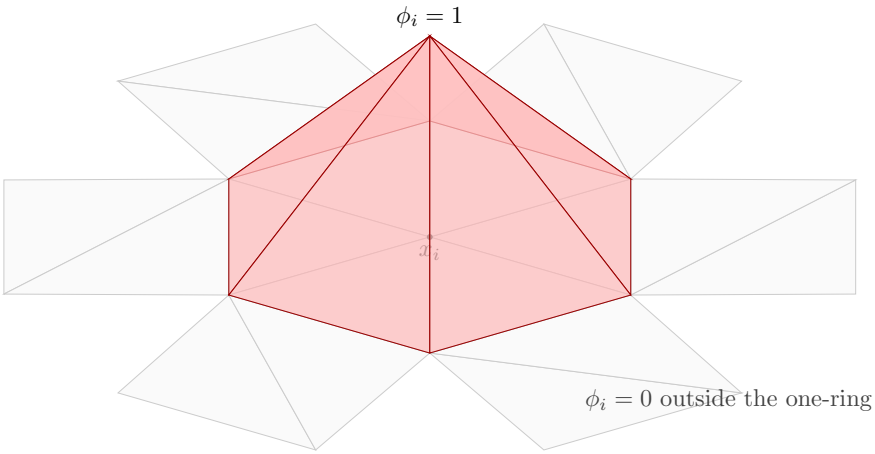


FIGURE 7. The hat function ϕ_i of a vertex: linear on every face, 1 at x_i, 0 at all other vertices, and therefore zero outside the one-ring of x_i.

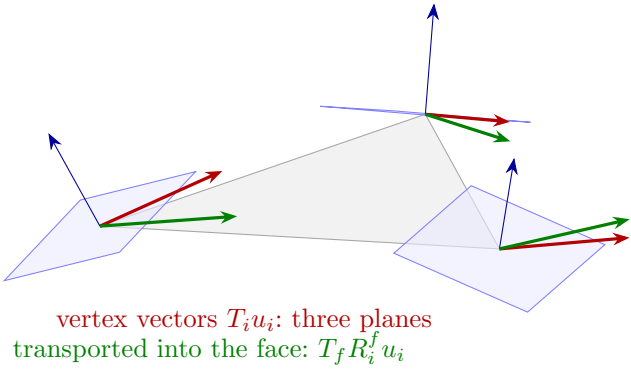


FIGURE 8. Why vectors must be transported before they are interpolated. The vertex vectors $T_i u_i$ (red) lie in three different vertex planes (blue), none of which is the plane of the face; transported into the face, $T_f R_i^f u_i$ (green), they lie in one plane and can be combined.

one tangent plane into another is parallel transport. So, on each face f, we interpolate the *transported* vertex vectors:

$$U\big|_f = \sum_{i \in f} \phi_i \, T_f R_i^f u_i, \qquad \text{in face coordinates:} \quad T_f^T U\big|_f = \sum_{i \in f} \phi_i \, R_i^f u_i. \tag{6.1}$$

Figure 8 shows the difference. This defines a tangent field on every face, linear in the unknowns u_i. The space of such fields is spanned by the $2n$ *vector hat*

functions

$$\Psi_{i,k}\big|_f = \phi_i \, T_f R_i^f \varepsilon_k, \qquad k = 1, 2, \tag{6.2}$$

where $\varepsilon_1 = (1, 0)$ and $\varepsilon_2 = (0, 1)$: the frame vector k of vertex i, transported into each incident face, and faded out by the scalar hat. Then $U = \sum_{i,k} u_{i,k} \Psi_{i,k}$, with the unknowns (2.5) as coefficients. At vertex i itself, $\phi_i = 1$ and the other hats vanish, so the field takes, in each incident face, the transported value $T_f R_i^f u_i$: the vertex vector seen from that face.

6.3. The covariant derivative of a basis function

Inside a face, the surface is a plane and the face frame T_f is constant, so the connection form vanishes, and by (3.3) the covariant derivative is the ordinary derivative of the face coordinates. In (6.1) the transported vectors $R_i^f u_i$ are constant on the face; only the hat functions vary. Hence, for a tangent direction X of the face,

$$T_f^T \nabla_X U\big|_f = \sum_{i \in f} (X \cdot \nabla \phi_i) \, R_i^f u_i, \qquad T_f^T \nabla U\big|_f = \sum_{i \in f} (R_i^f u_i) \, (g_i^f)^T, \tag{6.3}$$

where $g_i^f = T_f^T \nabla \phi_i \in \mathbb{R}^2$ is the gradient of the hat function in the coordinates of the face, a constant vector on the face, computed in Section 7. The total covariant derivative is a sum of rank-one 2×2 matrices: the transported vertex vector times the gradient of its hat. In tensor notation, $\nabla \Psi_{i,k} = (R_i^f \varepsilon_k) \otimes g_i^f$.

6.4. The discrete equations

Now take $U = \sum_{j,l} u_{j,l} \Psi_{j,l}$ and, as test fields, every basis function $V = \Psi_{i,k}$ (the *Galerkin* choice: test and trial spaces are the same). The weak form (5.1) becomes $2n$ linear equations,

$$\sum_{j,l} \Big(\int_\Omega \Psi_{j,l} \cdot \Psi_{i,k} \, dA + \Delta t \int_\Omega \nabla \Psi_{j,l} : \nabla \Psi_{i,k} \, dA \Big) \, u_{j,l} = \int_\Omega U^0 \cdot \Psi_{i,k} \, dA, \tag{6.4}$$

which, in matrix form, read

$$(\mathrm{M} + \Delta t \, \mathrm{L}) \, \mathrm{u} = \mathrm{M} \, \mathrm{u}^0, \tag{6.5}$$

with the *mass matrix* M and the *stiffness matrix* L, both $2n \times 2n$ and both made of 2×2 blocks, one per pair of vertices (i, j):

$$\mathrm{M}_{ij} = \Big[\int_\Omega \Psi_{j,l} \cdot \Psi_{i,k} \, dA \Big]_{k,l}, \qquad \mathrm{L}_{ij} = \Big[\int_\Omega \nabla \Psi_{j,l} : \nabla \Psi_{i,k} \, dA \Big]_{k,l}. \tag{6.6}$$

(For the right-hand side we have written $U^0 = \sum_j \Psi_{j,l} \, u^0_{j,l}$, its interpolant.) Each integral is a sum over faces, and a face contributes only if it contains both i and j, because the hats vanish elsewhere. So the matrices are sparse, with a nonzero block for each vertex and for each pair of vertices joined by an edge, and they can be computed face by face, which is the next step.

The stiffness block of a face. On a face f containing i and j, by (6.3) and the identity $(ab^T) : (cd^T) = (a \cdot c)(b \cdot d)$,

$$\nabla\Psi_{j,l} : \nabla\Psi_{i,k} = \big((R_j^f \varepsilon_l)(g_j^f)^T\big) : \big((R_i^f \varepsilon_k)(g_i^f)^T\big) = (g_i^f \cdot g_j^f)\,(R_i^f \varepsilon_k) \cdot (R_j^f \varepsilon_l),$$

and $(R_i^f \varepsilon_k) \cdot (R_j^f \varepsilon_l) = \varepsilon_k^T (R_i^f)^T R_j^f \varepsilon_l$ is the entry (k,l) of $(R_i^f)^T R_j^f$. The integrand is constant on the face, so its integral is the area times the constant:

$$\boxed{\mathrm{L}_{ij}^f = A_f\,(g_i^f \cdot g_j^f)\,(R_i^f)^T R_j^f} \tag{6.7}$$

This is the heart of the method, and it is worth reading slowly. The scalar factor $A_f\, g_i^f \cdot g_j^f$ is the entry (i,j) of the stiffness matrix of the ordinary scalar Laplacian; the 2×2 factor $(R_i^f)^T R_j^f$ is, by (3.6), the parallel transport from vertex j to vertex i through the face. The connection Laplacian is the scalar Laplacian in which every coupling between two vertices first transports the vector of one into the frame of the other. For $i = j$ the transport is the identity, $(R_i^f)^T R_i^f = I$, and the diagonal block is the scalar diagonal entry times I. On a flat mesh with a common frame, every $R_i^f = I$, and the connection Laplacian is two copies of the scalar Laplacian, one per coordinate.

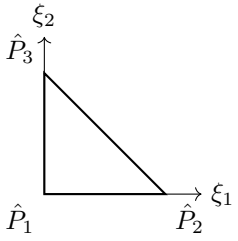
The block for (j,i) is $A_f(g_j^f \cdot g_i^f)(R_j^f)^T R_i^f = (\mathrm{L}_{ij}^f)^T$, so the matrix is symmetric. It is also positive semi-definite, because $\mathrm{u}^T \mathrm{L}\mathrm{u} = \int |\nabla U|^2\, dA \geq 0$ for the field U of coefficients u.

7. Shape Functions and Their Gradients on a Triangle in Space

The element matrices need the constant gradients g_i^f of the three hat functions on each face. The standard way to compute them is to do the work once, on a fixed *reference triangle*, and to carry the result to every face of the mesh with an affine map [15].

7.1. The reference triangle

The reference triangle $\hat{K}$ has the vertices $\hat{P}_1 = (0,0)$, $\hat{P}_2 = (1,0)$, $\hat{P}_3 = (0,1)$ in the plane of the coordinates $\xi = (\xi_1, \xi_2)$:



Its three linear *shape functions*, each equal to 1 at one vertex and 0 at the other two, are

$$\hat{\phi}_1(\xi) = 1 - \xi_1 - \xi_2, \qquad \hat{\phi}_2(\xi) = \xi_1, \qquad \hat{\phi}_3(\xi) = \xi_2, \tag{7.1}$$

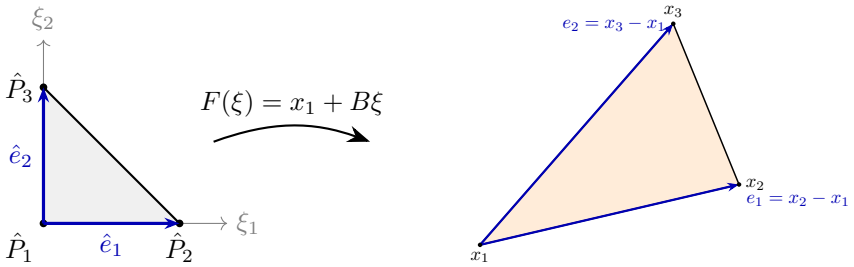


FIGURE 9. The affine map from the reference triangle to a face in space. The columns of B are the images of the reference edges.

as one checks at the three vertices: $\hat{\phi}_a(\hat{P}_b) = \delta_{ab}$. Their gradients are constant:

$$\hat{\nabla}\hat{\phi}_1 = (-1,-1), \qquad \hat{\nabla}\hat{\phi}_2 = (1,0), \qquad \hat{\nabla}\hat{\phi}_3 = (0,1). \tag{7.2}$$

7.2. The map to a face in space

The affine map that takes the reference triangle onto a face f with vertices x_1, x_2, x_3 (local numbering, in the order of the face) is

$$F(\xi) = x_1 + B\,\xi, \qquad B = \begin{bmatrix} x_2 - x_1 & x_3 - x_1 \end{bmatrix} \in \mathbb{R}^{3\times 2}. \tag{7.3}$$

It takes $\hat{P}_1, \hat{P}_2, \hat{P}_3$ to x_1, x_2, x_3, and the edges $\hat{e}_1, \hat{e}_2$ of the reference triangle to the edges $e_1 = x_2 - x_1$ and $e_2 = x_3 - x_1$ of the face, the columns of B (Figure 9).

The hat functions on the face are the reference shape functions composed with the inverse of the map: $\phi_a = \hat{\phi}_a \circ F^{-1}$. In the plane we would invert B, but here B is 3×2: it maps the plane of ξ onto the plane of the face, which is a plane in space, and has no inverse. What we need is the gradient of ϕ_a, a vector in the plane of the face, and the chain rule determines it.

Proposition 7.1. *On the face, $\nabla\phi_a = B(B^TB)^{-1}\,\hat{\nabla}\hat{\phi}_a$, and this vector lies in the plane of the face.*

Proof. Let $g = \nabla\phi_a$ be the gradient, a vector in the plane of the face. Its defining property is that, along any direction X of the face, the derivative of ϕ_a is $g\cdot X$. The edges $B\hat{e}_1 = x_2 - x_1$ and $B\hat{e}_2 = x_3 - x_1$ are two such directions, and along them $\phi_a(F(\xi)) = \hat{\phi}_a(\xi)$ changes at the rates $\partial\hat{\phi}_a/\partial\xi_m$: so $B^Tg = \hat{\nabla}\hat{\phi}_a$, two equations. Since g lies in the plane spanned by the columns of B, $g = Bc$ for some $c \in \mathbb{R}^2$, and then $B^TBc = \hat{\nabla}\hat{\phi}_a$, so $c = (B^TB)^{-1}\hat{\nabla}\hat{\phi}_a$, and $g = B(B^TB)^{-1}\hat{\nabla}\hat{\phi}_a$. The matrix B^TB is the Gram matrix of the two edges, invertible for a non-degenerate triangle. □

The matrix $B(B^TB)^{-1}$ is the transpose of the Moore–Penrose pseudo-inverse $B^+ = (B^TB)^{-1}B^T$ of B, and for a triangle in the plane, where B is square, it reduces to B^{-T}, the familiar formula of the planar finite element

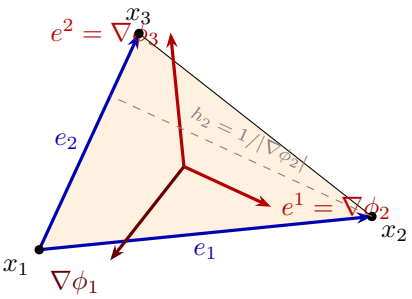


FIGURE 10. The gradients of the hat functions of a face. $\nabla\phi_2$ is orthogonal to the edge opposite to x_2, points towards x_2, and has length $1/h_2$, the reciprocal of the height; the gradients $\nabla\phi_2 = e^1$ and $\nabla\phi_3 = e^2$ form the *reciprocal frame* of the edges e_1, e_2 ($e^k \cdot e_m = \delta_{km}$), and the three gradients add up to zero.

method. In the code, `Binv` $= ((B^TB)^{-1}B^T)^T = B(B^TB)^{-1}$, and the three gradients are its columns combined as in (7.2):

$$\nabla\phi_1 = -C_{:,1} - C_{:,2}, \qquad \nabla\phi_2 = C_{:,1}, \qquad \nabla\phi_3 = C_{:,2}, \qquad C = \texttt{Binv}, \quad (7.4)$$

which is `gradN[0]`, `gradN[1]`, `gradN[2]` in `AssembleStiffnessElement-Embedded2` (and in `Gradient`). Note that the three gradients add up to zero, as they must, since the three hat functions add up to 1. Finally, the coordinates of the gradient in the face frame are

$$g_a^f = T_f^T \nabla\phi_a \in \mathbb{R}^2, \qquad (7.5)$$

`grad_i = Tf.transpose() * gradN[i]`. Since $\nabla\phi_a$ lies in the face plane and the frame is orthonormal, $g_a^f \cdot g_b^f = \nabla\phi_a \cdot \nabla\phi_b$: the projection does not change the stiffness, it only expresses the gradients in the same coordinates as the transported vectors.

Worked example 7.2. *Take the triangle* $x_1 = (0,0,0)$, $x_2 = (2,0,0)$, $x_3 = (0,1,1)$ *of Figure 11. Its edges are* $e_1 = (2,0,0)$ *and* $e_2 = (0,1,1)$, *so*

$$B = \begin{pmatrix} 2 & 0 \\ 0 & 1 \\ 0 & 1 \end{pmatrix}, \quad B^TB = \begin{pmatrix} 4 & 0 \\ 0 & 2 \end{pmatrix}, \quad B(B^TB)^{-1} = \begin{pmatrix} 1/2 & 0 \\ 0 & 1/2 \\ 0 & 1/2 \end{pmatrix}.$$

The edges are orthogonal, so B^TB *is diagonal and the reciprocal frame is just* $e_k/|e_k|^2$. *By (7.4),*

$$\nabla\phi_1 = \left(-\tfrac{1}{2}, -\tfrac{1}{2}, -\tfrac{1}{2}\right), \qquad \nabla\phi_2 = \left(\tfrac{1}{2}, 0, 0\right), \qquad \nabla\phi_3 = \left(0, \tfrac{1}{2}, \tfrac{1}{2}\right),$$

which add up to zero. Check $\nabla\phi_2$: *along* e_1, ϕ_2 *goes from* 0 *to* 1, *and* $\nabla\phi_2 \cdot e_1 = 1$; *along* e_2 *it stays* 0, *and* $\nabla\phi_2 \cdot e_2 = 0$. *The area is* $A_f = \frac{1}{2}|e_1 \times e_2| =$

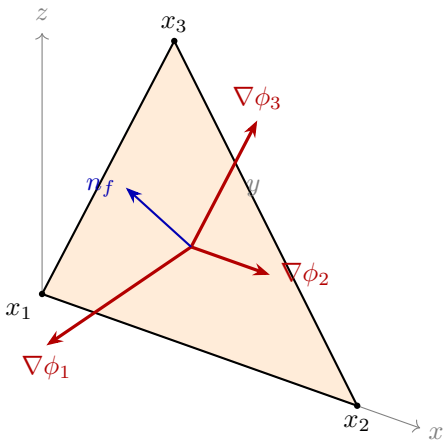


FIGURE 11. The triangle of Example 7.2, its normal $n_f = (0,-1,1)/\sqrt{2}$, and the gradients of its three hat functions, drawn from the centroid. All three lie in the plane of the face.

$\frac{1}{2}|(0,-2,2)| = \sqrt{2}$, *and the scalar stiffness* $A_f\,\nabla\phi_a \cdot \nabla\phi_b$ *of the face is*

$$\sqrt{2}\begin{pmatrix} 3/4 & -1/4 & -1/2 \\ -1/4 & 1/4 & 0 \\ -1/2 & 0 & 1/2 \end{pmatrix} = \begin{pmatrix} 1.0607 & -0.3536 & -0.7071 \\ -0.3536 & 0.3536 & 0 \\ -0.7071 & 0 & 0.7071 \end{pmatrix},$$

whose rows add up to zero, since constants have zero gradient. The entry $(2,3)$ *vanishes because the angle at* x_1 *is a right angle, in agreement with the cotangent formula of the next section; the entry* $(1,2)$ *is* $-\frac{1}{2}\cot\theta_3$, *where* $\cos\theta_3 = \frac{(x_1-x_3)\cdot(x_2-x_3)}{|x_1-x_3|\,|x_2-x_3|} = \frac{2}{\sqrt{2}\sqrt{6}} = \frac{1}{\sqrt{3}}$, *so* $\cot\theta_3 = \frac{1}{\sqrt{2}}$ *and* $-\frac{1}{2}\cot\theta_3 = -0.3536$.

7.3. A check: the cotangent formula

The scalar factor of (6.7) has a classical closed form, which connects the finite element computation with the cotangent Laplacian of geometry processing (`CalcCotangentWeights` in `Laplacian.cpp`).

Proposition 7.3. *For two distinct vertices* a, b *of a face with area* A_f, $A_f\,\nabla\phi_a \cdot \nabla\phi_b = -\frac{1}{2}\cot\theta_c$, *where* θ_c *is the angle at the third vertex* c; *and* $A_f\,|\nabla\phi_a|^2 = \frac{1}{2}(\cot\theta_b + \cot\theta_c)$.

Proof. $\nabla\phi_a$ is orthogonal to the edge opposite to a (along which ϕ_a is constant, zero), points towards a, and has length $1/h_a$, where $h_a = 2A_f/\ell_a$ is the height from a and ℓ_a the length of the opposite edge. So $\nabla\phi_a = \frac{\ell_a}{2A_f} m_a$ with m_a the unit inward normal of the opposite edge, and $\nabla\phi_a \cdot \nabla\phi_b = \frac{\ell_a \ell_b}{4A_f^2}\, m_a \cdot m_b$. The inward normals of two edges meeting at c make the angle $\pi - \theta_c$, so

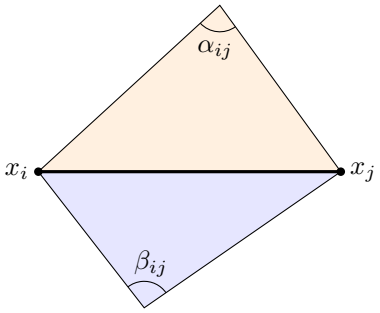


FIGURE 12. The two angles opposite to an edge ij in its two faces. The scalar part of the stiffness block of the edge is the cotangent weight; the connection Laplacian multiplies it by the transport from j to i.

$m_a \cdot m_b = -\cos\theta_c$, and with $2A_f = \ell_a \ell_b \sin\theta_c$ we get $A_f\, \nabla\phi_a \cdot \nabla\phi_b = -\frac{\ell_a \ell_b \cos\theta_c}{4A_f} = -\frac{1}{2}\cot\theta_c$. The diagonal follows from $\nabla\phi_a = -\nabla\phi_b - \nabla\phi_c$. □

Summing over the two faces of an edge ij, the off-diagonal stiffness is $-\frac{1}{2}(\cot\alpha_{ij} + \cot\beta_{ij})$, the cotangent weight of the edge. (Figure 12.) The connection Laplacian (6.7) is therefore the cotangent Laplacian with a 2×2 transport on every edge, as in the discrete connection Laplacians of geometry processing [10, 17]; the finite element derivation explains where both factors come from.

8. The Element Matrices

8.1. The element stiffness matrix

On one face, the hat functions of its three vertices are the only nonzero ones, so the face contributes to the 3×3 blocks of L indexed by its vertices, a 6×6 *element matrix* in the local numbering $a, b \in \{1, 2, 3\}$:

$$\mathrm{L}^f = \begin{pmatrix} \mathrm{L}^f_{11} & \mathrm{L}^f_{12} & \mathrm{L}^f_{13} \\ \mathrm{L}^f_{21} & \mathrm{L}^f_{22} & \mathrm{L}^f_{23} \\ \mathrm{L}^f_{31} & \mathrm{L}^f_{32} & \mathrm{L}^f_{33} \end{pmatrix}, \qquad \mathrm{L}^f_{ab} = A_f\,(g_a^f \cdot g_b^f)\,(R_a^f)^T R_b^f \in \mathbb{R}^{2\times 2}. \quad (8.1)$$

This is `AssembleStiffnessElementEmbedded2`: it computes B, `Binv`, A_f (`faceArea`), the three gradients, their face coordinates g_a^f and the transports R_a^f (`Rvf`), and fills the blocks with `elementMatrix.block<2,2>(a*2, b*2)`. Following the symmetry $\mathrm{L}^f_{ba} = (\mathrm{L}^f_{ab})^T$, it loops over $a \geq b$ only and writes both blocks; on the diagonal it writes $A_f |g_a^f|^2 I_2$.

Exactness of the integration. The integrand of (6.7) is constant on the face, so "area times integrand" is the exact integral, with no quadrature error. For higher order elements the gradients would vary and a quadrature rule would be needed; with linear elements it is not.

8.2. The element mass matrix and its lumping

The mass matrix needs $\int_f \Psi_{b,l} \cdot \Psi_{a,k}\, dA = \big(\int_f \phi_a \phi_b\, dA\big)\, \big((R_a^f)^T R_b^f\big)_{kl}$, by the same computation as for the stiffness but without derivatives. The integrals of products of linear hats over a triangle are $\int_f \phi_a \phi_b\, dA = \frac{A_f}{12}(1+\delta_{ab})$, so the *consistent* element mass matrix is

$$\mathrm{M}_{ab}^f = \frac{A_f}{12}(1+\delta_{ab})\,(R_a^f)^T R_b^f. \tag{8.2}$$

The code uses the simpler *lumped* mass matrix, which moves all the mass of each row to the diagonal. Since the three hats add up to 1 on the face, the row sums of the scalar factor are $\sum_b \int_f \phi_a \phi_b = \int_f \phi_a = \frac{A_f}{3}$, and the lumped element mass matrix is

$$\mathrm{M}_{aa}^f = \frac{A_f}{3}\, I_2, \qquad \mathrm{M}_{ab}^f = 0 \quad (a \neq b), \tag{8.3}$$

where the transport has disappeared because $(R_a^f)^T R_a^f = I$. Assembled, the mass of vertex i is one third of the area of its incident faces, the same for both coordinates. This is `AssembleDiagonalMassMatrix`. Lumping changes the discrete solution by an amount of the order of the discretization error, keeps M diagonal and positive, and makes M^{-1} trivial, which the code uses next.

9. Assembly

From local to global indices. The global matrices are sums of the element matrices, each placed at the rows and columns of the vertices of its face:

$$\mathrm{L} = \sum_f P_f^T\, \mathrm{L}^f\, P_f, \qquad \mathrm{M} = \sum_f P_f^T\, \mathrm{M}^f\, P_f, \tag{9.1}$$

where P_f is the $6 \times 2n$ matrix that extracts, from the global vector u, the six coordinates of the vertices of f in local order. In code, nobody forms P_f: the local block (a, b) of L^f, with entries (k, l), is added to the global entry

$$\big(2\,\mathrm{ID}(f_a) + k,\ 2\,\mathrm{ID}(f_b) + l\big), \quad k, l \in \{0, 1\}, \tag{9.2}$$

with the zero-based vertex indices ID of the mesh: `(*A)(v[i]->ID * 2 + row, v[j]->ID * 2 + col) += wij(row, col)`. The function `AssembleMatrix2` loops over the faces, computes L^f, and adds $\Delta t\, \mathrm{L}^f$ to the global matrix, so that it returns $\Delta t\, \mathrm{L}$; `AssembleDiagonalMassMatrix` does the same with (8.3). Each interior edge gets contributions from its two faces, and each vertex from all its incident faces: the sums (9.1) are exactly the sums over faces of the integrals in (6.6). Figure 13 shows one face being added.

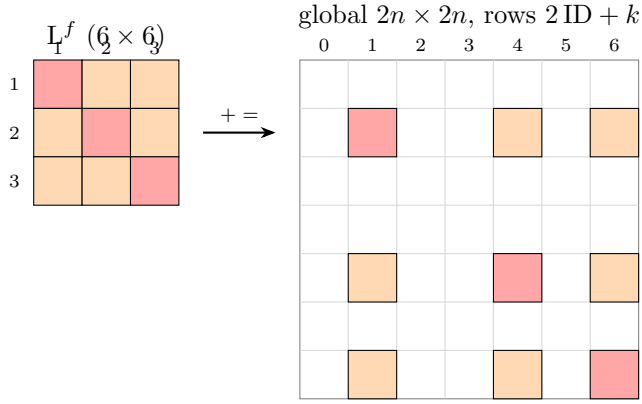


FIGURE 13. Assembly. The 6×6 element matrix of a face, made of 3×3 blocks of size 2×2 (diagonal blocks red), is added into the global matrix at the rows and columns of its vertices, here 1, 4 and 6.

Structure. The assembled L has a 2×2 block for each vertex (the diagonal) and for each edge, in both orders, with $\mathrm{L}_{ji} = \mathrm{L}_{ij}^T$. It is symmetric and positive semi-definite. It is singular on closed surfaces only if a parallel field exists, which on a curved closed surface is never the case, so in practice $\mathrm{M} + \Delta t\,\mathrm{L}$ is positive definite even before the boundary condition.

10. The Linear System and Its Solution

10.1. The system the code solves

The discrete backward Euler step is (6.5), $(\mathrm{M} + \Delta t\,\mathrm{L})\,\mathrm{u} = \mathrm{M}\,\mathrm{u}^0$. Multiplying by the diagonal M^{-1} on the left, which does not change the solution,

$$(I + \Delta t\,\mathrm{M}^{-1}\mathrm{L})\,\mathrm{u} = \mathrm{u}^0. \tag{10.1}$$

This is the form the code builds, in place: `IplusMinvTimesA` divides each row of $\Delta t\,\mathrm{L}$ by the diagonal mass of that row and adds 1 to the diagonal. The scaled matrix is no longer symmetric (its rows are scaled by different masses), which is one reason why the code factors it with a general sparse LU. The symmetric form (6.5) could be factored with a Cholesky factorization instead, at half the cost; the two have the same solution.

10.2. The Dirichlet condition

The source vector is prescribed at one vertex s: $u_s = \bar{u}_s$, given as the unit complex number $e^{2\pi \mathrm{i}/3}$, that is, the unit tangent vector at 120° from the first frame vector t_s. The two coordinates of s are then not unknowns, and their two equations of (10.1) are replaced by the prescribed values: in the matrix, the rows $2s$ and $2s + 1$ become rows of the identity, and in the right-hand side, the entries become $\bar{u}_s$. This *row replacement* is `PreFactor`,

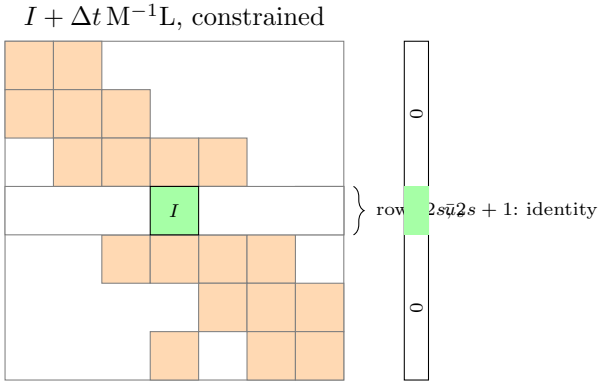


FIGURE 14. The Dirichlet condition by row replacement: the two rows of the source vertex s become rows of the identity, and the right-hand side holds the prescribed vector $\bar{u}_s$ there and zero elsewhere.

which copies every unconstrained row and writes 1 on the diagonal of each constrained row before the LU factorization (`solver.compute`). Every other entry of the right-hand side is zero, $\mathrm{u}^0 = 0$ away from the source, which is the problem (4.5).

Remark 10.1 (Which vertices are constrained). *The loop in* `main` *that chooses the source scans the vertices and stops (*`break`*) at the first one that is either near the center of the model or above the height* 0.83*. Exactly one vertex is constrained.*

Remark 10.2 (Symmetric elimination). *Row replacement makes the matrix non-symmetric even if it was symmetric. The symmetric alternative eliminates the constrained unknowns: move their columns, multiplied by the prescribed values, to the right-hand side, and keep only the rows and columns of the free unknowns. For (6.5), the reduced matrix is a principal submatrix of* $\mathrm{M} + \Delta t\, \mathrm{L}$*, hence symmetric positive definite.*

10.3. From the solution to vectors

The solve (`solver.solve(right_hand_side)`) returns u, two coordinates per vertex. Its magnitudes decay away from the source; its directions are the transported source vector. So the code keeps only the direction: reading u_i as the complex number $u_{i,1} + \mathrm{i}u_{i,2}$, it normalizes it, $\hat{u}_i = u_i/|u_i|$, takes its angle $\alpha_i = \arg \hat{u}_i$ (`log(R_i).imag()`), and rotates the first frame vector t_i by α_i about the vertex normal n_i, which is the tangent vector $T_i\hat{u}_i = \cos\alpha_i\, t_i + \sin\alpha_i\, b_i$. That vector is drawn as an arrow at the vertex (`display`).

10.4. What the program computes

Figure 15 shows the result on the cactus that `FEM.cpp` loads: one vector prescribed at one vertex has been spread over the whole surface, following its

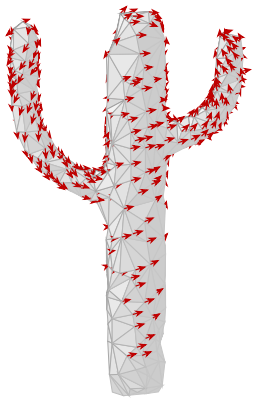


FIGURE 15. The output of the method on the cactus of `FEM.cpp` (502 vertices): the normalized solution at every visible vertex, drawn as a tangent vector.

arms and trunk. To see that the directions are right we need a surface where parallel transport is known exactly.

Worked example 10.3. *On the unit sphere, the geodesics from a point s are the great circles through it, and transporting a vector along the great circle from s to x is the smallest rotation taking s to x, applied to the vector: the rolling of Section 14, with the normal turning from s to x in their common plane. The program* `ga_fem` *solves the problem of this text on a geodesic sphere of* 2,562 *vertices (an icosahedron subdivided four times, mean edge length* 0.076*), with the source at its top vertex, and compares the direction of the result at every vertex with that exact transport (Figure 16). Out to three quarters of the way to the antipode, the angle between the two is* 0.07° *on average and* 0.30° *at most. Near the antipode the error grows without bound, and it must: every great circle from s reaches the antipode, each with a different transported vector, so there is no shortest path and no correct answer there. The magnitude $|u_i|$ decays by twenty orders of magnitude from the source to the antipode, the screening of (4.5) over a distance of order $\sqrt{\Delta t} = \bar{h}$, and yet the direction, which is all we keep, is accurate wherever it is defined.*

11. Summary: From the Equations to the Code

Algorithm 1 lists the steps of the program, and Table 1 the formula behind every function. Read from top to bottom, the table is the derivation of this text in the order the code executes it: the geometry first, then the connection, then the element matrices, their assembly, the boundary condition, and the solve.

Three ideas carry the whole construction. The weak form trades the second derivatives of the heat equation for first derivatives of a trial field

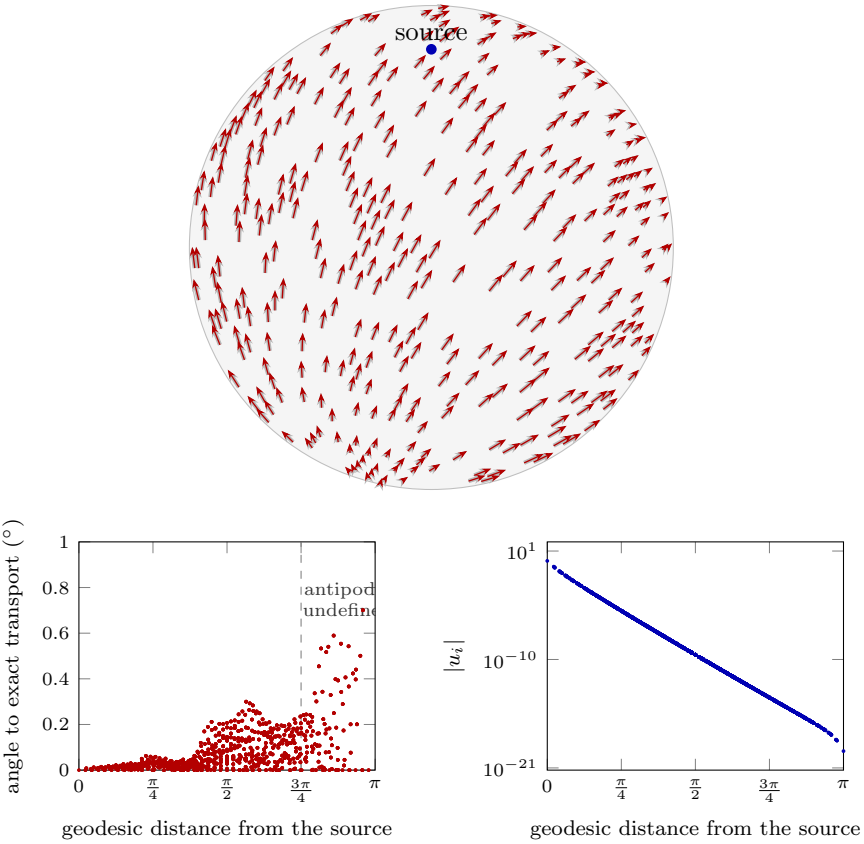


FIGURE 16. Validation on the sphere (Example 10.3). Top: the computed directions (thin red) over the exact parallel transport along great circles from the source (thick gray), on the visible half of the sphere. Bottom: the angle between the two at every vertex against the geodesic distance from the source (beyond the dashed line, three quarters of the way, the errors grow towards the antipode, where transport is not defined, and are not shown), and the magnitude of the solution, on a logarithmic scale.

and a test field, so that piecewise linear fields can be used. Piecewise linear *tangent* fields exist only once the vectors of the three vertices of a face are brought into the plane of the face, and that is done by the discrete Levi-Civita connection, a 2×2 rotation per vertex and face. And because a face is flat, the covariant derivative inside it is an ordinary derivative, so that the stiffness of the connection Laplacian is the stiffness of the scalar Laplacian, multiplied on every edge by the transport between its two vertices.

Algorithm 1 Parallel transport of a tangent vector by one step of the vector heat equation (`main` of `FEM.cpp`).

1: Vertex normals (2.3), vertex frames T_i (2.4), face normals, areas and frames T_f (2.2)
2: $\Delta t \leftarrow \bar{h}^2$, the square of the mean edge length
3: choose the source vertex s and its vector $\bar{u}_s$; $\mathrm{b} \leftarrow 0$, $\mathrm{b}_s \leftarrow \bar{u}_s$
4: **for** every face f **do**
5: $\quad B \leftarrow [x_2 - x_1,\ x_3 - x_1]$, $\quad \nabla\phi_a \leftarrow B(B^TB)^{-1}\hat{\nabla}\hat{\phi}_a$, $\quad g_a^f \leftarrow T_f^T \nabla\phi_a$
6: $\quad R_a^f \leftarrow T_f^T\, Q_{a\to f}\, T_a$ (Rodrigues rotation $n_a \to n_f$)
7: $\quad$ add $\Delta t\, A_f\, (g_a^f \cdot g_b^f)\, (R_a^f)^T R_b^f$ to the block (a,b) of $\Delta t\,\mathrm{L}$, $\quad \frac{A_f}{3} I$ to the block (a,a) of M
8: **end for**
9: $\mathrm{A} \leftarrow I + \mathrm{M}^{-1}(\Delta t\,\mathrm{L})$; replace the rows of s by identity rows
10: solve $\mathrm{A\,u} = \mathrm{b}$ by sparse LU
11: for every vertex i: draw $T_i\, u_i/|u_i|$

TABLE 1. Every formula and the function of `FEM.cpp` that evaluates it.

Quantity	Equation	Function
vertex normal n_i	(2.3)	`Mesh::computeNormals`
vertex frame T_i	(2.4)	`computeVertexTangentSpace`, `project`
face normal n_f, area A_f	(2.2)	`computeFaceNormals`
face frame T_f	Section 2	`computeFaceTangentSpace`
smallest rotation $Q_{i\to f}$	(3.4)	`alignVectors`, `bracket`
discrete connection R_i^f	(3.5)	`Rvf`
vertex-to-vertex transport (variant)	(3.6)	`Rvv`
edge matrix B, $B(B^TB)^{-1}$	(7.3), Prop. 7.1	`B`, `Binv`
gradients $\nabla\phi_a$, g_a^f	(7.4), (7.5)	`gradN`, `grad_i`
element stiffness L^f	(8.1)	`AssembleStiffnessElement-Embedded2`
lumped mass M	(8.3)	`AssembleDiagonalMassMatrix`
assembly of $\Delta t\,\mathrm{L}$	(9.1), (9.2)	`AssembleMatrix2`
time step $\Delta t = \bar{h}^2$	Section 4	`computeEdgeLengths`
$I + \Delta t\,\mathrm{M}^{-1}\mathrm{L}$	(10.1)	`IplusMinvTimesA`
Dirichlet rows, LU factorization	Section 10	`PreFactor`
normalization and drawing	Section 10	`display`

Part II. The Same Derivation in Geometric Algebra, and What Is Intrinsic

Part I worked with matrices: frames as 3×2 arrays, transports as 2×2 rotations, coordinates as pairs of numbers. Every one of those objects is a geometric object in disguise: a plane, a rotation, a direction in a plane. This part repeats the derivation with the objects themselves, in the geometric

algebra $\mathbb{G}_3$ of space [6,9], and uses the clearer language to answer a question Part I left open: which parts of the computation are *intrinsic*, that is, depend only on the curved surface as its inhabitants experience it, and which depend on how the surface sits in space.

12. The Tangent Plane as a Bivector

12.1. Geometric algebra in brief

In $\mathbb{G}_3$, vectors are multiplied with the geometric product $ab = a \cdot b + a \wedge b$, the sum of the inner product, a scalar, and the outer product, a *bivector*: an oriented plane element, with the orientation of the turn from a to b and magnitude the area of the parallelogram they span. Orthogonal vectors anticommute, parallel ones commute, and $aa = |a|^2$. The unit pseudoscalar $I = e_1e_2e_3$ squares to -1, commutes with everything, and turns a vector into the bivector of its orthogonal plane: $a \wedge b = I(a \times b)$. The reverse $\widetilde{X}$ of a product reverses the order of its vector factors. A *rotor* is an even element R with $R\widetilde{R} = 1$; it rotates any multivector by the sandwich $X \mapsto RX\widetilde{R}$, and the rotor that turns by the angle θ in the plane of the unit bivector B (in the sense of B) is $R = e^{-B\theta/2} = \cos\frac{\theta}{2} - B\sin\frac{\theta}{2}$.

12.2. The tangent bivector

At each point of the surface, instead of the normal n, take its dual, the unit bivector

$$\mathsf{T} = In, \tag{12.1}$$

the *tangent bivector*: the tangent plane itself, as an oriented plane element. Its square is $\mathsf{T}^2 = InIn = I^2n^2 = -1$. A vector a is tangent exactly when $a \wedge \mathsf{T} = 0$, and its projection onto the tangent plane and its rejection from it are

$$P(a) = (a\cdot\mathsf{T})\,\mathsf{T}^{-1} = -(a\cdot\mathsf{T})\,\mathsf{T}, \qquad a - P(a) = (a\wedge\mathsf{T})\,\mathsf{T}^{-1} = (a\cdot n)\,n, \tag{12.2}$$

which is the matrix $TT^T = I - nn^T$ of (2.1), now written without a choice of frame.

12.3. The tangent plane is the complex plane

The bivector T acts on tangent vectors by right multiplication. For a tangent vector a, $a \cdot n = 0$, so $an = a \wedge n$ and $a\mathsf{T} = aIn = I(an) = I(a \wedge n) = I^2(a \times n) = n \times a$:

$$a\,\mathsf{T} = n \times a, \tag{12.3}$$

the vector a turned by a quarter turn, counterclockwise about n, within the tangent plane. Applying it twice gives $a\mathsf{T}^2 = -a$. So the even elements $\alpha + \beta\mathsf{T}$ of the plane behave exactly like complex numbers, with T as the imaginary unit, and they act on tangent vectors by right multiplication:

$$a\,e^{\mathsf{T}\theta} = a\cos\theta + (n \times a)\sin\theta, \tag{12.4}$$

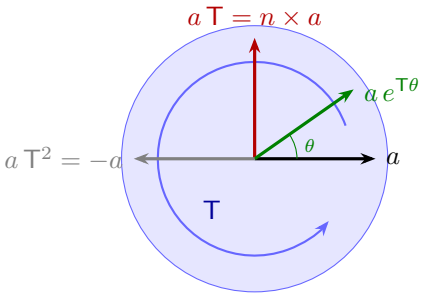


FIGURE 17. The tangent bivector $\mathsf{T} = In$ as an oriented disk in the tangent plane. Right multiplication by T turns a tangent vector a by a quarter turn in the sense of T, twice gives $-a$, and $e^{\mathsf{T}\theta} = \cos\theta + \mathsf{T}\sin\theta$ turns it by θ: the even elements of the plane are the complex numbers, with T as imaginary unit.

the rotation of a by θ about n (Figure 17). This is why Part I could read the coordinates u_i as complex numbers: with the first frame vector t, every tangent vector is

$$U = t\,\psi, \qquad \psi = u_1 + u_2\,\mathsf{T}, \tag{12.5}$$

since $t\mathsf{T} = n \times t = b$. We call ψ the *spinor coordinate* of U in the frame t: a complex number of the plane, $|\psi| = |U|$, and $\arg\psi$ is the angle from t to U. The pair (u_1, u_2) of the code is ψ written in the basis $(1, \mathsf{T})$. The frame is now a single unit vector t, the second vector $b = t\mathsf{T}$ being determined by the plane; and changing the frame, $t \to te^{\mathsf{T}\theta}$, changes the coordinate to $\psi \to e^{-\mathsf{T}\theta}\psi$.

13. Rotors Between Tangent Planes

13.1. The smallest rotor taking one normal to another

Proposition 13.1. *For unit vectors a and c with $a \cdot c \neq -1$,*

$$Q = \frac{1 + c\,a}{|1 + c\,a|} = \frac{1 + c\,a}{\sqrt{2(1 + a \cdot c)}} \tag{13.1}$$

is the rotor that turns a to c in the plane $a \wedge c$, by their angle; it is the rotation of the Rodrigues formula (3.4). It takes the bivector Ia to Ic.

Proof. $ca = c \cdot a + c \wedge a$ is the product of two unit vectors, a rotor up to normalization that turns by twice their angle ϕ. The half-way vector $m = (a + c)/|a + c|$ splits that angle in two, and cm, the product of two unit vectors at the angle $\phi/2$, is the rotor that turns by ϕ from a through m to c: $cm\,a\,mc = c(mam)c = c\,c\,c = c$, because reflecting a in the line of m gives c. Now $cm = c(a + c)/|a + c| = (1 + ca)/|a + c|$, and $|a + c|^2 = 2(1 + a \cdot c)$. The

plane of the rotation is that of $c \wedge a$, so its axis is $a \times c$, as in (3.4). Finally a rotor commutes with I, so $Q(Ia)\widetilde{Q} = I(Qa\widetilde{Q}) = Ic$. □

So the rotor

$$Q_{i\to f} = \frac{1 + n_f\, n_i}{|1 + n_f\, n_i|} \tag{13.2}$$

takes the normal of vertex i to that of face f and, by the last statement, the tangent bivector $\mathsf{T}_i = In_i$ to $\mathsf{T}_f = In_f$: it carries the tangent plane of the vertex onto that of the face, and with it every tangent vector, $a \mapsto Q_{i\to f}\, a\, \widetilde{Q}_{i\to f}$. The failure of `alignVectors` for opposite normals is visible here: $1 + n_f n_i = 0$ when $n_f = -n_i$, and no plane of rotation is singled out.

Worked example 13.2. *For the vertex and the face of Example 3.3,* $n_f n_i = n_f \cdot n_i + n_f \wedge n_i = \frac{1}{\sqrt{2}} + I(n_f \times n_i)$ *with* $n_f \times n_i = (-\frac{1}{\sqrt{2}}, 0, 0)$. *Normalizing* $1 + n_f n_i$, *whose norm is* $\sqrt{2(1 + 1/\sqrt{2})} = 1.8478$,

$$Q = 0.9239 - 0.3827\, Ie_1 = \cos 22.5^\circ - Ie_1 \sin 22.5^\circ = e^{-Ie_1\pi/8},$$

the rotor of the rotation by 45° *in the plane* $Ie_1 = e_2e_3$, *about the* x *axis, as found with matrices (the program prints* $Q = 0.923880 + I(-0.382683, 0, 0)$*). It takes* $t_i = e_2$ *to* $c = Qe_2\widetilde{Q} = (e_2 + e_3)/\sqrt{2}$, *and the connection is the geometric product* $t_f c = e_1(e_2 + e_3)/\sqrt{2} = (e_1e_2 + e_1e_3)/\sqrt{2}$, *a unit bivector with no scalar part. Since* $\mathsf{T}_f = In_f = (-Ie_2 + Ie_3)/\sqrt{2} = (-e_3e_1 + e_1e_2)/\sqrt{2} = (e_1e_3 + e_1e_2)/\sqrt{2}$, *this is* $t_f c = \mathsf{T}_f = e^{\mathsf{T}_f \pi/2}$: $\rho_i^f = 90^\circ$, *the same number, now read off a single product.*

13.2. The connection angle

The transported first frame vector $Qt_i\widetilde{Q}$ and the face frame vector t_f are two unit vectors of the face plane. Their geometric product is a complex number of that plane:

$$e^{\mathsf{T}_f \rho_i^f} = t_f \left(Q_{i\to f}\, t_i\, \widetilde{Q}_{i\to f}\right), \tag{13.3}$$

since for unit vectors t_f and c in the plane, $t_f c = t_f \cdot c + t_f \wedge c = \cos\rho + \mathsf{T}_f \sin\rho$, with ρ the angle from t_f to c (the bivector $t_f \wedge c$ is $\sin\rho$ times $t_f \wedge b_f = \mathsf{T}_f$). This single number is the discrete connection: transporting $U = t_i\psi$ from vertex i into face f gives

$$Q\, t_i\psi\, \widetilde{Q} = (Qt_i\widetilde{Q})(Q\psi\widetilde{Q}) = t_f\, e^{\mathsf{T}_f \rho_i^f}\, \psi_f, \tag{13.4}$$

where $\psi_f = u_1 + u_2\mathsf{T}_f$ is the same pair of numbers read in the face plane (because $Q\mathsf{T}_i\widetilde{Q} = \mathsf{T}_f$). In spinor coordinates, transport from the vertex to the face is *multiplication by the unit complex number* $e^{\mathsf{T}_f \rho_i^f}$, and the 2×2 matrix R_i^f of (3.5) is the matrix of that multiplication in the basis $(1, \mathsf{T}_f)$:

$$R_i^f = \begin{pmatrix} \cos\rho_i^f & -\sin\rho_i^f \\ \sin\rho_i^f & \cos\rho_i^f \end{pmatrix}. \tag{13.5}$$

The transport between two vertices through the face, $(R_i^f)^T R_j^f$, is multiplication by $e^{\mathsf{T}(\rho_j^f - \rho_i^f)}$: the connection angles subtract, and the arbitrary angle of the face frame cancels, as found in Section 3.

14. Intrinsic and Extrinsic Geometry

14.1. What an inhabitant of the surface can measure

A two-dimensional being living on the surface can measure lengths of curves and angles between them, and nothing else: it cannot see the normal, nor how the surface bends in space. Everything that can be computed from lengths and angles alone is *intrinsic*; it is unchanged if the surface is bent without stretching, as a sheet of paper rolled into a cylinder. Everything else, the normal n, the tangent bivector T as an element of $\mathbb{G}_3$, the bending of the surface, is *extrinsic*. Gauss's *Theorema Egregium* says that the Gaussian curvature K, defined extrinsically as the product of the principal curvatures, is intrinsic.

The questions this method answers are intrinsic: parallel transport, the covariant derivative, the Dirichlet energy, the connection Laplacian, and hence the heat flow and the transported field, depend only on the metric. A paper cylinder rolled from a flat sheet transports vectors as the flat sheet does, and the method should give the same field on both. The computation, however, uses extrinsic tools: normals, rotations of $\mathbb{R}^3$, projections. The rest of this section separates the two.

14.2. The Gauss formula

Let a be a tangent direction and b a tangent field, and differentiate b in $\mathbb{R}^3$: $a \cdot \nabla b$. Split the result into its tangent and normal parts with (12.2):

$$a \cdot \nabla b = \underbrace{P(a \cdot \nabla b)}_{\nabla_a b} + \underbrace{\big((a \cdot \nabla b) \cdot n\big)\, n}_{\mathrm{II}(a,b)\, n}. \tag{14.1}$$

The tangent part is the covariant derivative (3.1); the normal part is the *second fundamental form*, $\mathrm{II}(a, b) = (a \cdot \nabla b) \cdot n = -b \cdot (a \cdot \nabla n)$ (differentiate $b \cdot n = 0$), which measures how the surface bends under b: it is extrinsic. The covariant derivative is the remaining, intrinsic part. That it depends only on the metric is the content of the fundamental theorem of Riemannian geometry: there is exactly one connection on tangent fields that is compatible with the metric, (3.2), and torsion free, $\nabla_a b - \nabla_b a = [a, b]$; it is given by the metric alone (the Koszul formula), and the projection $\nabla_a b = P(a \cdot \nabla b)$ has both properties [4].

14.3. Parallel transport is rolling without twisting

How does a tangent plane move along a curve $\gamma(s)$ if it carries its vectors parallel? The plane must follow the surface, turning $n(s)$ along with it, but it should not spin about the normal. In geometric algebra this is a rotor equation.

Proposition 14.1. *Let $R(s)$ solve $R' = \frac{1}{2}\Omega\, R$ with $R(0) = 1$, where $\Omega = n' \wedge n$ and $n(s)$ is the normal along the curve. Then $Rn(0)\widetilde{R} = n(s)$, and for every tangent vector a_0 at $\gamma(0)$, the field $a(s) = R\, a_0 \widetilde{R}$ is tangent and parallel along the curve.*

Proof. For $v(s) = Rv_0\widetilde{R}$, the product rule and $\widetilde{R}' = -\frac{1}{2}\widetilde{R}\,\Omega$ (because Ω is a bivector, $\widetilde{\Omega} = -\Omega$) give $v' = \frac{1}{2}(\Omega v - v\Omega) = \Omega \cdot v$, the inner product of a bivector and a vector. With $\Omega = n' \wedge n$ and the rule $(x \wedge y)\cdot v = x(y\cdot v) - y(x\cdot v)$,

$$\Omega \cdot v = n'(n \cdot v) - n(n' \cdot v).$$

First, $m(s) = Rn(0)\widetilde{R}$ satisfies $m' = n'(n \cdot m) - n(n' \cdot m)$, and $m = n$ is a solution, since $n \cdot n = 1$ and $n' \cdot n = 0$; by uniqueness $m = n$. So R takes the normal along the curve, hence the tangent planes onto each other, and $a(s)$ stays tangent. Second, for the tangent field a, $n \cdot a = 0$, so $a' = -(n' \cdot a)\, n$: the derivative of a is purely normal, its tangent part vanishes, and $\nabla_{\gamma'} a = P(a') = 0$. □

Figure 18 shows the rolling planes along a curve on a sphere. The bivector $\Omega = n' \wedge n$ contains the normal, so it never rotates within the tangent plane: the plane *rolls* along the curve, as a coin rolls on a table, without twisting about its normal. Parallel transport is this rolling. The discrete transport $Q_{i\to f}$ of (13.2) is the same idea over a finite step: the rotor in the plane $n_f \wedge n_i$, which contains both normals and therefore has no component of spin about either of them.

14.4. What is intrinsic in the discrete transport

On a triangle mesh the intrinsic geometry is that of the flat faces glued along their edges: it is given by the edge lengths. Transport *across an edge*, from one face to the next, is intrinsic: by Proposition 13.1 with the two face normals, the rotor Q turns about the shared edge (the direction of $n_f \times n_g$), which is exactly the unfolding of one face onto the plane of the other, an isometry that depends only on the two triangles and not on the angle at which they meet in space.

A vertex is different. It has no tangent plane: it is the apex of a cone whose total angle $\Theta_i = \sum_f \alpha_{i,f}$, the sum of the angles of the incident faces at the vertex, is in general not 2π. The code gives the vertex a plane anyway, the plane orthogonal to the vertex normal (2.3), and transports between that plane and the faces with the rolling rotor $Q_{i\to f}$. This is an *extrinsic* choice: bending the mesh at the vertex without stretching any triangle changes the vertex normal and the rotors, although no length changes. The error it introduces vanishes as the mesh is refined on a smooth surface, because the vertex normal then converges to the normal of the smooth surface and the rolling rotors to its parallel transport. An intrinsic alternative [10,17] defines the tangent space of the vertex by the angles around it, rescaled by $2\pi/\Theta_i$ so that they close up, and the transport to a face, or across an edge, by those

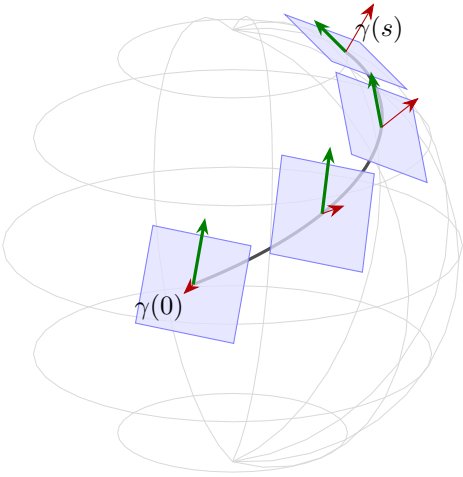


FIGURE 18. Parallel transport is rolling without twisting. Along a curve γ on a sphere, the tangent plane (blue) is rolled by the rotor of Proposition 14.1, here integrated as a sequence of smallest rotations between consecutive normals (red). The transported vector (green) changes only in the direction of the normal, so its covariant derivative vanishes.

rescaled angles; in the code it replaces only the computation of the connection angle ρ_i^f (`Rvf`), leaving the finite element machinery untouched. Part III develops it.

14.5. Holonomy and curvature

Carry a vector parallel around a small closed loop and it comes back rotated, in the tangent plane, by an angle: the *holonomy* of the loop. For rolling, Proposition 14.1, the angle has a geometric meaning: it is the area enclosed by the loop that the normal $n(s)$ traces on the unit sphere. By the Gauss–Bonnet theorem that area is $\int K\, dA$ over the region enclosed by the loop on the surface, which is why holonomy measures curvature and is intrinsic. On a mesh, the intrinsic transport around a vertex, unfolding the faces one after another, closes with the rotation $2\pi - \Theta_i$, the *angle defect*, the discrete total curvature of the vertex. The code's transport, which rolls through the vertex normals, closes with the area of the spherical polygon whose corners are the normals visited by the loop, those of the neighboring vertices and faces, which approximates the angle defect on fine meshes; the remark of Section 3 should be read in this sense.

15. The Covariant Derivative as a Connection on Spinor Coordinates

15.1. The connection form

Choose a frame field, a unit tangent vector $t(x)$ at each point, and write fields as $U = t\psi$ with spinor coordinates $\psi = u_1 + u_2\mathsf{T}$. Differentiating along a tangent direction a, $a\cdot\nabla U = (a\cdot\nabla t)\,\psi + t\,(a\cdot\nabla\psi)$. The covariant derivative of the frame vector is tangent and orthogonal to t (since $|t| = 1$), so it is a multiple of $t\mathsf{T}$:

$$\nabla_a t = \omega(a)\, t\,\mathsf{T}, \tag{15.1}$$

where the scalar $\omega(a)$, linear in a, is the rate at which the frame turns, measured against parallel transport: the *connection form* of the frame, the angle $\theta(a)$ of (3.3). Writing $U = u_1 t + u_2 b$ with $b = t\mathsf{T}$, and using also $\nabla_a b = -\omega(a)\, t$ (the frame turns rigidly, since $t\cdot b = 0$), the covariant derivative of U is

$$\nabla_a(t\psi) = t\left(a\cdot\nabla\psi + \omega(a)\,\mathsf{T}\,\psi\right) = t\,D_a\psi, \qquad D_a = a\cdot\nabla + \omega(a)\,\mathsf{T}, \tag{15.2}$$

where $a\cdot\nabla\psi = (a\cdot\nabla u_1) + (a\cdot\nabla u_2)\,\mathsf{T}$ differentiates the two coefficients, and $\omega\mathsf{T}\psi = \omega(u_1\mathsf{T} - u_2)$ collects the terms $u_1\nabla_a t + u_2\nabla_a b$. On spinor coordinates, the Levi-Civita connection is the ordinary derivative plus a multiplication by $\omega\mathsf{T}$: an infinitesimal rotation. In the language of physics, ψ is a charged field and $\omega\mathsf{T}$ a $U(1)$ gauge potential; the connection Laplacian is the corresponding *magnetic* Laplacian, and the curvature of the connection, the "magnetic field", is $d\omega$, which with the orientation $b = t\mathsf{T} = n\times t$ used here is minus the Gaussian curvature times the area form, $d\omega = -K\,dA$ [10] (Remark 15.1).

15.2. Frame independence

The frame t is arbitrary, and nothing physical may depend on it. Rotating the frame by a field of angles, $t \to t' = t\,e^{\mathsf{T}\theta}$, changes the coordinates to $\psi' = e^{-\mathsf{T}\theta}\psi$ and, since $\nabla_a t' = (\nabla_a t)\,e^{\mathsf{T}\theta} + t\,(a\cdot\nabla\theta)\,\mathsf{T}e^{\mathsf{T}\theta}$, the connection form to $\omega' = \omega + d\theta$; and then

$$D'_a\psi' = e^{-\mathsf{T}\theta}\,D_a\psi, \tag{15.3}$$

as one checks by differentiating $e^{-\mathsf{T}\theta}\psi$: the extra term $-(a\cdot\nabla\theta)\,\mathsf{T}\,e^{-\mathsf{T}\theta}\psi$ is cancelled by the extra term $(a\cdot\nabla\theta)\,\mathsf{T}\,e^{-\mathsf{T}\theta}\psi$ of ω'. So $\nabla U = t\,D\psi = t'D'\psi'$ is the same tangent object in both frames, $|D\psi|$ is frame independent, and so are the Dirichlet energy, the connection Laplacian and the solution of the heat equation. In the discrete setting this is the cancellation of the face frame in $(R_i^f)^T R_j^f$ and the statement that changing a vertex frame t_i rotates the computed u_i by the opposite angle and changes nothing else: the arrow $t_i\hat{\psi}_i$ drawn by the code is independent of the choice of t_i in `computeVertexTangentSpace`.

Remark 15.1 (Cartan's structure equations). *Write θ^1, θ^2 for the* coframe *of the frame (t, b), the 1-forms $\theta^1(X) = t\cdot X$ and $\theta^2(X) = b\cdot X$, so that $X = \theta^1(X)\,t + \theta^2(X)\,b$ and $dA = \theta^1\wedge\theta^2$. Then*

$$d\theta^1 = \omega\wedge\theta^2, \qquad d\theta^2 = -\,\omega\wedge\theta^1, \qquad d\omega = -K\,\theta^1\wedge\theta^2.$$

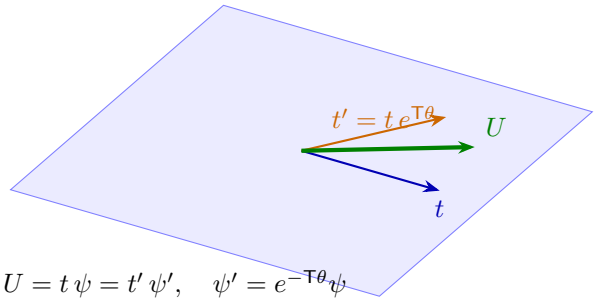


FIGURE 19. A change of frame is a change of gauge. The same tangent vector U has the spinor coordinate ψ in the frame t and $\psi' = e^{-\mathsf{T}\theta}\psi$ in the frame $t' = t\,e^{\mathsf{T}\theta}$; nothing geometric depends on the choice.

The first two determine ω from the coframe, hence from the metric alone, which is the Levi-Civita theorem in Cartan's form; the third says that the curvature is the exterior derivative of the connection form. On the unit sphere with $t = \partial_\vartheta$, $\theta^1 = d\vartheta$, $\theta^2 = \sin\vartheta\, d\varphi$, and the equations give $\omega = \cos\vartheta\, d\varphi$ and $d\omega = -\sin\vartheta\, d\vartheta \wedge d\varphi = -dA$, so $K = 1$. Section 42 proves the equations and uses them to build every element of this text, and a curved one.

15.3. The energy and the equation in spinor form

With $|U| = |\psi|$ and $|\nabla_a U| = |D_a\psi|$,

$$E = \frac{1}{2}\int_\Omega \sum_k |D_{e_k}\psi|^2\, dA, \qquad \frac{\partial\psi}{\partial t} = -D^*D\,\psi, \tag{15.4}$$

and the backward Euler step and its weak form, (4.4) and (5.1), read

$$\int_\Omega \langle\psi\,\widetilde{\chi}\rangle\, dA + \Delta t\int_\Omega \sum_k \langle (D_{e_k}\psi)\,\widetilde{(D_{e_k}\chi)}\rangle\, dA = \int_\Omega \langle\psi^0\,\widetilde{\chi}\rangle\, dA \tag{15.5}$$

for every test spinor χ, where $\langle\psi\widetilde{\chi}\rangle$ is the scalar part of $\psi\widetilde{\chi}$: for $\psi = u_1 + u_2\mathsf{T}$ and $\chi = v_1 + v_2\mathsf{T}$, $\widetilde{\chi} = v_1 - v_2\mathsf{T}$ and $\langle\psi\widetilde{\chi}\rangle = u_1v_1 + u_2v_2 = U\cdot V$. This is the real part of $\psi\bar{\chi}$, the familiar Hermitian inner product, with the reverse playing the role of complex conjugation.

16. The Finite Element Method in Geometric Algebra

16.1. Spinor hat functions

The trial field (6.1), in spinor coordinates of the face, is

$$U\big|_f = t_f\,\psi_f, \qquad \psi_f = \sum_{i\in f} \phi_i\, e^{\mathsf{T}_f\rho_i^f}\,\psi_i, \tag{16.1}$$

by (13.4): each vertex coordinate is turned by its connection angle into the face, and the three are interpolated with the hats. The face is flat and t_f constant, so $\omega = 0$ on the face and $D = \nabla$, and

$$D_a\psi_f = \sum_{i\in f} (a\cdot\nabla\phi_i)\, e^{\mathsf{T}_f \rho_i^f}\, \psi_i. \tag{16.2}$$

16.2. The gradients of the hat functions: the reciprocal frame

On the face with vertices x_1, x_2, x_3, take the *edge frame* $e_1 = x_2 - x_1$, $e_2 = x_3 - x_1$, the columns of B in (7.3), and their *area bivector* $E = e_1 \wedge e_2 = 2A_f\,\mathsf{T}_f$. The *reciprocal frame* e^1, e^2 is the pair of vectors of the face plane with $e^k \cdot e_m = \delta_{km}$. In geometric algebra it is written directly:

$$e^1 = e_2 \cdot E^{-1}, \qquad e^2 = -\,e_1 \cdot E^{-1}, \qquad E^{-1} = -\frac{E}{|E|^2}. \tag{16.3}$$

Indeed $e_2 \cdot E$ lies in the plane and is orthogonal to e_2, and $(e_2 \cdot E)\cdot e_1 = (e_2\cdot e_1)^2 - |e_2|^2|e_1|^2 = -|E|^2$, so $e^1 \cdot e_1 = 1$ and $e^1 \cdot e_2 = 0$; similarly for e^2.

Proposition 16.1. *The gradients of the hat functions of the face are $\nabla\phi_2 = e^1$, $\nabla\phi_3 = e^2$ and $\nabla\phi_1 = -e^1 - e^2$; the reciprocal frame is the pair of columns of $B(B^TB)^{-1}$,* `Binv`*. Equivalently, for each vertex a with the opposite edge $x_c - x_b$ (in the counterclockwise order a, b, c),*

$$\nabla\phi_a = \frac{(x_c - x_b)\,\mathsf{T}_f}{2A_f}\,: \tag{16.4}$$

the opposite edge, turned a quarter turn inward, divided by twice the area.

Proof. The gradient g of ϕ_a lies in the plane and is determined by its inner products with the edges, $g\cdot e_m = \partial\hat{\phi}_a/\partial\xi_m$ (proof of Proposition 7.1). Writing $g = \sum_k (\partial\hat{\phi}_a/\partial\xi_k)\, e^k$ satisfies these equations, by the defining property of the reciprocal frame, and with (7.2) gives the three gradients. The matrix $B(B^TB)^{-1}$ has the same defining property, $B^T\big(B(B^TB)^{-1}\big) = I$, and its columns lie in the plane, so its columns are e^1 and e^2. For (16.4), take $a = 2$; in the counterclockwise order $a, b, c = 2, 3, 1$ its opposite edge is $x_c - x_b = x_1 - x_3 = -e_2$. Since $E = 2A_f\mathsf{T}_f$ and $\mathsf{T}_f^{-1} = -\mathsf{T}_f$, $E^{-1} = -\mathsf{T}_f/(2A_f)$, and by (16.3) $e^1 = e_2\cdot E^{-1} = -e_2\mathsf{T}_f/(2A_f) = (x_1 - x_3)\,\mathsf{T}_f/(2A_f)$, using that for a vector in the plane of a bivector the inner and geometric products coincide. The other vertices follow by relabeling. (On the triangle $x_1 = 0$, $x_2 = e_x$, $x_3 = e_y$ with $n = e_z$: $(x_1 - x_3)\mathsf{T} = n \times (-e_y) = e_x$, which is indeed $\nabla\phi_2$.) $\square$

The formula (16.4) makes the cotangent formula of Proposition 7.3 immediate: the quarter turn preserves inner products, so $A_f\,\nabla\phi_a\cdot\nabla\phi_b = \frac{(x_c - x_b)\cdot(x_a - x_c)}{4A_f}$, and for the two edges meeting at c, the ratio of their inner product to $|(x_c - x_b)\wedge(x_a - x_c)| = 2A_f$ is $-\cot\theta_c$. In the vector manifold calculus of Hestenes and Sobczyk [9], the reciprocal frame is the general rule: the vector derivative on a face is $\nabla = e^1\partial_{\xi_1} + e^2\partial_{\xi_2}$.

Worked example 16.2. *For the triangle of Example 7.2,* $E = e_1 \wedge e_2 = I(e_1 \times e_2) = I(0,-2,2)$*, with* $|E| = 2\sqrt{2} = 2A_f$*, and* $\mathsf{T}_f = E/|E| = In_f$*. The gradient of* ϕ_2 *by (16.4) is the opposite edge* $x_1 - x_3 = (0,-1,-1)$*, turned a quarter turn in the face,* $(x_1 - x_3)\,\mathsf{T}_f = n_f \times (x_1 - x_3) = \frac{1}{\sqrt{2}}(0,-1,1) \times (0,-1,-1) = \frac{1}{\sqrt{2}}(2,0,0)$*, divided by* $2A_f = 2\sqrt{2}$*:* $\nabla\phi_2 = (\frac{1}{2},0,0)$*, as in Example 7.2, without inverting any matrix.*

16.3. The element matrices

For two vertices i, j of the face, the stiffness integrand of (15.5) with the trial field (16.2) and the test spinor $\chi = \phi_i\, e^{\mathsf{T}_f \rho_i^f}\, \chi_i$ is

$$\sum_k \big\langle (D_{e_k}\psi)\,\widetilde{(D_{e_k}\chi)} \big\rangle = (\nabla\phi_i \cdot \nabla\phi_j)\,\big\langle e^{\mathsf{T}_f \rho_j^f}\,\psi_j\;\widetilde{\chi_i}\, e^{-\mathsf{T}_f \rho_i^f} \big\rangle$$

$$= (\nabla\phi_i \cdot \nabla\phi_j)\,\big\langle e^{\mathsf{T}_f (\rho_j^f - \rho_i^f)}\,\psi_j\,\widetilde{\chi_i} \big\rangle,$$

using that the even elements of the plane commute and the cyclic property of the scalar part. So, as a complex matrix,

$$\mathrm{L}_{ij}^f = A_f\,(\nabla\phi_i \cdot \nabla\phi_j)\; e^{\mathsf{T}(\rho_j^f - \rho_i^f)}, \qquad \mathrm{M}_{ii}^f = \frac{A_f}{3}\,: \tag{16.5}$$

one complex number per pair of vertices instead of a 2×2 block. The global stiffness L is then an $n \times n$ *Hermitian* matrix, $\mathrm{L}_{ji} = \overline{\mathrm{L}_{ij}}$ (the reverse of $e^{\mathsf{T}\rho}$ is $e^{-\mathsf{T}\rho}$), the form in which discrete connection Laplacians are usually written [10, 17], and the backward Euler step is the complex system

$$(\mathrm{M} + \Delta t\,\mathrm{L})\,\psi = \mathrm{M}\,\psi^0, \qquad \psi \in \mathbb{C}^n. \tag{16.6}$$

16.4. The code solves the real form of the complex system

A complex number $z = \alpha + \beta\mathsf{T}$ acts on $\psi = u_1 + u_2\mathsf{T}$ by multiplication, and in the basis $(1, \mathsf{T})$ that multiplication is the real 2×2 matrix $\begin{pmatrix} \alpha & -\beta \\ \beta & \alpha \end{pmatrix}$. Replacing every complex entry of (16.6) by this matrix, and every complex unknown by its two real coordinates, turns the $n \times n$ complex system into a $2n \times 2n$ real one with the same solution: and that real system is exactly the one assembled in Part I. The element block (6.7) is the matrix of multiplication by (16.5), $(R_i^f)^T R_j^f$ being the matrix of $e^{\mathsf{T}(\rho_j^f - \rho_i^f)}$ by (13.5), and the real mass and the Dirichlet rows are the real forms of their complex counterparts. The code works with real arithmetic and the general sparse solver; the complex form halves the size of the system and shows that every block of the code has the structure "scalar times rotation".

17. The Geometric Algebra in Code

The repository contains a second program, `ga_fem.cpp`, which needs no graphics and implements Part II with the small header `GAConnection.h`. It is built and run from `src/` with

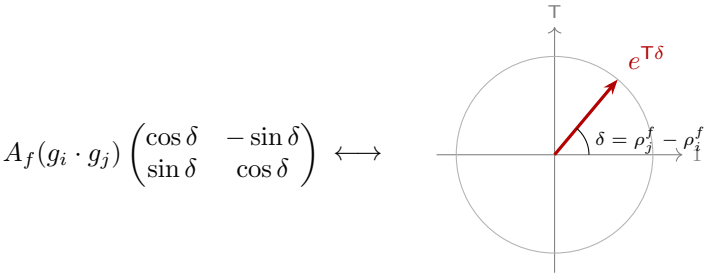


FIGURE 20. Every 2×2 block of the code is a real multiple of a rotation, the matrix of multiplication by a complex number $e^{\mathsf{T}\delta}$ of the tangent plane; the complex form stores only that number.

```
c++ -std=c++17 -O2 -I<eigen> ga_fem.cpp HalfEdge/Mesh.cpp -o ga_fem
./ga_fem ../fem-basic-2d/data
```

It prints the worked examples of this text, solves the cactus of `FEM.cpp` twice, with the real 2×2 blocks of Part I and with the complex Hermitian system of Part II, solves the sphere of Example 10.3, and writes the data of Figures 15 and 16. On the cactus, the two solutions agree to $6\cdot10^{-16}$, the rounding error of double precision: the two parts of this text compute the same thing.

The algebra. Only the grades the method needs are implemented: vectors, bivectors stored by their dual vectors ($B = Ib$), and rotors $w + Ib$. The product of two vectors is a rotor up to normalization, and the inner product of a vector with a bivector is the quarter turn (12.3):

```
19 using Vec3 = Eigen::Vector3d;
20 using Complex = std::complex<double>;
21
22 /// A bivector B = I b, stored by its dual vector b. Unit bivectors square to -1.
23 struct Bivector {
24     Vec3 b;
25     Bivector operator*(double s) const { return { b * s }; }
26     double norm() const { return b.norm(); }
27 };
28
29 /// The outer product a ^ c = I (a x c): the oriented plane of a and c.
30 inline Bivector wedge(const Vec3& a, const Vec3& c) { return { a.cross(c) }; }
31
32 /// The inner product of a vector and a bivector, a . B, a vector of the plane of B,
       orthogonal to a.
33 /// For B = I b: a . (I b) = I (a ^ b) = I I (a x b) = b x a.
34 inline Vec3 dot(const Vec3& a, const Bivector& B) { return B.b.cross(a); }
35
36 /// B^-1 = -B / |B|^2, since B^2 = -|B|^2.
37 inline Bivector inverse(const Bivector& B) { return B * (-1.0 / B.b.squaredNorm()); }
38
39 /// A rotor R = w + I b (Section 12). R a ~R rotates the vector a.
40 struct Rotor {
41     double w;
42     Vec3 b;
43     /// R a ~R. With R = w + I b, the rotation is the one of the unit quaternion (w, -b
          ):
44     /// a' = a + 2 w (u x a) + 2 u x (u x a), u = -b.
```

```
    Vec3 rotate(const Vec3& a) const {
        const Vec3 u = -b;
        const Vec3 t = 2.0 * u.cross(a);
        return a + w * t + u.cross(t);
    }
};

/// The geometric product of two vectors, c a = c . a + c ^ a, a rotor up to normalization.
inline Rotor product(const Vec3& c, const Vec3& a) { return { c.dot(a), c.cross(a) }; }

/// The tangent bivector T = I n of a unit normal n (eq. ga_T): the tangent plane as an oriented plane element.
inline Bivector tangentBivector(const Vec3& n) { return { n }; }

/// a T for a tangent vector a: the quarter turn n x a (eq. ga_quarter_turn).
/// For a in the plane of T the geometric product equals the inner product, a T = a . T.
inline Vec3 quarterTurn(const Vec3& a, const Bivector& T) { return dot(a, T); }
```

The connection. The smallest rotor of Proposition 13.1, and the connection (13.3) as the geometric product of two unit vectors of the face plane, a complex number of that plane:

```
/// The smallest rotor taking the unit vector a to the unit vector c (Proposition ga_rotor):
/// Q = (1 + c a) / |1 + c a|. Undefined for c = -a, where 1 + c a = 0.
inline Rotor rotorBetween(const Vec3& a, const Vec3& c) {
    const Rotor ca = product(c, a);
    const double w = 1.0 + ca.w;
    const double norm = std::sqrt(w * w + ca.b.squaredNorm()); // |1 + c a| = sqrt(2 (1 + a.c))
    return { w / norm, ca.b / norm };
}

/// The complex number of the plane with normal n given by the product of two unit vectors of that plane:
/// t c = t . c + t ^ c = cos(rho) + T sin(rho), with t ^ c = (t x c . n) T (eq. ga_connection).
inline Complex planeProduct(const Vec3& t, const Vec3& c, const Vec3& n) {
    return { t.dot(c), t.cross(c).dot(n) };
}

/// The discrete connection from vertex i to face f, as a unit complex number (eq. ga_connection):
/// e^{T_f rho_i^f} = t_f (Q t_i ~Q), Q = rotorBetween(n_i, n_f).
/// Transporting the spinor coordinate psi of vertex i into the face is multiplication by it (eq. ga_transport).
inline Complex connection(const Vec3& t_i, const Vec3& n_i, const Vec3& t_f, const Vec3& n_f) {
    const Rotor Q = rotorBetween(n_i, n_f);
    return planeProduct(t_f, Q.rotate(t_i), n_f);
}
```

The gradients. The reciprocal frame of Proposition 16.1, and the single-vertex form (16.4):

```
/// The gradients of the three hat functions of the triangle x1, x2, x3 (Proposition ga_reciprocal):
/// the reciprocal frame e^1 = e2 . E^-1, e^2 = -e1 . E^-1 of the edges e1 = x2 - x1, e2 = x3 - x1, with
/// E = e1 ^ e2; then grad phi_2 = e^1, grad phi_3 = e^2, grad phi_1 = -e^1 - e^2.
inline void hatGradients(const Vec3& x1, const Vec3& x2, const Vec3& x3, Vec3 grad[3]) {
    const Vec3 e1 = x2 - x1, e2 = x3 - x1;
```

```
    const Bivector Einv = inverse(wedge(e1, e2));
    const Vec3 r1 = dot(e2, Einv);
    const Vec3 r2 = -dot(e1, Einv);
    grad[0] = -r1 - r2;
    grad[1] = r1;
    grad[2] = r2;
}

/// The same gradient, one vertex at a time (eq. ga_grad_edge): the opposite edge, from b to c in the
/// counterclockwise order a, b, c, turned a quarter turn in the plane of the face, divided by twice the area.
inline Vec3 hatGradient(const Vec3& xb, const Vec3& xc, const Bivector& Tf, double area) {
    return quarterTurn(xc - xb, Tf) / (2.0 * area);
}
```

The complex system. The assembly of (16.5), the scaling by the lumped mass, and the Dirichlet row, with `std::complex` entries. Compare it line by line with `AssembleStiffnessElementEmbedded2`, `AssembleMatrix2`, `IplusMinvTimesA` and `PreFactor`, where each complex number is a 2×2 block:

```
/// Section 16: L_ij += A_f (grad phi_i . grad phi_j) e^{T (rho_j - rho_i)}, M_ii += A_f / 3, then the same
/// scaling I + M^-1 (dt L) and the same Dirichlet row as the matrix version, now with complex numbers.
static Eigen::VectorXcd solve(Mesh& mesh, const Geometry& g, double dt, int source, Complex value) {
    const int n = mesh.numVertices();
    std::vector<double> mass(n, 0.0);
    std::map<std::pair<int, int>, Complex> A;
    for (Face& f : mesh.getFaces()) {
        Vertex* v[3];
        faceVertices(f, v);
        ga::Vec3 grad[3];
        ga::hatGradients(v[0]->p, v[1]->p, v[2]->p, grad); // the reciprocal frame
        const double area = g.faceArea[f.ID];
        Complex rho[3]; // e^{T_f rho_a^f}
        for (int a = 0; a < 3; ++a)
            rho[a] = ga::connection(g.vertexT[v[a]->ID], v[a]->n, g.faceT[f.ID], g.faceNormal[f.ID]);
        for (int a = 0; a < 3; ++a) {
            mass[v[a]->ID] += area / 3.0;
            for (int b = 0; b < 3; ++b) {
                // transport from b to a through the face: e^{T (rho_b - rho_a)} = e^{T rho_b} conj(e^{T rho_a})
                const Complex transport = a == b ? Complex(1.0) : rho[b] * std::conj(rho[a]);
                A[{ v[a]->ID, v[b]->ID }] += dt * area * grad[a].dot(grad[b]) * transport;
            }
        }
    }
    std::vector<Eigen::Triplet<Complex>> triplets;
    for (const auto& [ij, value_ij] : A) {
        if (ij.first == source) continue;
        triplets.emplace_back(ij.first, ij.second, value_ij / mass[ij.first] + (ij.first == ij.second ? 1.0 : 0.0));
    }
    triplets.emplace_back(source, source, 1.0);
    Eigen::SparseMatrix<Complex> S(n, n);
    S.setFromTriplets(triplets.begin(), triplets.end());
    Eigen::VectorXcd rhs = Eigen::VectorXcd::Zero(n);
    rhs(source) = value;
    Eigen::SparseLU<Eigen::SparseMatrix<Complex>> lu(S);
```

TABLE 2. The objects of Part I and of Part II, and whether they depend only on the metric of the surface (intrinsic) or on its embedding in space (extrinsic).

Object	Matrices (Part I)	Geometric algebra	Nature
tangent plane	$TT^T = I - nn^T$	bivector $\mathsf{T} = In$	extrinsic
frame	$T = [t \;\; n \times t]$	one vector t, $b = t\mathsf{T}$	a gauge choice
coordinates	$u \in \mathbb{R}^2$	spinor $\psi = u_1 + u_2\mathsf{T}$	gauge dependent
rotation in the plane	2×2 rotation	$e^{\mathsf{T}\theta}$	intrinsic
plane to plane	Rodrigues Q	rotor $(1 + n_f n_i)/\lvert 1 + n_f n_i\rvert$	extrinsic tool
discrete connection	$R_i^f = T_f^T Q T_i$	$e^{\mathsf{T}\rho_i^f} = t_f Q t_i \widetilde{Q}$	see Sec. 14
transport $j \to i$	$(R_i^f)^T R_j^f$	$e^{\mathsf{T}(\rho_j - \rho_i)}$	gauge covariant
covariant derivative	$\partial u + \omega u$	$D = \nabla + \omega\mathsf{T}$	intrinsic
hat gradients	$B(B^TB)^{-1}\hat{\nabla}\hat{\phi}$	reciprocal frame, $\frac{(x_c - x_b)\mathsf{T}}{2A}$	intrinsic
stiffness block	$A(g_i \cdot g_j)R_i^T R_j$	$A(\nabla\phi_i \cdot \nabla\phi_j)e^{\mathsf{T}(\rho_j - \rho_i)}$	intrinsic up to the vertex planes
global system	real $2n \times 2n$	complex Hermitian $n \times n$	—

```
    return lu.solve(rhs);
}
```

18. Dictionary

Table 2 lists each object of the two parts side by side, with its nature, intrinsic or extrinsic.

The geometric algebra formulation removes the frames from the geometry: a tangent plane is a bivector, a direction in it is a complex number of that plane, and transport is a rotor. Frames reappear only as the gauge in which coordinates are stored, and the gauge covariance (15.3) guarantees that they do not affect the result. What remains of the embedding is the choice of a tangent plane at each vertex, the one extrinsic ingredient of the discrete connection, and the place where the intrinsic method of Part III differs.

Part III. The Intrinsic Method: Everything from Edge Lengths

Part II ended with one extrinsic ingredient left in the method: the tangent plane of a vertex, which the code takes orthogonal to the vertex normal. This part removes it. We build the whole method from the edge lengths of the mesh, and nothing else: no normal, no rotor of space, no position in $\mathbb{R}^3$. The finite element method itself does not change at all. The element matrix (16.5) needs three things per face, an area, the gradients of the three hat functions,

and the connection angle ρ_i^f of each corner; we compute the first two from the lengths of the face (Section 20), and the third from the angles of the faces around each vertex (Sections 21 and 22).

Nothing beyond Part II is needed, and only little of it: the law of cosines, angles, and the unit complex numbers $e^{\mathsf{T}\theta}$ of a plane, which here are nothing more than rotations of the plane by the angle θ (the reader may read T as i). We go slowly, derive every formula, and say at each step which concept of Part I or Part II it replaces. The program of this part is `IntrinsicConnection.h` with the function `intrinsicform::solve` of `ga_fem.cpp`; Section 24 compares it with Part II on folded sheets, on spheres and on the cactus.

19. What It Means to Be Intrinsic

19.1. A folded sheet of paper

Take a flat sheet of paper and draw on it a field of parallel arrows, all pointing the same way. On the flat sheet, these arrows are the parallel transport of each other: the vector heat method started from one of them should return all of them. Now fold the sheet, without stretching or tearing it, along a few straight lines. The arrows are still drawn on the paper. An ant walking on the paper cannot tell that the paper was folded: it measures the same lengths and the same angles as before, and to it the arrows are still parallel. Parallel transport is defined by what the ant can measure (Section 14), so the correct answer on the folded sheet is the same field of arrows, drawn on the folded paper.

The method of Parts I and II gives that answer for some foldings and not for others. If all the folds are parallel, as when the sheet is folded into a prism, it is exact. If the folds meet at angles, as in the Miura-ori pattern of Figure 21, a classical fold of maps and of solar panels in which four creases meet at every vertex, its vectors deviate from the drawn ones by 7.4° on average and by up to 21.9° (Section 24), although the paper is not stretched anywhere. The culprit is the vertex normal (2.3). At a vertex of the Miura-ori the four faces are tilted in four different directions, and their averaged normal defines a plane that is none of them and does not exist on the paper. Part II rolls each face into that plane, each about a different axis, and the rolled faces no longer fit together edge to edge as they do on the paper. When all the folds through a vertex are parallel, every face is rolled about the same line, the fold, and rolling is exactly unfolding: that is why the prism comes out right. The method is bent by the folding, although the paper, as far as the ant knows, is not.

19.2. Intrinsic quantities of a mesh

On a triangle mesh, what the ant can measure is the length of every edge, and from the lengths, the angles of every triangle. Two meshes with the same triangles, glued in the same way, and the same edge lengths are *isometric*:

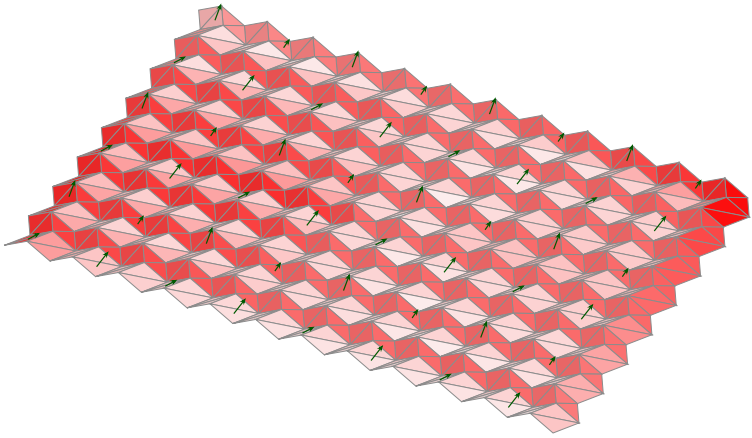


FIGURE 21. A Miura-ori fold: 24×16 parallelograms, each cut into two triangles, folded so that four creases meet at every vertex. The four angles at every vertex still add up to 2π, so the folded sheet is isometric to a flat one, and parallel transport on it is the transport of the flat sheet. Arrows: the exact field, started at the center, which the method of this part reproduces to $2 \cdot 10^{-13}$ degrees. Shading: the error of the method of Part II in each face, from $0°$ (white) to $21.9°$ (red).

the second may be a folded copy of the first, as in Figure 21, and the ant cannot tell them apart. A quantity is *intrinsic* if it can be computed from the edge lengths alone; it is then the same on all isometric meshes.

Let us list what the program of Parts I and II reads from the mesh:

1. the positions x_a of the vertices of each face, through the matrix B, to compute the area and the gradients of the hat functions (Section 7);
2. the face normal n_f, for the face frame and the rotor $Q_{i\to f}$;
3. the vertex normal n_i, for the vertex frame and the rotor $Q_{i\to f}$.

We shall see that item 1 only looks extrinsic: the area and the gradients depend on the three edge lengths alone (Section 20). Items 2 and 3 enter only through the connection angle ρ_i^f; we replace them by angles measured on the faces (Sections 21 and 22).

20. A Triangle Known Only by Its Edge Lengths

20.1. The metric of a face is its Gram matrix

Recall from Section 7 the affine map $F(\xi) = x_1 + B\xi$ from the reference triangle to the face, with $B = [\, x_2 - x_1 \;\; x_3 - x_1 \,] = [\, e_1 \;\; e_2 \,]$, a 3×2 matrix. A small step $d\xi$ in the reference triangle is carried to the step $B\, d\xi$ on the

face, whose squared length is

$$|B\,d\xi|^2 = d\xi^T (B^T B)\, d\xi.$$

So the 2×2 matrix

$$G_f = B^T B = \begin{pmatrix} e_1 \cdot e_1 & e_1 \cdot e_2 \\ e_1 \cdot e_2 & e_2 \cdot e_2 \end{pmatrix} \tag{20.1}$$

is the rule that measures lengths, and therefore angles, on the face, in the coordinates ξ of the reference triangle. In differential geometry such a rule is called a *metric*, or the *first fundamental form* of the surface in the coordinates ξ; in linear algebra, G_f is the *Gram matrix* of the two edges. The matrix B itself says more: where the face is in space and how it is turned. The metric keeps only what the ant can measure.

The metric is determined by the three edge lengths. Write $\ell_{12} = |e_1|$, $\ell_{13} = |e_2|$ and $\ell_{23} = |x_3 - x_2|$. The diagonal entries are ℓ_{12}^2 and ℓ_{13}^2. For the off-diagonal entry, the third edge is $x_3 - x_2 = e_2 - e_1$, so

$$\ell_{23}^2 = |e_2 - e_1|^2 = |e_1|^2 - 2\, e_1 \cdot e_2 + |e_2|^2,$$

and solving for $e_1 \cdot e_2$,

$$G_f = \begin{pmatrix} \ell_{12}^2 & \frac{1}{2}(\ell_{12}^2 + \ell_{13}^2 - \ell_{23}^2) \\ \frac{1}{2}(\ell_{12}^2 + \ell_{13}^2 - \ell_{23}^2) & \ell_{13}^2 \end{pmatrix}. \tag{20.2}$$

This is the law of cosines in disguise: $e_1 \cdot e_2 = \ell_{12}\ell_{13} \cos\alpha_1$, where α_1 is the angle of the face at x_1.

Proposition 20.1. *The area of a face, the gradients' inner products $\nabla\phi_a \cdot \nabla\phi_b$, and therefore the element stiffness and mass matrices of the scalar parts, $A_f\, \nabla\phi_a \cdot \nabla\phi_b$ and $A_f/3$, depend on B only through G_f, and so only on the three edge lengths.*

Proof. Area: by Lagrange's identity $|e_1 \times e_2|^2 = |e_1|^2|e_2|^2 - (e_1 \cdot e_2)^2 = \det G_f$, so $A_f = \frac{1}{2}|e_1 \times e_2| = \frac{1}{2}\sqrt{\det G_f}$. Gradients: by Proposition 7.1, $\nabla\phi_a = B\, G_f^{-1} \hat{\nabla}\hat{\phi}_a$, so

$$\nabla\phi_a \cdot \nabla\phi_b = \hat{\nabla}\hat{\phi}_a^T\, G_f^{-1} B^T B\, G_f^{-1}\, \hat{\nabla}\hat{\phi}_b = \hat{\nabla}\hat{\phi}_a^T\, G_f^{-1}\, \hat{\nabla}\hat{\phi}_b,$$

where the reference gradients $\hat{\nabla}\hat{\phi}_a$ of (7.2) are fixed numbers. Finally G_f is given by the lengths, (20.2). □

So the matrix B, which Part I needed in order to find the gradients as vectors of space, was carrying more than the method uses: the method uses only its "shadow" $B^T B$. This settles item 1 of the list of Section 19.

20.2. Heron's formula and the angles

Two formulas from lengths will be used directly. The first gives the area. From the proof above and (20.2),

$$16A_f^2 = 4 \det G_f = 4\ell_{12}^2\ell_{13}^2 - (\ell_{12}^2 + \ell_{13}^2 - \ell_{23}^2)^2,$$

a difference of two squares, $(2\ell_{12}\ell_{13})^2 - (\ell_{12}^2 + \ell_{13}^2 - \ell_{23}^2)^2$. Factoring it, and then each factor again as a difference of squares,

$$\begin{aligned} 16A_f^2 &= \left(\ell_{23}^2 - (\ell_{12} - \ell_{13})^2\right)\left((\ell_{12} + \ell_{13})^2 - \ell_{23}^2\right) \\ &= (\ell_{23} - \ell_{12} + \ell_{13})(\ell_{23} + \ell_{12} - \ell_{13})(\ell_{12} + \ell_{13} - \ell_{23})(\ell_{12} + \ell_{13} + \ell_{23}). \end{aligned}$$

With the half perimeter $\sigma = \frac{1}{2}(\ell_{12} + \ell_{13} + \ell_{23})$, the four factors are $2(\sigma - \ell_{12})$, $2(\sigma - \ell_{13})$, $2(\sigma - \ell_{23})$ and 2σ, which is *Heron's formula*:

$$A_f = \sqrt{\sigma(\sigma - \ell_{12})(\sigma - \ell_{13})(\sigma - \ell_{23})}. \tag{20.3}$$

The second gives the angles. Dividing the off-diagonal entry of (20.2) by $\ell_{12}\ell_{13}$,

$$\cos\alpha_1 = \frac{\ell_{12}^2 + \ell_{13}^2 - \ell_{23}^2}{2\,\ell_{12}\,\ell_{13}}, \tag{20.4}$$

and the same at the other two corners, with the lengths relabeled. As a check against Part I: $\sin\alpha_1 = 2A_f/(\ell_{12}\ell_{13})$ (twice the area is base times height), so $\cot\alpha_1 = (\ell_{12}^2 + \ell_{13}^2 - \ell_{23}^2)/(4A_f)$, and the cotangent weights of Proposition 7.3 are functions of the lengths, as they must be.

20.3. Laying the triangle out in the plane

It is convenient to have actual coordinates, so we draw an isometric copy of the triangle in the plane $\mathbb{R}^2$, its *layout* (Figure 22):

$$\bar{x}_1 = (0,0), \qquad \bar{x}_2 = (\ell_{12}, 0), \qquad \bar{x}_3 = \ell_{13}\,(\cos\alpha_1, \sin\alpha_1). \tag{20.5}$$

It has the right lengths: $|\bar{x}_2 - \bar{x}_1| = \ell_{12}$, $|\bar{x}_3 - \bar{x}_1| = \ell_{13}$, and $|\bar{x}_3 - \bar{x}_2|^2 = \ell_{13}^2 - 2\ell_{12}\ell_{13}\cos\alpha_1 + \ell_{12}^2 = \ell_{23}^2$ by (20.4). It is counterclockwise, because $\sin\alpha_1 > 0$. Its Gram matrix is therefore (20.2), and by Proposition 20.1 any quantity of the method computed on the layout equals the one computed on the face in space.

On the layout we compute the gradients with the formula (16.4) of Part II: the opposite edge, turned a quarter turn counterclockwise, divided by twice the area. In the plane the quarter turn is $J(x, y) = (-y, x)$, so for the counterclockwise order a, b, c,

$$\nabla\phi_a = \frac{J(\bar{x}_c - \bar{x}_b)}{2A_f}. \tag{20.6}$$

The layout also gives the face its frame: the x axis, which is the direction of the first edge, from $\bar{x}_1$ to $\bar{x}_2$. This is the same first frame vector t_f that `computeFaceTangentSpace` of `FEM.cpp` takes. A direction in the face is now simply an angle measured from the x axis; we call β_e the angle of the direction of an edge e of the face in this frame (the code's `edgeAngle`).

Worked example 20.2. *The triangle of Example 7.2, $x_1 = 0$, $x_2 = (2, 0, 0)$, $x_3 = (0, 1, 1)$, has $\ell_{12} = 2$, $\ell_{13} = \sqrt{2}$ and $\ell_{23} = |(-2, 1, 1)| = \sqrt{6}$. By (20.2), $G_f = \begin{pmatrix} 4 & 0 \\ 0 & 2 \end{pmatrix}$, exactly the B^TB of Example 7.2, now obtained without the positions. Heron's formula with $\sigma = (2 + \sqrt{2} + \sqrt{6})/2$ gives $A_f = 1.414214 =$*

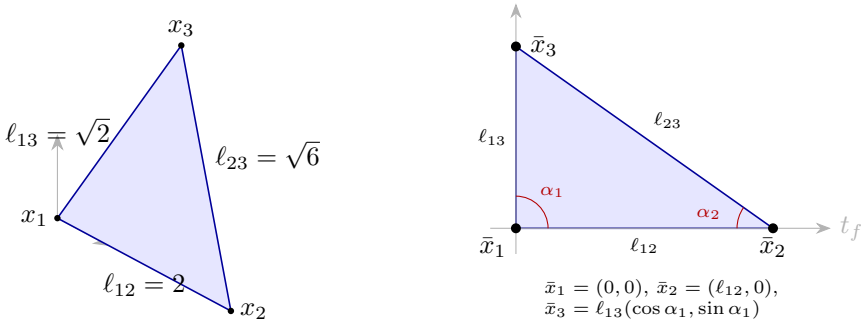


FIGURE 22. Left: the triangle of Example 7.2 in space; Part III reads only its three edge lengths. Right: its layout (20.5), an isometric copy in the plane, with the face frame t_f along the first edge. Every quantity of the method can be computed on the layout.

$\sqrt{2}$, *and (20.4) gives the angles* $\alpha_1 = 90°$, $\alpha_2 = 35.26°$, $\alpha_3 = 54.74°$. *The layout is* $\bar{x}_1 = (0,0)$, $\bar{x}_2 = (2,0)$, $\bar{x}_3 = (0,\sqrt{2})$, *and (20.6) gives*

$$\nabla\phi_1 = (-\tfrac{1}{2}, -\tfrac{1}{\sqrt{2}}), \qquad \nabla\phi_2 = (\tfrac{1}{2}, 0), \qquad \nabla\phi_3 = (0, \tfrac{1}{\sqrt{2}}),$$

with squared lengths $\frac{3}{4}$, $\frac{1}{4}$, $\frac{1}{2}$: *those of the gradients* $(-\frac{1}{2}, -\frac{1}{2}, -\frac{1}{2})$, $(\frac{1}{2}, 0, 0)$, $(0, \frac{1}{2}, \frac{1}{2})$ *of Example 7.2, which are the same vectors seen in a different plane. The stiffness* $A_f\, \nabla\phi_a \cdot \nabla\phi_b$ *is the matrix of Example 7.2, entry for entry (the program prints both).*

21. The Tangent Plane of a Vertex, from Angles

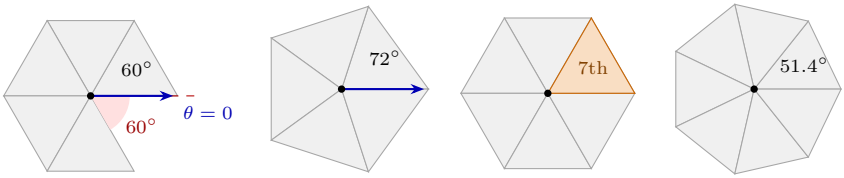
21.1. The cone around a vertex

Inside a face, the tangent plane is the plane of the face, and the layout describes it intrinsically. Across an edge, from one face to its neighbor, parallel transport is the unfolding of one face onto the plane of the other (Section 14), and the unfolding depends only on the two triangles. So faces and edges pose no problem. The problem is the vertex.

Cut the faces around a vertex i along one edge and flatten them one after another into the plane, as in Figure 23. Their angles at i add up to the *total angle*

$$\Theta_i = \sum_{f \ni i} \alpha_{i,f}, \tag{21.1}$$

and there are three cases. If $\Theta_i = 2\pi$, the faces fit exactly: around i the mesh is flat, even if it is folded in space, and the ant cannot tell the vertex from a point of a plane. If $\Theta_i < 2\pi$, there is a gap: the vertex is the tip of a cone, like the apex of a pyramid or a vertex of a mesh of a sphere. If $\Theta_i > 2\pi$,



cone, cut open $\Theta_i = 300°$ | rescaled by $s_i = \frac{6}{5}$ closes up | saddle, cut open $\Theta_i = 420°$: overlaps | rescaled by $s_i = \frac{6}{7}$ closes up

FIGURE 23. The faces around a vertex, cut along the reference edge (blue, $\theta = 0$) and flattened. Five equilateral triangles: $\Theta_i = 300°$, a gap of 60°, the angle defect. Rescaling every angle by $s_i = 2\pi/\Theta_i = 6/5$ closes the gap: every 60° corner becomes 72°. Seven equilateral triangles: $\Theta_i = 420°$, the seventh overlaps the first; rescaling by 6/7 turns every corner into 51.4°.

the faces overlap: the vertex is a saddle, like a point of a ruffled collar. The difference

$$\kappa_i = 2\pi - \Theta_i \tag{21.2}$$

is the *angle defect*, the discrete counterpart of the Gaussian curvature: the total curvature $\int K\, dA$ of the neighborhood of i, concentrated at the vertex [17]. It is intrinsic, a sum of angles of triangles, which is the discrete form of Gauss's *Theorema Egregium*.

The angle defects of a closed mesh add up to $2\pi\chi$, where $\chi = V - E + F$ is the Euler characteristic: this is the discrete *Gauss–Bonnet theorem*, and its proof is a count. The angles of each triangle add up to π, so $\sum_i \Theta_i = \pi F$. Each triangle has three edges and each edge is shared by two triangles, so $E = \frac{3}{2}F$. Then

$$\sum_i \kappa_i = 2\pi V - \pi F = 2\pi\big(V - \tfrac{3}{2}F + F\big) = 2\pi(V - E + F).$$

On a mesh with boundary the same count works with $\pi - \Theta_i$ at the boundary vertices, the discrete geodesic curvature of the boundary. The program checks it on the cactus: the angle defects add up to $12.566371 = 4\pi$, the value of a sphere.

21.2. Why a vertex has no tangent plane

The directions at a point of a smooth surface form a circle, a full turn of 2π. The directions at the vertex i of a mesh, as the ant measures them, form a circle of Θ_i: walking once around i at a small distance, the ant turns by Θ_i, not by 2π. So when $\Theta_i \neq 2\pi$ there is no plane whose directions match those of the vertex, and none is "correct". To build the finite element method we need a plane anyway, because the coordinates of the unknowns at the vertex must be added and multiplied as vectors of one plane. Some map from the circle of Θ_i to a circle of 2π must be chosen, and that choice is a modeling

decision. Part I made it with the vertex normal. Here we make it with the angles alone.

21.3. Directions at a vertex as angles

Choose a reference edge e_0 leaving the vertex: we take the same edge `vi.edge` whose projection is the vertex frame vector t_i of Part I. Walk counterclockwise around the vertex, from one outgoing edge to the next, $e_0, e_1, e_2, \ldots$, and assign to each outgoing edge its *cone angle*, the angle the ant measures from e_0:

$$\theta(e_0) = 0, \qquad \theta(e_{m+1}) = \theta(e_m) + \alpha_m, \tag{21.3}$$

where α_m is the angle at i of the face between e_m and e_{m+1}. Every direction at the vertex points into some corner and has a cone angle θ in $[0, \Theta_i)$; after a full turn θ reaches Θ_i, which is again the direction of e_0. (The program walks clockwise, because that is the order of the half-edge iterator, and subtracts; its angles differ from these by Θ_i, the same direction.)

Now rescale. Define the *scale* of the vertex and the *angle* of a direction in the plane of the vertex,

$$s_i = \frac{2\pi}{\Theta_i}, \qquad \varphi = s_i\,\theta. \tag{21.4}$$

As θ runs over $[0, \Theta_i)$, φ runs over $[0, 2\pi)$: a full turn. This is our tangent plane of the vertex: a plane whose directions are the angles φ, with $\varphi = 0$ along the reference edge. A tangent vector at i is a length and an angle φ, that is, the complex number $\psi = |U|\, e^{\mathsf{T}\varphi}$ of Part II, the spinor coordinate of U in the frame of the reference edge. Rescaling stretches every corner by the same factor s_i, so the corners tile the plane without gap or overlap (Figure 23). The angle defect has not disappeared: it has been spread evenly over the corners, each corner absorbing the share $(s_i - 1)\,\alpha_m$ of it, proportional to its angle. On a fine mesh of a smooth surface $\Theta_i \to 2\pi$, $s_i \to 1$, and the stretching vanishes.

At a boundary vertex the directions do not form a circle but an arc, from the boundary edge on one side to the boundary edge on the other: there is no loop to close, and we do not rescale, $s_i = 1$. (Sharp, Soliman and Crane [17] use $s_i = \pi/\Theta_i$ instead, so that the boundary becomes a straight line in the plane of the vertex. Our choice keeps a flat sheet exactly flat: the corner of a rectangle has $\Theta_i = \pi/2$, and π/Θ_i would double its angles.)

21.4. What the vertex normal does instead

The vertex plane of Part II solves the same problem in another way. Each face is rolled into the plane orthogonal to n_i by the rotor $Q_{i\to f}$ of (13.2). A rotation preserves angles, so every corner keeps its true angle $\alpha_{i,f}$ in the vertex plane, and the rolled corners together cover Θ_i, not 2π. They cannot tile the plane: they leave gaps between neighbors when $\Theta_i < 2\pi$ and overlap when $\Theta_i > 2\pi$, and the gaps add up to the angle defect. Where the gaps fall depends on how each face is tilted with respect to the vertex normal, an extrinsic fact. The rescaling of (21.4) spreads the defect in proportion to the

angles, an intrinsic fact. Even at a flat vertex, $\Theta_i = 2\pi$, the rolled corners fit edge to edge only if all the faces are rolled about one line, as along a straight fold; otherwise they leave gaps and overlaps that cancel in total but not one by one, which is what happens on the Miura-ori of Figure 21. On a symmetric vertex the two planes agree; on any other they differ, as the next two examples show.

Worked example 21.1. *The apex of a pyramid over a regular pentagon of radius* 1*, at the height* $h = 0.6180$ *that makes the five faces equilateral. Each corner at the apex is* $60°$*, so* $\Theta_i = 300°$*,* $\kappa_i = 60°$ *and* $s_i = 1.2$*; the vertex normal is vertical, by symmetry. The program prints, for the corners in counterclockwise order from the reference edge:*

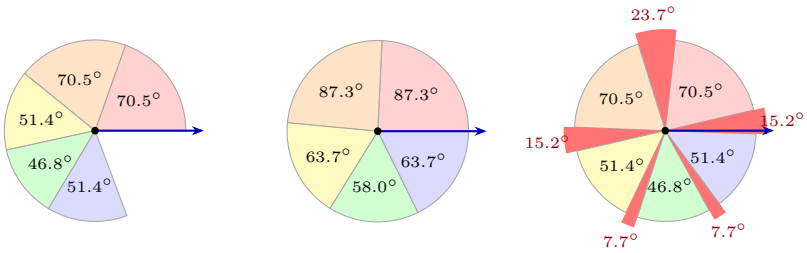
		intrinsic		*extrinsic (rolled)*		
corner	α	*from*	*width*	*from*	*width*	*gap*
0	$60°$	$0°$	$72°$	$6°$	$60°$	$12°$
1	$60°$	$72°$	$72°$	$78°$	$60°$	$12°$
⋮						
4	$60°$	$288°$	$72°$	$294°$	$60°$	$12°$

The intrinsic plane stretches each corner from $60°$ *to* $72°$*. The extrinsic plane keeps each at* $60°$ *and leaves five gaps of* $12°$*, again* $60°$ *in total. The middle directions of the corners, their bisectors, are the same in both:* $36°, 108°, \ldots$ *Since the connection of the next section matches bisectors, the two connections are equal at this vertex; the program finds them equal to rounding.*

Worked example 21.2. *Move the apex of Example 21.1 by* 0.4 *towards one corner of the pentagon. The corners become* $70.49°$*,* $70.49°$*,* $51.37°$*,* $46.81°$*,* $51.37°$*, with* $\Theta_i = 290.53°$*,* $\kappa_i = 69.47°$ *and* $s_i = 1.2391$*. The vertex normal is still vertical: the sum of the cross products* $(x_k - x_i) \times (x_{k+1} - x_i)$ *over the faces, which (2.3) averages, does not depend on* x_i *at all (the terms with* x_i *cancel around the ring), and equals twice the vector area of the pentagon. Figure 24 shows where each method puts the corners:*

		intrinsic		*extrinsic (rolled)*		
corner	α	*from*	*width*	*from*	*width*	*gap*
0	$70.488°$	$0°$	$87.341°$	$13.146°$	$70.488°$	$23.662°$
1	$70.488°$	$87.341°$	$87.341°$	$107.296°$	$70.488°$	$15.181°$
2	$51.373°$	$174.682°$	$63.655°$	$192.964°$	$51.373°$	$7.720°$
3	$46.814°$	$238.337°$	$58.007°$	$252.057°$	$46.814°$	$7.720°$
4	$51.373°$	$296.345°$	$63.655°$	$306.592°$	$51.373°$	$15.181°$

Both spread the defect of $69.47°$ *over the five corners, but differently. The intrinsic plane gives each corner* $(s_i - 1)\alpha$*:* $16.85°$*,* $16.85°$*,* $12.28°$*,* $11.19°$*,* $12.28°$*, in proportion to the angles. The extrinsic plane leaves gaps of* $23.66°$*,* $15.18°$*,* $7.72°$*,* $7.72°$*,* $15.18°$*, which depend on the tilt of the faces: the largest gap opens between the two faces on the side towards which the apex leans, which are tilted most with respect to the vertex normal. The bisectors now differ by* $4.7°, 11.5°, 12.1°, 8.1°$ *and* $4.1°$*, and so do the two connections. For*



cut open and flattened: $\Theta_i = 290.5°$, a gap of 69.5°

intrinsic: every corner stretched by $s_i = 1.2391$

extrinsic: every corner rolled rigidly, gaps in red

FIGURE 24. The leaning pyramid of Example 21.2, seen from its apex, corners counterclockwise from the reference edge (blue). Left: cut open and flattened, with a gap of 69.5°, the angle defect. Middle: the intrinsic vertex plane, every corner stretched by the same factor. Right: the extrinsic vertex plane of Part II, every corner rolled rigidly into the plane orthogonal to the vertex normal; the defect falls into the gaps between them (red), unevenly.

the ant both pyramids are cones of the same kind, and only the intrinsic plane treats them alike.

22. The Connection from a Vertex into a Face

22.1. What the finite element method needs

The element matrix (16.5) uses, for each corner of a face, a unit complex number $e^{\mathsf{T}\rho_i^f}$ with the meaning of (13.3): a vector of angle φ in the plane of vertex i is the vector of angle $\varphi + \rho_i^f$ in the face frame. In words, ρ_i^f is where the reference direction of the vertex, $\varphi = 0$, lands in the face. Transport from vertex to face is a *rotation*, multiplication by $e^{\mathsf{T}\rho_i^f}$, and it must be a rotation: the spinor hat function $\phi_i\, e^{\mathsf{T}\rho_i^f} \psi_i$ of Section 16 must be a rigid copy of the vertex vector in the face, so that the lengths and the angles of vectors, which the mass and the stiffness measure, are those of the vertex.

22.2. The exact correspondence is not a rotation

Look at one corner of the face f at the vertex i. Let e be the first edge of the corner in the counterclockwise order, with cone angle θ_e at the vertex (21.3) and angle β_e in the face frame; let α be the angle of the corner. A direction inside the corner that makes the angle $u \in [0, \alpha]$ with e has

$$\text{in the face:} \quad \beta = \beta_e + u, \qquad \text{at the vertex:} \quad \varphi = s_i(\theta_e + u),$$

the first because the face is flat, the second by (21.4). The rotation that would carry the vertex direction to the face direction is

$$\beta - \varphi = (\beta_e - s_i\theta_e) + (1 - s_i)\, u,$$

which changes across the corner, by $(1 - s_i)\,\alpha$ in total, unless $s_i = 1$. The exact correspondence stretches the corner; it is not a rotation. We need a single number ρ_i^f for the corner.

22.3. The best rotation: match the bisectors

Choose ρ to make the mismatch as small as possible on average over the corner, in the least squares sense:

$$\min_{\rho} \int_0^{\alpha} \big(\rho - (\beta_e - s_i\theta_e) - (1 - s_i)\, u\big)^2\, du.$$

Setting the derivative with respect to ρ to zero, ρ is the mean of the linear function over $[0, \alpha]$, its value at the middle, $u = \alpha/2$:

$$\rho_i^f = \Big(\beta_e + \frac{\alpha}{2}\Big) - s_i\Big(\theta_e + \frac{\alpha}{2}\Big). \tag{22.1}$$

The two brackets are the angle of the bisector of the corner in the face and in the plane of the vertex: the intrinsic connection *matches the bisectors*. The two edges of the corner are then off by $\mp(s_i - 1)\alpha/2$, half the stretching on each side (Figure 25).

For a flat vertex, $s_i = 1$, (22.1) is $\rho_i^f = \beta_e - \theta_e$ with no mismatch at all: the vertex plane is the plane of the flattened faces, and transport into each face is the exact unfolding. So on every part of a mesh that is intrinsically flat, however it is folded in space, the intrinsic connection is the exact parallel transport. This is why it reproduces the field of the flat paper on the Miura-ori of Figure 21.

Worked example 22.1. *In the cone of five equilateral triangles, the corner of Figure 25 has* $\theta_e = 60°$ *and* $\alpha = 60°$*; with the face frame along its first edge,* $\beta_e = 0$*. Then (22.1) gives* $\rho_i^f = (0{+}30°){-}1.2\,(60°{+}30°) = 30° {-} 108° = -78°$*: the reference direction of the vertex lands at* $-78°$ *in the face, and the bisector of the vertex sector, at* $108°$*, lands at* $108° - 78° = 30°$*, the bisector of the face corner.*

22.4. Going around the vertex

The transport from a face f into its neighbor g around the vertex i can be done in two ways: by unfolding g onto f across their common edge, the exact transport, or through the plane of the vertex, from f to the vertex with $e^{-\mathsf{T}\rho_i^f}$ and from the vertex into g with $e^{\mathsf{T}\rho_i^g}$, which is what the finite element method does. Unfolding preserves the cone angle: a direction of f at the cone angle θ is carried to the direction of g at the same cone angle. Through the vertex, by (22.1), the direction is carried with the extra rotation

$$(1 - s_i)\,\big[(\theta_g + \tfrac{1}{2}\alpha_g) - (\theta_f + \tfrac{1}{2}\alpha_f)\big] = (1 - s_i)\,\tfrac{1}{2}(\alpha_f + \alpha_g),$$

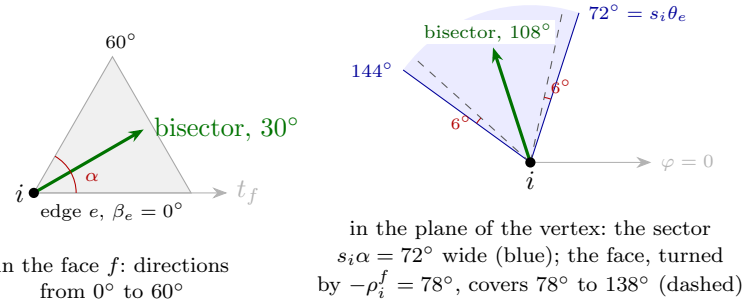


FIGURE 25. The second corner of the cone of Figure 23 ($\theta_e = 60°$, $\alpha = 60°$, $s_i = 1.2$) with the face frame along its first edge, $\beta_e = 0$. Left: in the face, the directions of the corner run from 0° to 60°. Right: in the plane of the vertex, the corner is the sector from 72° to 144°. The connection (22.1), $\rho_i^f = 30° - 108° = -78°$, carries bisector to bisector; the face's edges land 6° inside the sector on each side.

$(1 - s_i)$ times the cone angle from the bisector of f to that of g. Going once around the vertex, the cone angles between consecutive bisectors add up to Θ_i, so the extra rotations add up to $(1 - s_i)\,\Theta_i = \Theta_i - 2\pi = -\kappa_i$. The unfolding, on the other hand, comes back rotated by the holonomy of the vertex, the angle defect κ_i (Section 14). The two cancel: through the plane of the vertex, a vector comes back unchanged, as it must, since the vertex plane is one plane. The vertex plane does not ignore the curvature; it accounts for it by correcting the transport between consecutive faces, each by a share of the defect proportional to the angle crossed.

In the language of moving frames (Section 42), this part uses a frame that is constant on each face, the x axis of its layout, and one frame at each vertex, the reference edge of its stretched cone. The connection form vanishes inside every face, and the corner angles ρ_i^f are the *transition functions* between the frame of a vertex and the frames of its faces, the discrete counterpart of the change of frame (15.3).

23. The Finite Element Method, Unchanged

With (20.3), (20.6) and (22.1), the element matrix (16.5) is computed without leaving the surface:

$$\mathrm{L}_{ij}^f = A_f\,(\nabla\phi_i \cdot \nabla\phi_j)\; e^{\mathsf{T}(\rho_j^f - \rho_i^f)}, \qquad \mathrm{M}_{ii}^f = \frac{A_f}{3}, \tag{23.1}$$

with A_f from Heron's formula, the gradients from the layout, and the connection from the bisectors. The rest is Part II without a single change: the assembly into the $n \times n$ Hermitian matrix, the scaling by the lumped mass,

the Dirichlet row of the source, the sparse LU factorization and the solution. The transport between two vertices of a face is still $e^{\mathsf{T}(\rho_j^f - \rho_i^f)}$, in which the arbitrary angle of the face frame cancels.

The same gauge as Part I.. The reference direction $\varphi = 0$ of a vertex is its reference edge, the same edge whose projection is the frame vector t_i of Part I. So both methods read a spinor coordinate as an angle from the same edge, and the value prescribed at the source, $e^{\mathsf{T}\,2\pi/3}$, means in both "120° from the reference edge" in each method's own vertex plane. The two solutions can be compared number by number, as Section 24 does.

Drawing the result. The positions of the vertices return only at the very end, to draw. To draw the coordinate ψ_i, take the cone angle $\theta = \arg(\psi_i)/s_i$, find the corner that contains it, and draw the direction in the plane of that face, turned from the first edge of the corner by $\theta - \theta_e$ (the function `lift`).

The code. `IntrinsicConnection.h` computes the geometry. Heron's formula (20.3) and the law of cosines (20.4):

```
/// Heron's formula (eq. heron): the area of a triangle from its three edge lengths.
inline double heronArea(double l0, double l1, double l2) {
    const double s = 0.5 * (l0 + l1 + l2);
    return std::sqrt(std::max(0.0, s * (s - l0) * (s - l1) * (s - l2)));
}

/// The law of cosines (eq. law_of_cosines): the angle opposite to the edge of length c
      in a triangle with sides a, b, c.
inline double cornerAngle(double a, double b, double c) {
    return std::acos(std::max(-1.0, std::min(1.0, (a * a + b * b - c * c) / (2.0 * a *
          b))));
}
```

The layout (20.5), the angles and the gradients (20.6) of a face:

```
/// Section 20: lay a face out in the plane from its lengths alone and compute its area
     , angles and gradients.
inline void layOut(FaceGeometry& g) {
    const double l01 = g.length[0], l12 = g.length[1], l20 = g.length[2];
    g.angle[0] = cornerAngle(l01, l20, l12);
    g.angle[1] = cornerAngle(l01, l12, l20);
    g.angle[2] = M_PI - g.angle[0] - g.angle[1];
    g.area = heronArea(l01, l12, l20);
    g.x[0] = Vec2(0, 0);
    g.x[1] = Vec2(l01, 0);
    g.x[2] = l20 * Vec2(std::cos(g.angle[0]), std::sin(g.angle[0]));
    for (int a = 0; a < 3; ++a) {
        const Vec2 e = g.x[(a + 1) % 3] - g.x[a];
        g.edgeAngle[a] = std::atan2(e.y(), e.x());
        // the opposite edge, from v[a+1] to v[a+2], turned a quarter turn
              counterclockwise, over twice the area
        const Vec2 opposite = g.x[(a + 2) % 3] - g.x[(a + 1) % 3];
        g.grad[a] = Vec2(-opposite.y(), opposite.x()) / (2.0 * g.area);
    }
}
```

The walk around each vertex (21.3), its scale (21.4), and the connection (22.1) of every corner:

```
    // Section 21: walk clockwise around each vertex from its reference edge,
         accumulating the cone angle
    G.vertices.resize(mesh.numVertices());
    for (Vertex& vi : mesh.getVertices()) {
        const Edge* e = vi.edge;
```

```
        double theta = 0.0;
        G.outAngle[e] = theta;
        bool boundary = false;
        while (true) {
            const Edge* pair = e->pair;
            if (!pair->face) { boundary = true; break; } // past the last face of a boundary vertex
            const Edge* next = pair->next; // the next outgoing half edge, clockwise
            theta -= angleAtOrigin(next); // the face between e and next lies to the left of next
            if (next == vi.edge) break;
            G.outAngle[next] = theta;
            e = next;
        }
        VertexGeometry& g = G.vertices[vi.ID];
        g.totalAngle = -theta;
        g.boundary = boundary || !vi.edge->face;
        g.scale = g.boundary ? 1.0 : 2.0 * M_PI / g.totalAngle;
    }
    // eq. intrinsic_rho: compare the bisector of each corner in the face and in the plane of the vertex
    for (Face& f : mesh.getFaces()) {
        Vertex* v[3];
        Edge* h[3];
        faceCorners(f, v, h);
        FaceGeometry& g = G.faces[f.ID];
        for (int a = 0; a < 3; ++a) {
            const double s = G.vertices[v[a]->ID].scale;
            const double theta = G.outAngle.at(h[a]); // the corner spans [theta, theta + angle] in the cone
            const double inFace = g.edgeAngle[a] + 0.5 * g.angle[a];
            const double inVertex = s * (theta + 0.5 * g.angle[a]);
            g.rho[a] = std::polar(1.0, inFace - inVertex);
        }
    }
```

The assembly, line by line that of `gaform::solve` of Section 17, with the three numbers of each corner taken from the intrinsic geometry:

```
/// Section 23: the assembly of gaform::solve, with the gradients, areas and connection of IntrinsicConnection.h.
static Eigen::VectorXcd solve(Mesh& mesh, const intrinsic::Geometry& G, double dt, int source, Complex value) {
    const int n = mesh.numVertices();
    std::vector<double> mass(n, 0.0);
    std::map<std::pair<int, int>, Complex> A;
    for (Face& f : mesh.getFaces()) {
        Vertex* v[3];
        Edge* h[3];
        intrinsic::faceCorners(f, v, h);
        const intrinsic::FaceGeometry& g = G.faces[f.ID];
        for (int a = 0; a < 3; ++a) {
            mass[v[a]->ID] += g.area / 3.0;
            for (int b = 0; b < 3; ++b) {
                const Complex transport = a == b ? Complex(1.0) : g.rho[b] * std::conj(g.rho[a]);
                A[{ v[a]->ID, v[b]->ID }] += dt * g.area * g.grad[a].dot(g.grad[b]) * transport;
            }
        }
    }
    std::vector<Eigen::Triplet<Complex>> triplets;
    for (const auto& [ij, value_ij] : A) {
        if (ij.first == source) continue;
        triplets.emplace_back(ij.first, ij.second, value_ij / mass[ij.first] + (ij.first == ij.second ? 1.0 : 0.0));
    }
```

```
    triplets.emplace_back(source, source, 1.0);
    Eigen::SparseMatrix<Complex> S(n, n);
    S.setFromTriplets(triplets.begin(), triplets.end());
    Eigen::VectorXcd rhs = Eigen::VectorXcd::Zero(n);
    rhs(source) = value;
    Eigen::SparseLU<Eigen::SparseMatrix<Complex>> lu(S);
    return lu.solve(rhs);
}
```

24. Experiments

`ga_fem` runs the vector heat method with both connections, the extrinsic one of Part II and the intrinsic one of this part, on three folded sheets, five spheres and the cactus, with the same time step $\Delta t = h^2$ (h the mean edge length) and the same solver.

How the error is measured. A vertex has no tangent plane (Section 21), so a vector at a vertex cannot be compared with an exact vector without choosing one, and any choice would favor one of the two methods. We compare in the faces instead. For each corner (i, f), each method puts the vector of its solution at i into the face f with its own connection, $t_f\, e^{\mathsf{T}(\arg u_i + \rho_i^f)}$: this is the vector the finite element method actually uses in that face. We measure its angle to the exact vector of the face, at every corner of every face. The exact vectors are computed independently of both methods: on a flat or folded sheet, by carrying the source vector from face to face across the edges with the rotation about the shared edge, the unfolding of Section 14, which on an intrinsically flat mesh does not depend on the path; on the sphere, by transport along the great circle to the vertex, followed by the smallest rotation from the tangent plane of the sphere into the face. The source vector is the vector each method puts into the first face of the source vertex.

Folded sheets: exact bendings. Three sheets, all intrinsically flat, every interior angle sum 2π to 10^{-13} degrees: a flat sheet of 48×16 squares, each cut by a diagonal into two right isosceles triangles (833 vertices); the same sheet folded along every fourth column of vertices by $30°$, into twelve flat strips, almost a closed twelve-sided prism (parallel folds); and the Miura-ori of Figure 21, 24×16 parallelograms, $X_{jk} = P_j + Q_k$ with the zigzags $P_j = (ju, 0, (j \bmod 2)\, w)$ and $Q_k = ((k \bmod 2)\, r, kt, 0)$. (Every face of such a net is a parallelogram, and at every vertex the four angles come in two pairs $\arccos(x) + \arccos(-x) = \pi$, so they add up to 2π exactly.) The exact answer on each is the field of the flat paper.

	extrinsic, mean	extrinsic, max	intrinsic, max
flat sheet	$7 \cdot 10^{-15}$	$5 \cdot 10^{-14}$	$7 \cdot 10^{-13}$
parallel folds	$1 \cdot 10^{-14}$	$4 \cdot 10^{-14}$	$3 \cdot 10^{-13}$
Miura-ori	$7.42°$	$21.9°$	$2 \cdot 10^{-13}$

The intrinsic method is exact on all three, as Section 22 predicts: every vertex has $s_i = 1$, and every transport into a face is an unfolding. The extrinsic method is exact on the parallel folds too, because there every face is rolled into

the vertex plane about the fold line, and rolling about the fold is unfolding. On the Miura-ori the faces around a vertex are rolled about four different axes, the rolled corners keep their angles but do not fit edge to edge, and the transports are wrong by several degrees at every vertex; the errors add up along the paths from the source.

The sphere: convergence. On the geodesic spheres of Example 10.3, refined from 42 to 10242 vertices, we measure at every corner within three quarters of the way to the antipode (the antipode itself is the one point where the shortest path is not unique):

			extrinsic		intrinsic	
vertices	h	largest κ_i	mean	max	mean	max
42	0.5823	17.72°	0.277°	1.340°	0.331°	1.502°
162	0.2993	4.71°	0.209°	0.888°	0.210°	0.881°
642	0.1507	1.30°	0.111°	0.473°	0.111°	0.492°
2562	0.0755	0.34°	0.067°	0.307°	0.067°	0.306°
10242	0.0378	0.08°	0.079°	0.506°	0.079°	0.504°

On a curved surface neither connection is exact, and the two are practically equal from 162 vertices on (Figure 26, left). Only on the coarsest sphere, whose vertices are cones with angle defects up to 17.7°, is the extrinsic one slightly better. The vertices of a geodesic sphere lie on the exact sphere, and their averaged normal is very nearly its exact normal, so Part II's vertex plane is nearly the true tangent plane of the smooth surface: information about the smooth sphere which the edge lengths do not contain, and which the intrinsic method, by design, does not use. As the mesh is refined the defects shrink, $s_i \to 1$, and the two vertex planes converge to the same plane. The slight increase of the error on the finest sphere is shared by both, so it belongs to the vector heat method with the time step $\Delta t = h^2$, not to the connection.

The cactus: how large is the choice? On the cactus of Figure 15 there is no exact answer, but we can measure how much the two connections disagree, corner by corner, with the same source value. The angle defects add up to 4π, as the discrete Gauss–Bonnet theorem requires for a surface of genus zero; the largest is 63.7°, a sharp vertex. The angle between the two solutions has median 2.20°, 90% of the corners are below 6.15°, and the largest difference is 23.4°, at a vertex where the field has decayed to $|u_i| = 1.1 \cdot 10^{-7}$. Figure 26, right, shows that the large differences occur only where the field is small, on the far side of the cactus, where several shortest paths of nearly equal length compete and a small change of the connection decides between them. The few degrees elsewhere measure the size of the modeling choice that a coarse, strongly curved mesh leaves open at its vertices.

25. Summary: Two Ways to Give a Vertex a Plane

Table 3 lists what changes between Part II and this part. Every ingredient of a face, the area, the gradients, the stiffness, was intrinsic all along, hidden in the Gram matrix $B^T B$. The one real difference is the tangent plane of

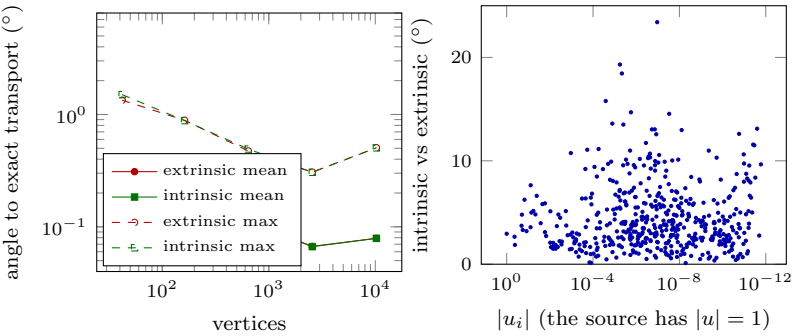


FIGURE 26. Left: the error of the vector heat method on geodesic spheres, against the number of vertices, with the extrinsic connection of Part II (red) and the intrinsic one (green); mean (solid) and maximum (dashed), over all corners within three quarters of the way to the antipode. Right: on the cactus, the largest angle between the two solutions over the corners of each vertex, against the magnitude $|u_i|$ of the intrinsic solution: the two disagree strongly only where the field has almost vanished.

TABLE 3. The ingredients of the element matrix in the extrinsic method of Part II and in the intrinsic method of Part III.

Ingredient	Extrinsic (Part II)	Intrinsic (Part III)
input	positions x_i, normals	edge lengths ℓ_{ij}
face geometry	B, 3×2	metric $G_f = B^T B$ from lengths, (20.2)
area	$\frac{1}{2}\lvert e_1 \times e_2 \rvert$	Heron, (20.3)
gradients	reciprocal frame in space	quarter turn in the layout, (20.6)
face frame	first edge, in space	first edge, the x axis of the layout
vertex plane	orthogonal to the vertex normal n_i	angles rescaled by $s_i = 2\pi/\Theta_i$, (21.4)
where the defect goes	gaps between rolled corners, by the tilt of the faces	into every corner, in proportion to its angle
connection ρ_i^f	$e^{\mathsf{T}\rho} = t_f\, Q t_i \widetilde{Q}$, (13.3)	bisector to bisector, (22.1)
intrinsically flat parts	exact for parallel folds only	exact
undefined when	$n_f = -n_i$	never (non-degenerate triangles)

a vertex, which a mesh does not have and which each method must invent: Part II invents it from the vertex normal and rolls each face rigidly into it, leaving the angle defect in the gaps; this part invents it from the angles, stretching every corner by the same factor $2\pi/\Theta_i$. The connection then follows by matching the bisectors of the corners.

When to use which? The intrinsic connection is the natural one when the geometry is given by lengths, when the mesh is bent or folded (exactly, on every intrinsically flat part), when vertex normals are poor (thin or sharp features, noisy meshes), and it is the form used in geometry processing [10,17]; it needs no rotor of space and has no singular configuration. The extrinsic connection uses one more piece of information, the vertex normal, which can pay off slightly when the mesh coarsely samples a smooth surface whose normals it approximates well, as on the coarsest sphere above. On fine meshes of smooth surfaces the two agree, and their cost is the same.

Remark 25.1. *Readers who know the gauge theory of gravity in geometric algebra [6] will recognize the two halves of this part. The metric $G_f = B^T B$, a rule for measuring lengths in coordinates, is what that theory encodes in its* displacement gauge *field, the map from flat coordinates to the curved surface; the connection angle ρ, the rule for comparing frames, is its* rotation gauge $\omega\mathsf{T}$ *of Section 15. On a mesh, the first is flat inside each face and given by three lengths, and the second is flat inside each face and carries all the curvature at the vertices, as the angle defects. Nothing in this part depends on that language.*

Part IV. Curvature on the Faces: the Connection as a Discrete 1-Form

Parts I to III carry a vector from a vertex into a face, and from there into the other vertices of the face. Inside a face that transport is flat, so going around a face changes nothing, and all the curvature of the mesh is left at the vertices, as the angle defects. This part does the opposite. It keeps the unknowns at the vertices, in the planes of Part III, but carries vectors directly from vertex to vertex, along the edges. A face then becomes a closed loop of three edges, and a vector carried around it comes back turned: the curvature moves from the vertices into the faces.

In the language of differential forms this is natural. The edge transports are one number per oriented edge, a discrete 1-form, the *connection form*; the turning around a face is the sum of the 1-form around the boundary of the face, its exterior derivative, the *curvature* 2-form; and Gauss–Bonnet becomes the theorem of Stokes. Section 26 builds the edge transport and its curvature. The finite element method then needs a new element, because a face with curvature has no flat frame: Section 27 derives it, slowly. Part V will show that discrete exterior calculus is this element with the curvature terms dropped.

26. Parallel Transport Along the Edges

26.1. The transport along one edge

An edge is a straight segment, and it lies in both of its faces: when the two faces are unfolded into one plane, the edge is a straight line in that plane. Parallel transport along a straight line in a plane keeps the angle between

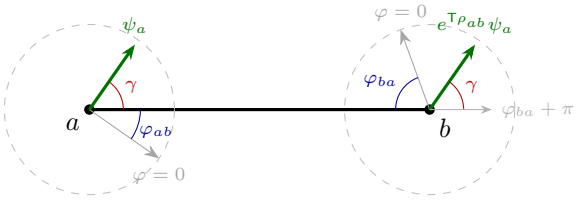


FIGURE 27. Transport along the edge from a to b keeps the angle γ between the vector and the edge. Each vertex measures angles from its own reference direction ($\varphi = 0$, gray); the edge points along φ_{ab} at a and along $\varphi_{ba}+\pi$ at b, so the transport turns the coordinate by $\rho_{ab} = \varphi_{ba}+\pi-\varphi_{ab}$.

the vector and the line. So the natural transport along the edge from a to b is: *keep the angle to the edge* (Figure 27).

In the vertex planes of Part III, the edge leaves a in the direction φ_{ab}, the rescaled angle of the half edge $a \to b$, (21.4). A vector of angle θ at a makes the angle $\gamma = \theta - \varphi_{ab}$ with the edge. At b, the edge leaves in the direction φ_{ba}, back towards a; the direction of travel, away from a, is $\varphi_{ba}+\pi$. The transported vector makes the same angle γ with the direction of travel, so its angle at b is $\varphi_{ba}+\pi+\gamma = \theta+\rho_{ab}$, with

$$\rho_{ab} = \varphi_{ba} + \pi - \varphi_{ab}. \tag{26.1}$$

In spinor coordinates the transport is the multiplication $\psi \mapsto r_{ab}\,\psi$, $r_{ab} = e^{\mathsf{T}\rho_{ab}}$. Transporting back undoes it: $\rho_{ba} = \varphi_{ab}+\pi-\varphi_{ba} = -\rho_{ab}+2\pi$, so $r_{ba} = \overline{r_{ab}}$. One number per edge, which changes sign with the orientation of the edge: this is what a discrete 1-form is.

26.2. Going around a face: the holonomy

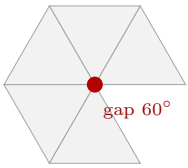
Carry a vector around the face f with vertices a, b, c in counterclockwise order: from a to b, to c and back to a. It comes back multiplied by $e^{\mathsf{T}\Omega_f}$, with $\Omega_f = \rho_{ab}+\rho_{bc}+\rho_{ca}$, the *holonomy* of the face. Collect the terms of (26.1) vertex by vertex:

$$\Omega_f = (\varphi_{ac}-\varphi_{ab}) + (\varphi_{ba}-\varphi_{bc}) + (\varphi_{cb}-\varphi_{ca}) + 3\pi.$$

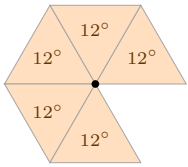
Each bracket is the angle, in the plane of one vertex, between the two edges of its corner: at a, the edge $a \to b$ comes first counterclockwise, then $a \to c$, and by (21.3) and (21.4) $\varphi_{ac}-\varphi_{ab} = s_a\alpha_a$, the rescaled angle of the corner. So $\Omega_f = s_a\alpha_a + s_b\alpha_b + s_c\alpha_c + 3\pi$, and since $\alpha_a+\alpha_b+\alpha_c = \pi$,

$$\Omega_f = (s_a-1)\,\alpha_a + (s_b-1)\,\alpha_b + (s_c-1)\,\alpha_c \qquad (\text{up to } 4\pi, \text{ two full turns}). \tag{26.2}$$

If the three vertices are flat, $s = 1$, the face has no holonomy, as a triangle of the plane should not. Otherwise each vertex v gives the face the share $(s_v-1)\alpha_v$ of its angle defect: summing over the faces around v, the shares add up to $(s_v-1)\Theta_v = 2\pi - \Theta_v = \kappa_v$. So the face holonomies are the



Parts II and III: the curvature sits at the apex, $\kappa_i = 60°$



Parts IV and V: each face holds $\Omega_f = (s_i - 1)\,\alpha = 12°$

FIGURE 28. The upright pyramid of Example 21.1, cut open. With the corner connections of Parts II and III the curvature is at the apex, the angle defect 60°. With the edge connection each face holds $(s_i - 1)\alpha = 0.2 \cdot 60° = 12°$ of it, (26.2).

vertex curvatures of Part III, redistributed to the faces in proportion to the angles (Figure 28). Adding them over a closed mesh gives $\sum_f \Omega_f = \sum_v \kappa_v = 2\pi\chi$: the discrete Gauss–Bonnet theorem again. In the language of forms, the holonomy is the sum of the connection 1-form around the boundary of the face, which is how the exterior derivative of a discrete 1-form is defined (Section 30): $\Omega = d\rho$, and $\sum_f \Omega_f = 2\pi\chi$ is what Stokes's theorem says about it. The program checks on the cactus that the face holonomies add up to 4π; the largest is 34.5°.

Worked example 26.1. *On the upright pyramid of Example 21.1, take the face with the apex* 0 *and the base vertices* 1 *and* 2*. The program prints the angles of the half edges in the vertex planes and the transports (26.1):*

edge	φ_{ab}	φ_{ba}	$\rho_{ab} = \varphi_{ba} + 180° - \varphi_{ab}$
$0 \to 1$	72°	300°	$408° \equiv 48°$
$1 \to 2$	240°	0°	$-60°$
$2 \to 0$	300°	144°	24°

The three transports add up to $\Omega_f = 12°$*, which is (26.2): the apex has* $s = 1.2$ *and the corner* 60°*, the base vertices are on the boundary,* $s = 1$*, so* $\Omega_f = 0.2 \cdot 60°$*. The five faces hold* $5 \cdot 12° = 60°$*, the angle defect of the apex.*

27. A Face with a Curved Connection

27.1. Why a new element is needed

The elements of Parts I to III carry the three vertex values into one flat frame of the face and interpolate them there. With the edge transports that is impossible: a flat frame would make the transports around the face compose to the identity, but they compose to $e^{\mathsf{T}\Omega_f}$. To interpolate consistently with the edge transports, the face itself must be given a connection with total

curvature Ω_f. The simplest choice spreads it evenly, with the constant density

$$\kappa = \frac{\Omega_f}{A_f}. \tag{27.1}$$

This curved connection lives on a flat triangle and is not its Levi-Civita connection: it is a model, the one that agrees with the edge transports, as a small piece of a sphere ($\kappa > 0$) or of a saddle ($\kappa < 0$) would.

27.2. A connection with constant curvature in the plane

Work in the layout of the face, Section 20, with the face frame along the x axis, and write vectors of the face by their complex coordinates u. A connection is a rule $D_X u = X \cdot \nabla u - \mathsf{T}\,(A \cdot X)\,u$, the ordinary derivative corrected by an infinitesimal rotation, where A is a vector field, the *connection 1-form* of the face; it is the form (15.2) of Part II, with $\omega(X) = -A \cdot X$. A field is parallel along a curve $x(t)$ when $D_{\dot x} u = 0$, that is, $\dot u = \mathsf{T}\,(A \cdot \dot x)\,u$, so

$$u(\text{end}) = e^{\mathsf{T} \int A \cdot dx}\, u(\text{start}).$$

The curvature of the connection is the curl of A, $\partial_x A_y - \partial_y A_x$: by Stokes's theorem the rotation around a closed curve is $\oint A \cdot dx = \int (\operatorname{curl} A)\, dA$. A constant curvature κ is obtained with

$$A(x) = \frac{\kappa}{2}\, J(x - c), \qquad J(x, y) = (-y, x), \tag{27.2}$$

for any point c, since $\operatorname{curl} A = \frac{\kappa}{2} + \frac{\kappa}{2} = \kappa$. (A physicist recognizes the vector potential of a uniform magnetic field in the symmetric gauge: Part II already noted that the connection Laplacian is a magnetic Laplacian.) Along the straight segment from p to q, $x = p + t(q - p)$, the integrand $A \cdot dx = \frac{\kappa}{2}\, J(p - c) \cdot (q - p)\, dt$ is constant, and $J(p - c) \cdot (q - p) = (p - c) \times (q - c)$, twice the signed area of the triangle c, p, q. So

$$\text{transport from } p \text{ to } q \text{ along the segment} = e^{\mathsf{T} \kappa\, \mathcal{A}(c,p,q)}, \tag{27.3}$$

where $\mathcal{A}(c, p, q)$ is the signed area, positive when c, p, q are counterclockwise. Around the face, the three signed areas $\mathcal{A}(c, \bar x_a, \bar x_b)$, $\mathcal{A}(c, \bar x_b, \bar x_c)$, $\mathcal{A}(c, \bar x_c, \bar x_a)$ add up to A_f for any c, and the holonomy is $\kappa A_f = \Omega_f$, as required (Figure 29).

27.3. Matching the vertex planes

The vertex values ψ_a live in the vertex planes. In the face they become $u_a = e^{\mathsf{T}\sigma_a}\psi_a$, with one angle σ_a per corner, like ρ_i^f of Part III. The angles must make the two transports along each edge agree: carrying ψ_a to b along the edge and then into the face must give the same as carrying it into the face and then along the segment, $e^{\mathsf{T}\sigma_b} r_{ab}\, \psi_a = e^{\mathsf{T}\kappa \mathcal{A}(c,\bar x_a,\bar x_b)} e^{\mathsf{T}\sigma_a} \psi_a$, that is,

$$\sigma_b - \sigma_a = \kappa\, \mathcal{A}(c, \bar x_a, \bar x_b) - \rho_{ab} \tag{27.4}$$

for the three edges. Adding the three equations gives $0 = \kappa A_f - \Omega_f$, which holds by (27.1): the equations are consistent, and they fix the σ_a up to one common angle, the arbitrary angle of the face frame, which will cancel.

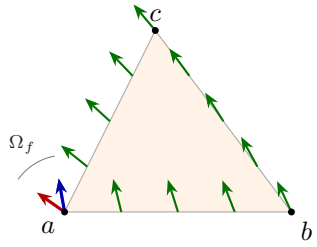


FIGURE 29. A vector (blue) carried around a face with the curved connection (27.2), here with an exaggerated $\Omega_f = 45°$ and c at the centroid: along each edge it turns steadily, by a third of Ω_f, and it comes back to a turned by Ω_f (red).

27.4. The trial functions

Inside the face, interpolate the vertex values after carrying each one to the point x along the straight segment from its vertex:

$$u(x) = \sum_a \phi_a(x)\, e^{\mathsf{T}\theta_a(x)}\, u_a, \qquad \theta_a(x) = \kappa\, \mathcal{A}(c, \bar{x}_a, x). \tag{27.5}$$

At the vertex b it takes the value u_b, since $\phi_a(\bar{x}_b) = \delta_{ab}$ and $\theta_b(\bar{x}_b) = 0$. On the edge from a to b only ϕ_a and ϕ_b are nonzero, and if the two values agree, u_b being the transport of u_a, the interpolant is the parallel field along the edge (the two transported values coincide on the segment, because the signed areas add up along it). So the trace of u on an edge depends only on the two vertex values and the transport of that edge, which both faces share: the element is conforming. (Along a curve a connection can always be written without rotation, so the two faces' different frames on the edge are related by a change of gauge.) In Cartan's terms (Section 42), the face carries the frame of its layout with the connection form $\omega(X) = -A{\cdot}X$, whose curvature is $d\omega = -\operatorname{curl} A\; dA = -\kappa\, dA$: a moving frame whose curvature is prescribed, rather than computed from a curved geometry. The angles σ_a are the transitions from the vertex frames into it, and $\theta_a(x)$ is the angle of parallel transport in it. Section 42 replaces the prescribed curvature by the true curvature of a curved element.

27.5. The covariant derivative of a trial function

By the product rule, with $D = \nabla - \mathsf{T}A$,

$$D\big(\phi_a e^{\mathsf{T}\theta_a}\big) = e^{\mathsf{T}\theta_a}\big(\nabla\phi_a + \mathsf{T}\phi_a\,(\nabla\theta_a - A)\big).$$

Now $\theta_a(x) = \frac{\kappa}{2}(\bar{x}_a - c) \times (x - c)$ is linear in x, with gradient $\frac{\kappa}{2}J(\bar{x}_a - c)$, and by (27.2)

$$\nabla\theta_a - A = \frac{\kappa}{2}J(\bar{x}_a - c) - \frac{\kappa}{2}J(x - c) = \frac{\kappa}{2}J(\bar{x}_a - x) =: w_a(x), \tag{27.6}$$

which no longer depends on c. It vanishes at the vertex a: near its own vertex, the carried value is parallel.

27.6. The element matrix

The Dirichlet energy of the face is $E_f = \int_f |Du|^2\, dA = \sum_{a,b} \overline{\psi_a}\, \mathrm{L}^f_{ab}\, \psi_b$, with

$$\mathrm{L}^f_{ab} = \int_f \overline{e^{\mathsf{T}(\sigma_a+\theta_a)}}\; e^{\mathsf{T}(\sigma_b+\theta_b)}\; \overline{(\nabla\phi_a + \mathsf{T}\phi_a w_a)} \cdot (\nabla\phi_b + \mathsf{T}\phi_b w_b)\, dA.$$

The phase is $\sigma_b - \sigma_a + \theta_b - \theta_a$. By (27.4) and (27.5) it equals

$$\kappa\big[\mathcal{A}(c,\bar{x}_a,\bar{x}_b) + \mathcal{A}(c,\bar{x}_b,x) - \mathcal{A}(c,\bar{x}_a,x)\big] - \rho_{ab} = \kappa\,\mathcal{A}(\bar{x}_a,\bar{x}_b,x) - \rho_{ab},$$

because the signed areas of the three triangles that c makes with the sides of the triangle $\bar{x}_a, \bar{x}_b, x$ add up to the area of that triangle. The point c is gone, and with $-\rho_{ab} \equiv \rho_{ba}$,

$$\boxed{\mathrm{L}^f_{ab} = e^{\mathsf{T}\rho_{ba}} \int_f e^{\mathsf{T}\kappa\,\mathcal{A}(\bar{x}_a,\bar{x}_b,x)}\; \overline{(\nabla\phi_a + \mathsf{T}\phi_a w_a)} \cdot (\nabla\phi_b + \mathsf{T}\phi_b w_b)\; dA} \qquad (27.7)$$

The factor in front, $e^{\mathsf{T}\rho_{ba}}$, carries ψ_b into the plane of a along their common edge; the integral is the curved element in the frame of the face. Three properties follow at once.

1. The matrix is Hermitian, $\mathrm{L}^f_{ba} = \overline{\mathrm{L}^f_{ab}}$: exchanging a and b conjugates both the transport and the integrand, since $\mathcal{A}(\bar{x}_b,\bar{x}_a,x) = -\mathcal{A}(\bar{x}_a,\bar{x}_b,x)$.
2. Without curvature, $\kappa = 0$, the integral is $A_f\, \nabla\phi_a \cdot \nabla\phi_b$, and $\mathrm{L}^f_{ab} = e^{\mathsf{T}\rho_{ba}} A_f\, \nabla\phi_a \cdot \nabla\phi_b$: the element of Part III with the edge transport in place of the corner connections.
3. On the diagonal, $\mathrm{L}^f_{aa} = \int_f |\nabla\phi_a|^2 + \phi_a^2 |w_a|^2\, dA$: the curvature costs energy, $\frac{\kappa^2}{4}\int_f \phi_a^2\, |\bar{x}_a - x|^2\, dA$, the "diamagnetic" term of the magnetic Laplacian.

The mass matrix is lumped as before, $\mathrm{M}_{aa} = A_f/3$, and the assembly, the Dirichlet row and the solution are those of Part II.

Computing the integral. The integrand is smooth: a polynomial of degree two in x times $e^{\mathsf{T}\kappa\mathcal{A}}$, whose phase is linear in x. We integrate it with a Gauss rule on the triangle, seen as a square with one side collapsed: the barycentric coordinates $(1-s,\ s(1-t),\ st)$ with $s,t \in [0,1]$, the area element $2A_f\, s\, ds\, dt$, and 8 Gauss–Legendre points in each of s and t. On the cactus face with the largest holonomy, 34.5°, the rule agrees with a 16×16 rule to $7 \cdot 10^{-16}$.

Worked example 27.1. *For the face of Example 26.1, an equilateral triangle of area* 0.5984 *with* $\Omega_f = 12° = 0.2094$, $\kappa = 0.35$. *The program prints the integrals of (27.7), without the transports in front:*

$$\begin{pmatrix} 0.57805 & -0.28587 - 0.05025\mathsf{T} & -0.28587 + 0.05025\mathsf{T} \\ \cdot & 0.57805 & -0.28587 - 0.05025\mathsf{T} \\ \cdot & \cdot & 0.57805 \end{pmatrix},$$

the lower triangle by conjugation. The flat element of the same triangle has 0.57735 *on the diagonal and* -0.28868 *off it. The curvature adds* 0.0007 *to the diagonal and turns each off-diagonal entry by about* 10°, *and its rows*

no longer add up to zero: a field that is "constant" in the face frame is not parallel in a curved face.

28. The Code

`EdgeConnection.h` computes the transports (26.1) and the holonomies (26.2) from the intrinsic geometry of Part III:

```
/// Section 26: the transport along every edge and the holonomy of every face.
inline Geometry compute(Mesh& mesh, const intrinsic::Geometry& G) {
    Geometry E;
    for (const Edge* e : mesh.getEdges()) {
        // carry a vector from a to b along the edge: keep its angle to the edge, which points along phi_ab at a and
        // along phi_ba + pi at b
        E.transport[e] = vertexAngle(G, e->pair) + M_PI - vertexAngle(G, e);
    }
    E.holonomy.resize(mesh.numFaces());
    for (Face& f : mesh.getFaces()) {
        Vertex* v[3];
        Edge* h[3];
        intrinsic::faceCorners(f, v, h);
        double omega = 0;
        for (int a = 0; a < 3; ++a) omega += (G.vertices[v[a]->ID].scale - 1.0) * G.faces[f.ID].angle[a];
        E.holonomy[f.ID] = omega;
    }
    return E;
}
```

and the element (27.7), with the collapsed-square Gauss rule:

```
/// Section 27, eq. curved_element: the 3 x 3 Hermitian element matrix of a face whose connection has the constant
/// curvature kappa = Omega_f / A_f, for coordinates in the planes of the three vertices:
/// L_ab = e^{i rho_ba} \int_f e^{i kappa A(x_a, x_b, x)} (grad phi_a - i phi_a w_a) . (grad phi_b + i phi_b w_b) dA,
/// w_a(x) = (kappa / 2) J (x_a - x),
/// with A(x_a, x_b, x) the signed area of the triangle x_a x_b x. Integrated with an n x n Gauss rule on the triangle,
/// seen as a collapsed square.
inline void curvedElement(const intrinsic::FaceGeometry& g, double omega, const Complex transportBA[3][3], int n,
                          Complex L[3][3]) {
    const double kappa = omega / g.area;
    std::vector<double> x, w;
    gaussLegendre(n, x, w);
    auto J = [](const Vec2& v) { return Vec2(-v.y(), v.x()); };
    auto cross = [](const Vec2& p, const Vec2& q) { return p.x() * q.y() - p.y() * q.x(); };
    for (int a = 0; a < 3; ++a) for (int b = 0; b < 3; ++b) L[a][b] = 0;
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j) {
            // the collapsed square: lambda = (1 - s, s (1 - t), s t), dA = 2 A s ds dt
            const double s = x[i], t = x[j];
            const double phi[3] = { 1 - s, s * (1 - t), s * t };
            const double weight = w[i] * w[j] * 2.0 * g.area * s;
            const Vec2 p = phi[0] * g.x[0] + phi[1] * g.x[1] + phi[2] * g.x[2];
            Eigen::Vector2cd D[3]; // grad phi_a + i phi_a w_a
            for (int a = 0; a < 3; ++a) {
                const Vec2 wa = 0.5 * kappa * J(g.x[a] - p);
                D[a] = g.grad[a].cast<Complex>() + Complex(0, 1) * phi[a] * wa.cast<Complex>();
            }
```

```
            for (int a = 0; a < 3; ++a)
                for (int b = 0; b < 3; ++b) {
                    const double area = 0.5 * cross(g.x[b] - g.x[a], p - g.x[a]);
                    L[a][b] += weight * std::polar(1.0, kappa * area) * D[a].dot(D[b]);
                         // dot conjugates D[a]
                }
        }
    for (int a = 0; a < 3; ++a)
        for (int b = 0; b < 3; ++b) L[a][b] *= transportBA[a][b];
}
```

The assembly in `ga_fem.cpp` is that of the previous parts, element by element:

```
/// Section 27: assemble the curved elements (eq. curved_element) with the lumped mass A_f / 3.
static Eigen::VectorXcd solve(Mesh& mesh, const intrinsic::Geometry& G, const edgeconn::Geometry& E, double dt, int source,
                              Complex value, int quadrature = 8) {
    const int n = mesh.numVertices();
    std::vector<double> mass(n, 0.0);
    std::map<std::pair<int, int>, Complex> A;
    for (Face& f : mesh.getFaces()) {
        Vertex* v[3];
        Edge* h[3];
        intrinsic::faceCorners(f, v, h);
        Complex transportBA[3][3], L[3][3]; // e^{i rho_ba}: from the plane of v[b] to that of v[a]
        for (int a = 0; a < 3; ++a)
            for (int b = 0; b < 3; ++b) transportBA[a][b] = a == b ? Complex(1.0) : std::polar(1.0, E.transport.at(halfEdge(h, b, a)));
        edgeconn::curvedElement(G.faces[f.ID], E.holonomy[f.ID], transportBA, quadrature, L);
        for (int a = 0; a < 3; ++a) {
            mass[v[a]->ID] += G.faces[f.ID].area / 3.0;
            for (int b = 0; b < 3; ++b) A[{ v[a]->ID, v[b]->ID }] += dt * L[a][b];
        }
    }
    return solveScaled(n, A, mass, source, value);
}
```

29. Results

Section 35 compares all the methods on the meshes of Section 24. In short: on the flat and folded sheets and on the Miura-ori every vertex is flat, $s = 1$, every face holonomy vanishes, the curved element is the flat one with exact transports, and the method is exact to 10^{-12} degrees. On the spheres it is as accurate as the intrinsic method of Part III, within a few percent at every resolution: moving the curvature from the vertices into the faces neither helps nor hurts the vector heat method. On the cactus the two differ by a median of 2.1°, comparable to the difference between Parts II and III: the choice of where the curvature of a coarse mesh lives is a modeling choice of that size. The value of this part is elsewhere: it is a conforming finite element method for the edge connection, and it explains the method of the next part.

Part V. Discrete Exterior Calculus, Slowly

The vector heat method of Sharp, Soliman and Crane [17], and the direction fields of Knöppel et al. [10] before it, discretize the connection Laplacian with

the tools of *discrete exterior calculus* (DEC), in a few lines that assume the reader knows them. This part builds those tools from the beginning, with one triangle as the running example, and then shows that the resulting operator is the finite element method of Part IV with its curvature terms dropped.

30. Discrete Differential Forms

30.1. Store integrals, not values

A differential k-form is something that is integrated over k-dimensional pieces: a 0-form (a function) is evaluated at points, a 1-form is integrated along curves, a 2-form over regions. The idea of DEC is to store a k-form on a mesh by its integrals over the k-dimensional pieces of the mesh: a discrete 0-form is a number at each vertex, a discrete 1-form a number on each oriented edge (its integral along the edge), a discrete 2-form a number on each face (its integral over the face). Such lists of numbers are called *cochains*. Reversing the orientation of an edge changes the sign of its number, as reversing a path changes the sign of an integral.

30.2. The exterior derivative is exact

The reason for storing integrals is that the exterior derivative d then needs no approximation at all, because of Stokes's theorem $\int_S d\eta = \int_{\partial S} \eta$. For a function f, the integral of df along the edge from a to b is $f_b - f_a$: the fundamental theorem of calculus. For a 1-form η, the integral of $d\eta$ over the face abc is $\eta_{ab} + \eta_{bc} + \eta_{ca}$: Green's theorem. So

$$(d_0 f)_{ab} = f_b - f_a, \qquad (d_1 \eta)_{abc} = \eta_{ab} + \eta_{bc} + \eta_{ca}, \tag{30.1}$$

and d_0, d_1 are the incidence matrices of the mesh, with entries 0 and ± 1. Applying both to a function, $(d_1 d_0 f)_{abc} = (f_b - f_a) + (f_c - f_b) + (f_a - f_c) = 0$: the discrete counterpart of $d \circ d = 0$, "the curl of a gradient vanishes", holds exactly (Figure 30, left). The edge transports of Part IV are a discrete 1-form, and the face holonomy (26.2) is its d_1.

31. The Hodge Star: Where the Geometry Enters

31.1. Why lengths and areas are needed

The exterior derivative uses only how the mesh is connected; it knows no lengths. Lengths enter with the inner product of forms, which the Dirichlet energy $\int |\nabla f|^2$ needs. In the smooth setting this is the *Hodge star* $\star$, which turns a k-form into a $(2-k)$-form; on a surface, $\star$ of a 1-form is the same 1-form turned by a quarter turn, and $\int \eta \wedge \star\eta = \int |\eta|^2$. In DEC, $\star$ is a diagonal matrix that converts the integral of a form over a piece of the mesh into its integral over the *dual* piece, and it is the only approximation in the method.

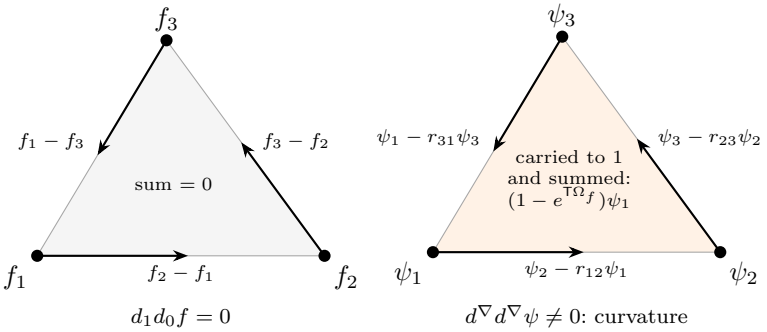


FIGURE 30. Left: a function at the vertices, its exterior derivative on the edges (the differences), and its second derivative on the face, which is zero. Right: the same for vector coordinates, with the edge transports of Part IV: the differences must be taken after transport, and around a face they add up to $(1 - e^{\mathsf{T}\Omega_f})\psi_1$, the curvature.

31.2. The dual mesh

Put a dual vertex at the circumcenter of every face; join the circumcenters of two faces that share an edge e by the dual edge e^*, which crosses e at a right angle (both circumcenters lie on the perpendicular bisector of e); the dual edges around a vertex i bound its dual cell i^* (Figure 31). Every primal piece has a dual piece of complementary dimension: vertex and cell, edge and dual edge, face and dual vertex. The Hodge stars are the ratios of their sizes:

$$(\star_0)_{ii} = |i^*|, \qquad (\star_1)_{ee} = \frac{|e^*|}{|e|}, \qquad (\star_2)_{ff} = \frac{1}{A_f}. \tag{31.1}$$

For instance $\star_1$ says: a 1-form whose integral along e is η_e is, near e, roughly constant with magnitude $\eta_e/|e|$ along e; turned by a quarter turn it points along e^*, and its integral along e^* is $\eta_e\,|e^*|/|e|$.

31.3. The length of a dual edge

Proposition 31.1. *The distance from the circumcenter of a triangle to the midpoint of its edge of length ℓ is $\frac{\ell}{2}\cot\alpha$, with α the angle opposite to the edge (negative, that is, on the other side of the edge, when α is obtuse). Hence $|e^*| = \frac{\ell}{2}(\cot\alpha + \cot\beta)$ and*

$$(\star_1)_{ee} = \frac{1}{2}(\cot\alpha + \cot\beta). \tag{31.2}$$

Proof. Let O be the circumcenter, a and b the ends of the edge and m its midpoint. By the inscribed angle theorem, the angle aOb at the center is twice the angle α at the circumference, so the angle aOm is α. The triangle aOm has a right angle at m and the side $|am| = \ell/2$, so $|Om| = |am|/\tan\alpha = \frac{\ell}{2}\cot\alpha$. The dual edge is made of the two pieces from O_f and O_g to m. □

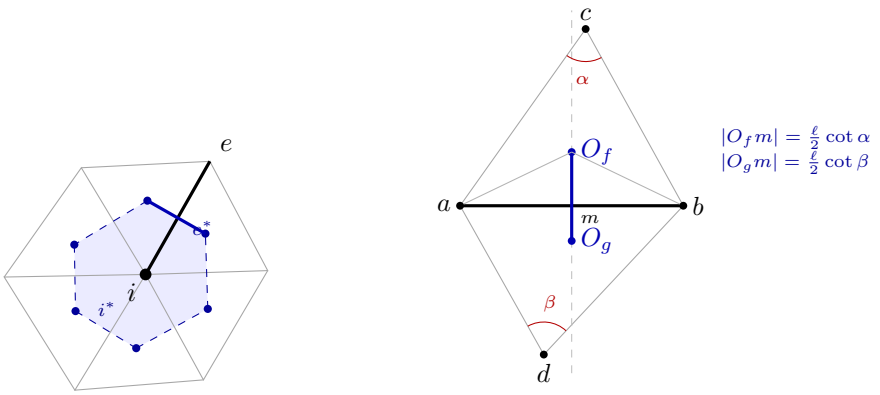


FIGURE 31. Left: the circumcentric dual of a one-ring: dual vertices at the circumcenters (blue dots), dual edges (dashed) orthogonal to the primal edges, and the dual cell i^* of the center vertex (shaded); one edge e and its dual e^* in bold. Right: the dual edge of $e = ab$ runs from the circumcenter O_f of one face to O_g of the other, through the midpoint m; its two pieces are $\frac{\ell}{2}\cot\alpha$ and $\frac{\ell}{2}\cot\beta$, with α and β the angles opposite to e.

The dual edge of a boundary edge has only one piece, and its $\star_1$ only one cotangent. For $\star_0$ two choices are common: the area of the circumcentric dual cell, or a third of the area of each face around the vertex, the *barycentric* cell. The second is the lumped mass of the finite element method, (8.3), and is the one we use in all five parts.

32. The Scalar Laplacian of DEC Is the Finite Element Laplacian

The Dirichlet energy of a function, in DEC, is the inner product of its derivative with itself: $\int |\nabla f|^2 \approx (d_0 f)^T \star_1 (d_0 f) = \sum_e (\star_1)_{ee} (f_b - f_a)^2$. Its matrix is

$$\mathrm{L} = d_0^T \star_1 d_0 : \qquad \mathrm{L}_{ab} = -\tfrac{1}{2}(\cot\alpha + \cot\beta), \qquad \mathrm{L}_{aa} = -\textstyle\sum_{b\neq a} \mathrm{L}_{ab}, \quad (32.1)$$

for neighbors $a \neq b$, with α and β the angles opposite to their edge, and the Laplacian is $\Delta = -\star_0^{-1} \mathrm{L}$, with the inverse of $\star_0$ as the codifferential's "division by area". These are exactly the stiffness matrix of the linear finite element method, Proposition 7.3, and its lumped mass: for scalar functions, DEC and linear finite elements with lumped mass are the same method, reached by two roads. The finite element road computes $\int \nabla\phi_a \cdot \nabla\phi_b$; the DEC road computes the ratio of a dual length to a primal one; Proposition 31.1 is why they agree.

Worked example 32.1. *For the triangle of Example 7.2, with angles* $90°$ *at* x_1*,* $35.26°$ *at* x_2*,* $54.74°$ *at* x_3*, the edges* 12*,* 23*,* 31 *are opposite to* x_3*,* x_1*,* x_2*, so (one face, one cotangent each)*

$$\star_1 = \operatorname{diag}\left(\tfrac{1}{2}\cot 54.74°,\ \tfrac{1}{2}\cot 90°,\ \tfrac{1}{2}\cot 35.26°\right) = \operatorname{diag}(0.3536,\ 0,\ 0.7071),$$

$$d_0 = \begin{pmatrix} -1 & 1 & 0 \\ 0 & -1 & 1 \\ 1 & 0 & -1 \end{pmatrix} \quad (\textit{rows: edges } 12, 23, 31;\ \textit{columns: vertices } 1, 2, 3),$$

and $d_0^T \star_1 d_0$ *is the stiffness matrix of Example 7.2, entry for entry (the program prints it). The edge* 23 *is opposite to the right angle: it is a diameter of the circumcircle, the circumcenter is its midpoint* $(1, 0.707107)$ *in the layout, its dual edge piece has length zero, and so has its weight. The other two pieces have lengths* $0.707107 = \frac{2}{2}\cot 54.74°$ *and* $1 = \frac{\sqrt{2}}{2}\cot 35.26°$*, as Proposition 31.1 says.*

33. Vector Fields: the Covariant Exterior Derivative

33.1. Differences after transport

A vector field at the vertices is a list of spinor coordinates ψ_a, each in the plane of its vertex. The difference $\psi_b - \psi_a$ means nothing, because the two numbers are angles from different reference directions. The difference that means something is the one taken after carrying ψ_a to b along the edge with the transport of Part IV:

$$(d^{\nabla}\psi)_{ab} = \psi_b - r_{ab}\,\psi_a, \tag{33.1}$$

the *covariant exterior derivative*, a discrete 1-form with values in the plane of the end vertex b. It is zero exactly when ψ_b is the parallel transport of ψ_a. As a matrix it is d_0 with the entry -1 of each row replaced by $-r_{ab}$. In a moving frame (Section 42), the equation of a parallel field, $d\psi + \omega \mathsf{T} \psi = 0$, integrates along the edge to $\psi_b = e^{-\mathsf{T}\int_a^b \omega}\,\psi_a$, (41.1); so $d^{\nabla}\psi$ measures, at b, how far ψ_b is from the value that the moving frame equation carries from a.

33.2. The energy and the connection Laplacian

The Dirichlet energy of the field is the same expression as for functions, with d^{∇} in place of d_0:

$$E(\psi) = \sum_e (\star_1)_{ee}\,|\psi_b - r_{ab}\psi_a|^2 = (d^{\nabla}\psi)^H \star_1 (d^{\nabla}\psi), \qquad \mathrm{L}^{\nabla} = (d^{\nabla})^H \star_1 d^{\nabla}, \tag{33.2}$$

where ^{H} is the conjugate transpose. Expanding $|\psi_b - r_{ab}\psi_a|^2 = |\psi_b|^2 + |\psi_a|^2 - 2\,\mathrm{Re}\,(\overline{\psi_a}\, r_{ba}\,\psi_b)$ gives the entries

$$\mathrm{L}^{\nabla}_{aa} = \sum_b \tfrac{1}{2}(\cot\alpha_{ab} + \cot\beta_{ab}), \qquad \mathrm{L}^{\nabla}_{ab} = -\tfrac{1}{2}(\cot\alpha_{ab} + \cot\beta_{ab})\, r_{ba}, \tag{33.3}$$

the cotangent Laplacian with the transport r_{ba}, from b into the plane of a, on every off-diagonal entry. This is the connection Laplacian of [10, 17]. The

backward Euler step is $(\star_0 + \Delta t\, \mathrm{L}^\nabla)\psi = \star_0 \psi^0$, and the rest of the program is unchanged.

33.3. Curvature: $d^\nabla d^\nabla \neq 0$

For functions $d_1 d_0 = 0$. For vector coordinates it fails, and the failure is the curvature. Extend d^∇ to 1-forms in the same spirit: on the face abc, carry the three edge values to the vertex a and add them,

$$(d^\nabla \eta)_{abc} = \eta_{ca} + r_{ca}\,\eta_{bc} + r_{ca} r_{bc}\,\eta_{ab}$$

(η_{ab} lives at b, so it is carried to c and then to a; η_{bc} lives at c; η_{ca} at a). For $\eta = d^\nabla \psi$ the terms telescope:

$$\begin{aligned}(\psi_a - r_{ca}\psi_c) + r_{ca}(\psi_c - r_{bc}\psi_b) + r_{ca}r_{bc}(\psi_b - r_{ab}\psi_a) &= (1 - r_{ca}r_{bc}r_{ab})\,\psi_a \\ &= (1 - e^{\mathsf{T}\Omega_f})\,\psi_a.\end{aligned}$$

So $d^\nabla d^\nabla = 1 - e^{\mathsf{T}\Omega_f}$ on every face (Figure 30, right): it vanishes exactly on the faces without holonomy. A connection is *flat* when $d^\nabla d^\nabla = 0$, and the curvature is the obstruction to it, as in the smooth theory, where $d^\nabla d^\nabla$ is multiplication by the curvature 2-form.

Worked example 33.1. *On the face of Example 26.1,* $\Omega_f = 12^\circ$ *and* $1 - e^{\mathsf{T}\Omega_f} = 0.021852 - 0.207912\,\mathsf{T}$*, of modulus* $2\sin 6^\circ = 0.209057$*. For the vertex values* $\psi = (0.3 - 0.7\mathsf{T},\ 1.1 + 0.2\mathsf{T},\ -0.4 + 0.5\mathsf{T})$ *the program computes* d^∇ *twice and finds* $-0.138982 - 0.077670\,\mathsf{T}$*, which is* $(1 - e^{\mathsf{T}\Omega_f})\,\psi_1$ *to all digits.*

34. DEC as a Finite Element Method

34.1. The linear element written edge by edge

The rows of the scalar element matrix add up to zero, because the hat functions add up to one and their gradients to zero. So the scalar element energy can be written as a sum over the three edges of the face:

$$\begin{aligned}\sum_{a,b} \overline{u_a}\, A_f (\nabla\phi_a \cdot \nabla\phi_b)\, u_b &= \sum_{\text{edges } ab \text{ of } f} \big(- A_f\, \nabla\phi_a \cdot \nabla\phi_b \big)\, |u_b - u_a|^2 \\ &= \sum_{\text{edges } ab \text{ of } f} \tfrac{1}{2} \cot\alpha_{ab}\, |u_b - u_a|^2,\end{aligned}$$

using $-A_f \nabla\phi_a \cdot \nabla\phi_b = \frac{1}{2}\cot\alpha_{ab}$ (Proposition 7.3). Summing over the two faces of each edge gives $\sum_e (\star_1)_{ee}\, |u_b - u_a|^2$: the DEC energy for functions.

34.2. Where DEC puts the transports

Now let the values live in the vertex planes, $u_a = e^{\mathsf{T}\sigma_a}\psi_a$ in a flat frame of the face. Then $|u_b - u_a| = |\psi_b - e^{\mathsf{T}(\sigma_a - \sigma_b)}\psi_a|$, and this is the DEC difference $|\psi_b - r_{ab}\psi_a|$ when $\sigma_a - \sigma_b = \rho_{ab}$ on every edge of the face. By (27.4) with $\kappa = 0$ that is possible only if $\Omega_f = 0$. DEC does not ask: it uses, edge by edge, the transport of that edge, as if each pair of vertices of the face could

be compared in its own flat frame. In terms of Part IV, DEC is the curved element (27.7) with $\kappa = 0$ inside the integral,

$$\mathrm{L}^{f}_{ab} = e^{\mathsf{T}\rho_{ba}}\, A_f\, \nabla\phi_a \cdot \nabla\phi_b,$$

and summing over the two faces of each edge gives exactly (33.3). The holonomy is kept, in the transports, but the curvature inside the faces is dropped: the factor $e^{\mathsf{T}\kappa\mathcal{A}}$ and the terms $\mathsf{T}\phi\, w$. Those terms are of the order of Ω_f, so the two methods agree on intrinsically flat meshes, exactly, and on fine meshes of smooth surfaces, where $\Omega_f \to 0$; on coarse curved meshes DEC is a slightly cruder finite element method. In the finite element vocabulary, DEC commits a *variational crime*: its energy is not the energy of any field in a conforming space, but it converges, because the crime vanishes with the curvature of the faces.

35. The Code and the Results

The DEC operator is built in `ga_fem.cpp` exactly as written in (33.2): the matrix d^{∇}, one row per edge, the diagonal $\star_1$ and $\star_0$, and the product $(d^{\nabla})^{H}\star_1 d^{\nabla}$ computed by Eigen:

```
/// Section 33: the covariant exterior derivative d^nabla (one row per edge a -> b: psi_b - r_ab psi_a), the diagonal Hodge
/// stars *1 (half the sum of the cotangents opposite to each edge) and *0 (a third of the area of the faces around each
/// vertex), and the connection Laplacian L = (d^nabla)^H *1 d^nabla.
static Eigen::SparseMatrix<Complex> laplacian(Mesh& mesh, const intrinsic::Geometry& G, const edgeconn::Geometry& E,
                                              std::vector<double>& star0) {
    std::vector<const Edge*> edges;
    for (const Edge* e : mesh.getEdges()) if (e->vertex->ID < e->pair->vertex->ID) edges.push_back(e);
    auto cotOpposite = [&](const Edge* e) { // the cotangent of the angle opposite to e in the face to its left
        if (!e->face) return 0.0;
        Vertex* v[3];
        Edge* h[3];
        intrinsic::faceCorners(*e->face, v, h);
        int k = 0;
        while (h[k] != e) ++k;
        return 1.0 / std::tan(G.faces[e->face->ID].angle[(k + 2) % 3]);
    };
    std::vector<Eigen::Triplet<Complex>> d, star1;
    for (int k = 0; k < (int)edges.size(); ++k) {
        const Edge* e = edges[k];
        d.emplace_back(k, e->pair->vertex->ID, 1.0); // + psi_b
        d.emplace_back(k, e->vertex->ID, -std::polar(1.0, E.transport.at(e))); // - r_ab psi_a
        star1.emplace_back(k, k, 0.5 * (cotOpposite(e) + cotOpposite(e->pair)));
    }
    const int n = mesh.numVertices(), m = (int)edges.size();
    Eigen::SparseMatrix<Complex> D(m, n), S1(m, m);
    D.setFromTriplets(d.begin(), d.end());
    S1.setFromTriplets(star1.begin(), star1.end());
    star0.assign(n, 0.0);
    for (Face& f : mesh.getFaces()) {
        Vertex* v[3];
        Edge* h[3];
        intrinsic::faceCorners(f, v, h);
        for (int a = 0; a < 3; ++a) star0[v[a]->ID] += G.faces[f.ID].area / 3.0;
```

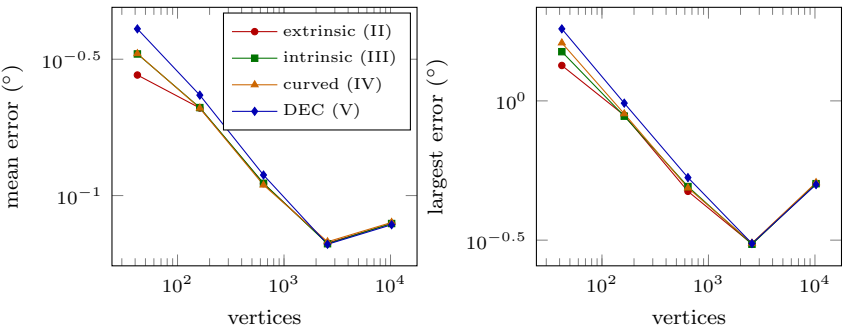


FIGURE 32. The vector heat method on geodesic spheres with the four connections: mean (left) and largest (right) angle to exact transport, against the number of vertices. DEC (blue) is the least accurate on the coarse spheres, where the faces have the largest holonomies; all four converge together.

```
    }
    return Eigen::SparseMatrix<Complex>(D.adjoint() * S1 * D);
}
```

The program runs the four connections on the meshes of Section 24, with the same measurement, corner by corner in the faces; Parts IV and V put a vertex vector into a face by keeping its angle to the edge of the corner. On the flat sheet, the parallel folds and the Miura-ori, the methods of Parts III, IV and V are all exact, to 10^{-12} degrees, and only Part II fails on the Miura-ori (7.42° mean, 21.9° max). On the spheres (Figure 32), mean and largest error:

vertices	largest Ω_f	II extrinsic	III intrinsic	IV curved	V DEC
42	9.32°	0.277° / 1.340°	0.331° / 1.502°	0.332° / 1.615°	0.409° / 1.814°
162	2.39°	0.209° / 0.888°	0.210° / 0.881°	0.209° / 0.899°	0.233° / 0.981°
642	0.65°	0.111° / 0.473°	0.111° / 0.492°	0.109° / 0.487°	0.119° / 0.530°
2562	0.17°	0.067° / 0.307°	0.067° / 0.306°	0.068° / 0.308°	0.066° / 0.308°
10242	0.04°	0.079° / 0.506°	0.079° / 0.504°	0.080° / 0.509°	0.078° / 0.501°

DEC is the least accurate on the coarse spheres, 23% above the curved element on the coarsest and 11% on the next, and the difference disappears with the face holonomies, as Section 34 predicts: it is the curvature inside the faces that DEC drops. On the cactus, whose faces have holonomies up to 34.5°, the curved element and DEC differ by a median of 3.6° (90% of the corners below 6.2°), more than any other pair of methods: the intrinsic and curved methods differ by 2.1°, the extrinsic and intrinsic by 2.2°.

36. Summary of the Five Parts

All five parts solve the same equation with the same unknowns, one complex number per vertex, the same lumped mass, the same time step and the same solver. They differ in one thing only: how a vector at one vertex is compared

TABLE 4. The five discretizations of the connection Laplacian in this text.

	Parts I, II	Part III	Part IV	Part V (DEC)
input	positions, normals	edge lengths	edge lengths	edge lengths
vertex plane	orthogonal to n_i	angles $\times 2\pi/\Theta_i$	angles $\times 2\pi/\Theta_i$	angles $\times 2\pi/\Theta_i$
transport	vertex $\to$ face, rolling	vertex $\to$ face, bisectors	vertex $\to$ vertex, edges	vertex $\to$ vertex, edges
curvature lives	at the vertices (gaps of rolled corners)	at the vertices (stretched corners)	in the faces, as a curved connection	in the faces, as holonomy only
element	flat, $A\nabla\phi \cdot \nabla\phi\, e^{\mathsf{T}\Delta\rho}$	flat, same form	curved, (27.7), quadrature	flat with edge transports
exact on flat folded meshes	parallel folds only	yes	yes	yes
coarsest sphere, mean	0.28°	0.33°	0.33°	0.41°

with a vector at another, and therefore where the curvature of the mesh is kept (Table 4).

For the vector heat method on reasonable meshes the four agree to a fraction of a degree, and the simplest, DEC, is a fine choice: its matrix is one product of three sparse matrices. The other parts explain what that matrix is. It is a finite element method (Part V) that drops the curvature terms of Part IV; it works in the vertex planes of Part III; and those planes replace the vertex normals of Parts I and II, which are exact on a folded flat mesh only when the folds are parallel. Part VI puts all of this on one page, and goes further.

Part VI. How the Pieces Fit

The five parts were written one after another, each answering a question that the previous one left open: Part I derived the method, Part II asked what is intrinsic, Part III removed the extrinsic ingredient, Part IV moved the curvature into the faces, and Part V explained discrete exterior calculus. Read in that order they are a story. Coming back to them after a long time, a reader needs something else: a map. What is the skeleton that all five share? Where exactly do they differ, and why there and nowhere else? How are the concepts (connection, connection form, holonomy, curvature, 1-forms, the Laplacians) related to each other? Which of the methods should be used for a new problem, and what can go wrong?

This part is that map. It adds no new discretization. It puts the existing ones side by side, proves two facts that join them, and places the whole family among the other ways of computing with vector fields on surfaces. The two facts are Proposition 40.1, which says exactly how the corner transport of

Part III differs from the edge transport of Part IV, and the Weitzenböck identity (43.3), which says how the connection Laplacian of this text differs from the Hodge Laplacian of differential forms. Both are checked numerically by `ga_fem`.

Every section begins with a short summary and points back to where the details are, so that the sections can be read in any order:

- the common skeleton of all the methods: Section 37;
- the three choices that make them different, and why there are only three: Section 38;
- where each method keeps the curvature of the mesh: Section 39;
- corner transport against edge transport, exactly: Section 40;
- the connection form, in every part: Section 41;
- Cartan's moving frames as the common language of Parts I to IV, and a sketch of a curved element of degree two built on them: Section 42;
- vector-valued forms, scalar 1-forms, the Hodge Laplacian and the Weitzenböck identity: Section 43;
- the standard method of numerical analysis, which needs no discrete connection at all: Section 44;
- what matrices, geometric algebra and complex numbers each contribute: Section 45;
- one small mesh traced through every step with every method: Section 46;
- the concept map, the master table and the list of symbols: Sections 47 and 48;
- which method to choose, how to extend it to a new problem, and the mistakes to avoid: Section 49.

37. One Pipeline for All the Methods

Summary. All five parts run the same eight steps (Figure 33). Five of them are identical in every part. The methods differ only in steps 2, 3 and 6: where the unknown vector of a vertex lives, how the vectors of two vertices are compared, and which element matrix integrates the energy.

The problem. Given a tangent vector at one vertex of a triangle mesh, find at every other vertex the vector obtained by parallel transport along the shortest path. The vector heat method (Section 4) answers it by one backward Euler step of the heat equation for tangent fields, $(I + \Delta t\, \nabla^*\nabla)\, U = U^0$, followed by normalization. Everything in this text is a way of discretizing $\nabla^*\nabla$, the connection Laplacian, with one complex unknown per vertex.

Step 1: the geometry of the mesh. What is read from the mesh. Parts I and II read the positions x_i, and compute normals, frames and rotations of space from them. Parts III to V read only the edge lengths ℓ_{ij}. The difference is smaller than it looks: by Proposition 20.1, everything a single face contributes (area, gradients of the hat functions, the scalar stiffness and mass) depends on the positions only through the Gram matrix $G_f = B^T B$, which the three lengths determine (20.2). The positions carry, in addition, how the faces are

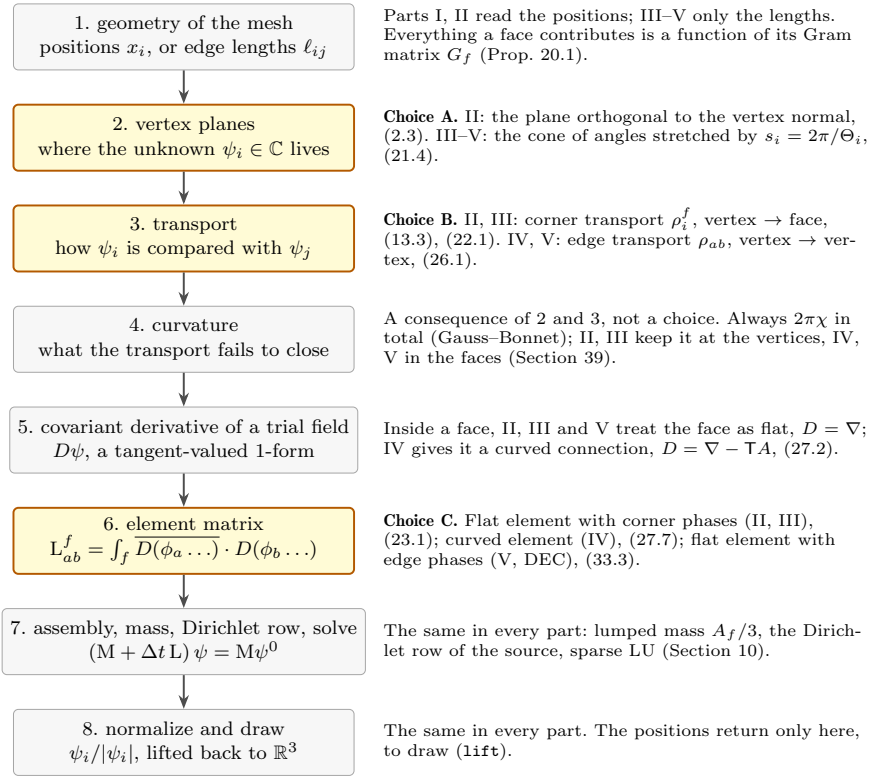


FIGURE 33. The pipeline that every part of this text follows. The highlighted steps are the only ones in which the methods differ (Section 38); step 4 is not a choice but a consequence of steps 2 and 3.

tilted with respect to each other in space. That extra information is used in one place only: step 2 of Parts I and II.

Step 2: the vertex planes. The unknown at vertex i is a tangent vector, stored as a complex number ψ_i, a length and an angle. An angle needs a plane to be measured in, and a vertex of a mesh has no tangent plane (Section 21): its directions form a circle of total angle Θ_i, not 2π. So each method must *invent* a plane for every vertex. Part II takes the plane orthogonal to the vertex normal (2.3) and rolls each face rigidly into it. Parts III to V take the cone of directions and stretch every angle by $s_i = 2\pi/\Theta_i$ (21.4). In both, the angle $\varphi = 0$ is the reference edge `vi.edge`, so that ψ_i means the same thing ("this many degrees from the reference edge") in all the methods, and their solutions can be compared.

Step 3: the transport. Two numbers ψ_i and ψ_j live in two different planes and cannot be subtracted until one of them is carried into the plane of the other. The rule that carries them is the discrete connection. Parts I to III carry

each vertex value into each face around it, one angle ρ_i^f per *corner* (i, f); two vertices of a face are then compared inside the face. Parts IV and V carry a vertex value directly into the plane of a neighbor, one angle ρ_{ab} per *edge*. Section 40 compares the two exactly.

Step 4: the curvature. A transport that is followed around a closed loop comes back turned by some angle, its *holonomy*, and that angle is the curvature enclosed by the loop. Which loops close and which do not is decided by steps 2 and 3, not chosen separately. The total is always the same, $2\pi\chi$ on a closed mesh, because of the discrete Gauss–Bonnet theorem (Section 21); what changes from method to method is *where* it is kept (Section 39).

Step 5: the covariant derivative of a trial field. The finite element method interpolates the vertex values inside each face and differentiates the result. The derivative is the covariant derivative $D\psi = d\psi + \omega\mathsf{T}\psi$ of Part II, with the connection form ω of the face. If the transport of step 3 makes the face flat, $\omega = 0$ inside it and D is the ordinary gradient; that is the case in Parts I to III, and also, by decision rather than necessity, in Part V. Part IV gives the face the curved connection (27.2).

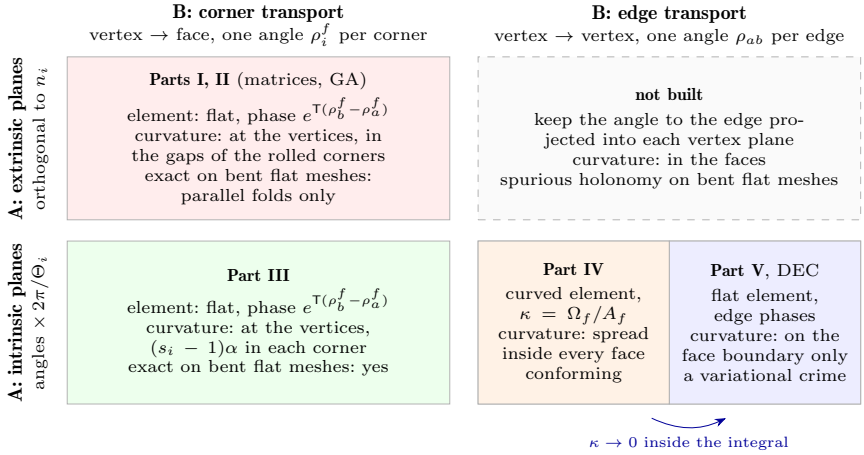
Step 6: the element matrix. The integral of $|D\psi|^2$ over a face, written as a Hermitian 3×3 matrix in the three vertex values. For a flat face it is always the scalar cotangent element with a phase on every entry: $\mathrm{L}_{ab}^f = A_f\, \nabla\phi_a \cdot \nabla\phi_b\, e^{\mathsf{T}\delta_{ab}}$, where $e^{\mathsf{T}\delta_{ab}}$ is the transport from b to a that the method uses *in that face*. The curved element of Part IV changes the integrand itself.

Steps 7 and 8: assembly, solve, draw. The same in every part. The element matrices are added into an $n \times n$ Hermitian matrix L; the lumped mass is $\mathrm{M}_{ii} = \frac{1}{3}\sum_{f \ni i} A_f$; the backward Euler step $(\mathrm{M} + \Delta t\, \mathrm{L})\psi = \mathrm{M}\psi^0$ is scaled by M^{-1}, the row of the source is replaced by the prescribed value, and the system is solved by sparse LU (Section 10). Finally $\psi_i/|\psi_i|$ is drawn, which needs the positions again (the function `lift`, Section 23).

Why so few steps differ. The weak form, the Galerkin method, linear hat functions, assembly and the solver are standard finite element machinery, the same as for the scalar heat equation [11, 15]. The only ingredient that the scalar method does not have is the comparison of vectors at different points, the connection, and every difference between the methods is a different way of discretizing it.

38. Three Choices, and One That Is Not a Choice

Summary. The methods are the cells of a small grid (Figure 34): choice A, the vertex planes (extrinsic or intrinsic), times choice B, the transport (corner or edge), with choice C, the element, nearly forced by B. The notation (matrices, geometric algebra, complex numbers) is not a fourth choice: it changes no number.



Choice C, the element, follows from B: corner transport puts the three values in one flat frame, so the flat element is the only one; edge transport around a face closes with $e^{\intercal\Omega_f}$, so the element must either curve (IV) or ignore it (V). Fitting a flat frame to edge transports gives corner transport again (Section 38).

FIGURE 34. The methods of this text as cells of a grid. Rows: choice A, how a vertex gets a plane. Columns: choice B, what the transport connects. Inside each cell, choice C, the element, and the consequences: where the curvature lives, and whether a mesh that is bent but intrinsically flat is treated exactly. The empty cell is discussed in Section 38.

38.1. Choice A: how a vertex gets a plane

Extrinsic (Parts I, II): the plane orthogonal to the vertex normal, with each face rolled into it by the smallest rotation of space. It uses one piece of information more than the lengths, the tilt of the faces. That information helps when the mesh samples a smooth surface whose normals it approximates well (the coarse spheres of Section 24, where Part II is the most accurate), and it hurts when the tilt is not related to the intrinsic geometry, as on the Miura-ori, where it produces errors of up to 21.9° on a mesh that is isometric to a flat sheet. The construction fails when a face normal is opposite to the vertex normal.

Intrinsic (Parts III to V): the cone of directions stretched by $s_i = 2\pi/\Theta_i$. It uses only the lengths, so it gives the same answer on all isometric meshes: exactly the answer of the flat sheet on every bent flat mesh. It never fails for non-degenerate triangles.

38.2. Choice B: what the transport connects

Corner transport (Parts I to III): an angle ρ_i^f for every pair of a vertex and a face around it. A vertex value is carried into a face; two vertex values meet inside the face. *Edge transport* (Parts IV, V): an angle ρ_{ab} for every oriented edge, with $\rho_{ba} = -\rho_{ab}$ (up to whole turns). A vertex value is carried directly

into the plane of a neighbor. Section 40 shows that the two differ by a small, explicit correction, and Section 41 that only the second is a discrete 1-form of the mesh.

38.3. Choice C: the element, nearly forced by B

With corner transport, the three values of a face are carried into one flat frame of that face, and around the face the transports compose to the identity: $(\rho_a - \rho_b) + (\rho_b - \rho_c) + (\rho_c - \rho_a) = 0$. There is nothing to curve, and the element is the flat one (23.1). With edge transport, the three transports around a face compose to $e^{\mathsf{T}\Omega_f}$, which is not the identity, and no flat frame of the face can agree with all three of them (Section 27). Then there are exactly three things one can do:

1. give the face a connection with total curvature Ω_f, and integrate the energy of the curved trial functions: the curved element of Part IV, conforming, with quadrature;
2. keep the flat element but put the edge transport of each pair on its entry anyway: DEC, Part V, a variational crime of order Ω_f;
3. choose for each corner the angle σ_a^f that makes a flat frame agree with the three edges as well as possible. But a choice of one angle per corner *is* a corner transport, and its own transports around the face close; the mismatch with the edge transports puts the curvature back at the vertices. This option is corner transport again, and nothing new.

So the element is not an independent choice: B decides it, except for the decision between options 1 and 2.

38.4. The empty cell

Extrinsic planes with edge transport were not built in this text. They are easy to define: keep the angle of a vector to the edge, as in (26.1), but measure the angle of the edge in each vertex plane after projecting the edge into that plane. The face holonomies would then add up to $2\pi\chi$ as long as the projected edges keep their order around each vertex. On a bent flat mesh, however, the projected angles differ from the true ones, so the faces would get holonomies that the paper does not have, and the method would not be exact on the Miura-ori. The extrinsic error moves from the vertices into the faces; it does not disappear. Since the purpose of Part IV was a method that is exact on bent flat meshes, this cell was left empty.

38.5. The notation is not a choice

Part I writes every transport as a real 2×2 rotation, Part II as a unit complex number of the face plane $e^{\mathsf{T}\rho}$ computed with rotors of space, and Parts III to V as a `std::complex`. On the cactus the real and the complex systems agree to $6 \cdot 10^{-16}$ (Section 17). The notation decides how clearly the method can be explained and how much code is needed, not which numbers come out. Section 45 says what each notation is good for.

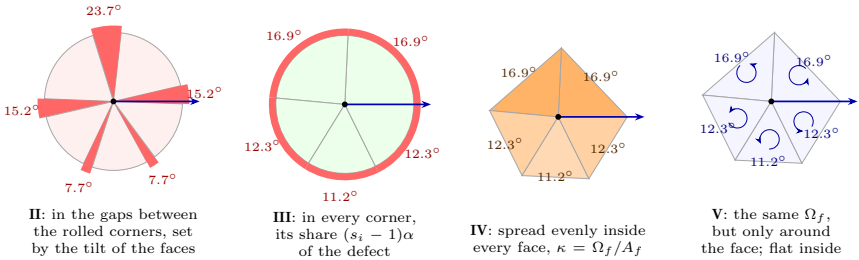


FIGURE 35. The leaning pyramid of Example 21.2, seen from its apex, with an angle defect of 69.5°; corners counterclockwise from the reference edge (blue). Each method keeps the 69.5° somewhere else (red numbers, adding up to 69.5° in every panel). II: in the gaps between the corners rolled into the plane of the vertex normal. III: in every corner, stretched by $s_i = 1.2391$. IV: the same shares, as the holonomies of the five faces, spread evenly inside each face (the shade is the density $\kappa = \Omega_f / A_f$). V: the same holonomies, kept only in the transports around each face.

39. Where the Curvature Lives

Summary. Every method keeps the whole curvature of the mesh, $2\pi\chi$; they differ in where they put it. Parts I and II put it at the vertices, in the gaps between the rolled corners, unevenly, according to the tilt of the faces. Part III puts it at the vertices, in every corner in proportion to its angle. Part IV puts the same shares inside the faces, spread evenly. Part V keeps the same shares on the boundaries of the faces, and treats the inside as flat (Figure 35).

The upright pyramid. The apex of the pyramid over a regular pentagon with equilateral faces (Example 21.1) has five corners of 60°, $\Theta = 300°$, a defect of 60° and $s = 1.2$. Part II rolls the five faces into the horizontal plane and leaves five gaps of 12°. Part III stretches each corner to 72°: each absorbs 12°. Part IV computes the edge transports (26.1), and each face gets the holonomy $\Omega_f = (s-1) \cdot 60° = 12°$ (26.2). The base vertices are on the boundary ($s = 1$) and contribute nothing. Five times 12° is 60° in all four methods, and by symmetry it is distributed in the same way; the methods agree here, and the program finds the four solutions with the source at the apex equal to rounding at every base corner.

The leaning pyramid. Moving the apex by 0.4 breaks the symmetry (Example 21.2): the defect is 69.47°, $s = 1.2391$, and the corners are 70.49°, 70.49°, 51.37°, 46.81°, 51.37°. Part III gives each corner $(s-1)\alpha$: 16.85°, 16.85°, 12.28°, 11.19°, 12.28°. Part IV gives each face the holonomy $\Omega_f = \sum_v (s_v - 1)\alpha_v$, and since the base vertices have $s = 1$, these are the same five numbers, now attached to the faces. Part II leaves the gaps 23.66°, 15.18°, 7.72°, 7.72°, 15.18°, which depend on how far each face is tilted from the

vertex normal. With the source at the apex, the solutions of Parts III and IV differ at the base corners by 0.09° at most, DEC differs from Part III by 0.97°, and Part II by 10.9°.

The sum is fixed, the place is not. In all four methods the total over a closed mesh is $2\pi\chi$. For Parts III to V this is the discrete Gauss–Bonnet theorem, because $\sum_f \Omega_f = \sum_v (s_v - 1)\Theta_v = \sum_v \kappa_v$ (Section 26). For Part II the gaps around each vertex add up to its defect (Section 21). So the methods do not disagree about *how much* curvature the mesh has; they disagree about *where* inside a one-ring it is. On a fine mesh of a smooth surface the defect of every vertex is small, $s_i \to 1$, all the shares shrink, and the place stops mattering: that is why all four converge to the same answer (Figure 32). On a coarse, strongly curved mesh the place is a modeling decision, worth a few degrees on the cactus (Section 35).

Consequences of the place. Three facts of the earlier parts follow from this section alone.

- *Exactness on bent flat meshes.* If every vertex is flat, $\Theta_i = 2\pi$, the intrinsic shares are all zero and there is no curvature anywhere for Parts III to V: they are exact. The gaps of Part II cancel in total at such a vertex but not one by one, unless all the faces are rolled about one line, which is why only the parallel folds are exact for it.
- *The accuracy of DEC on coarse meshes.* DEC knows Ω_f only through the transports on the boundary of the face; the trial functions inside are flat. The terms it drops are of order Ω_f, so it is the least accurate method when the faces hold large holonomies, the coarse spheres, and the difference disappears when they do not (Section 35).
- *The cost of a curved face.* Spreading Ω_f inside the face, Part IV, makes the element conforming but requires quadrature, and on the vector heat method it buys nothing over Part III. On the energy of a smooth field it buys a factor of seven to ten in accuracy (Section 43).

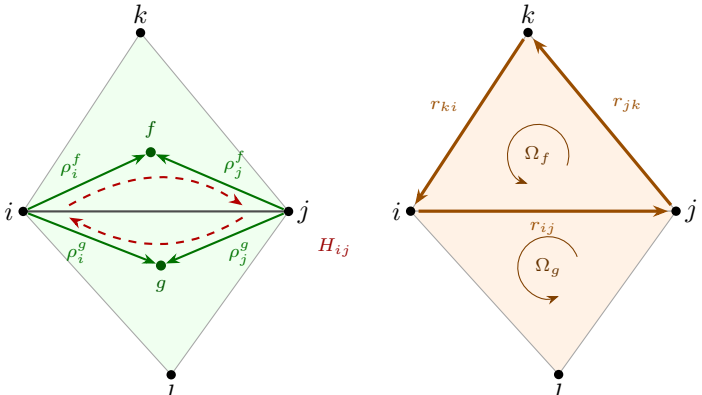
40. Corner Transport and Edge Transport, Exactly

Summary. The corner transport of Part III, followed from a vertex into a face and out to another vertex, is the edge transport of Part IV along the same edge, turned by half the stretching of the two corners at the ends of the edge in that face (Proposition 40.1). The two faces of an edge turn it in opposite directions, so the corner transport along an edge depends on the face it goes through, and the difference between the two faces, H_{ij}, is where Part III keeps its curvature.

40.1. Two vertices compared through a face

In the face f, the value ψ_b of vertex b is carried into the face by $e^{\mathsf{T}\rho_b^f}$ and from there into the plane of vertex a by $e^{-\mathsf{T}\rho_a^f}$. So corner transport defines, for every face and every pair of its vertices, a vertex-to-vertex transport

$$\text{from } b \text{ to } a \text{ through } f: \quad e^{\mathsf{T}(\rho_b^f - \rho_a^f)}, \tag{40.1}$$



corner transport (II, III): vertex to face. Around a face it closes; from i to j through f and back through g it turns by H_{ij}

edge transport (IV, V): vertex to vertex. One transport per edge, shared by f and g; around a face it turns by $\Omega_f = d\rho$

FIGURE 36. Left: corner transport connects every vertex with every face around it (green). Around one face it closes, but going from i to j through f and coming back through g turns a vector by H_{ij}, (40.3). Right: edge transport connects neighboring vertices directly (orange), with one transport per edge that both faces share; around a face it turns by Ω_f.

the phase that the element (23.1) puts on its entry (a, b). An interior edge ab has two faces, so it gets two transports, one through each, and nothing makes them equal. Edge transport instead fixes one transport per edge, $e^{\mathsf{T}\rho_{ab}}$, which both faces use. How far apart are the two? The answer is short.

Proposition 40.1. *Let f be the face to the left of the half edge from a to b, with corner angles α_a^f at a and α_b^f at b, and let s_a, s_b be the scales (21.4) of the two vertices. Then the corner transport of Part III from a to b through f and the edge transport of Part IV are related by*

$$\rho_a^f - \rho_b^f = \rho_{ab} - \tfrac{1}{2}\big[(s_a - 1)\,\alpha_a^f + (s_b - 1)\,\alpha_b^f\big]. \tag{40.2}$$

Through the face g to the right of the half edge the sign of the correction is reversed: $\rho_a^g - \rho_b^g = \rho_{ab} + \frac{1}{2}\big[(s_a - 1)\,\alpha_a^g + (s_b - 1)\,\alpha_b^g\big]$.

Proof. Let c be the third vertex of f, so that a, b, c are counterclockwise. Measure angles at each vertex with the cone angles θ of (21.3), and write θ_{ab} for the cone angle at a of the half edge from a to b. The corner of f at a begins at the edge to b: in the cone of a it is the interval $[\theta_{ab}, \theta_{ab} + \alpha_a^f]$. The corner of f at b begins at the edge to c and ends at the edge to a: it is $[\theta_{bc}, \theta_{bc} + \alpha_b^f]$, with $\theta_{ba} = \theta_{bc} + \alpha_b^f$. In the face frame let β_{ab} and β_{bc} be the directions of the edges from a to b and from b to c; walking counterclockwise

around the face, the direction turns left by $\pi - \alpha_b^f$ at b, so $\beta_{bc} = \beta_{ab} + \pi - \alpha_b^f$. By (22.1),

$$\rho_a^f = \beta_{ab} + \tfrac{1}{2}\alpha_a^f - s_a\big(\theta_{ab} + \tfrac{1}{2}\alpha_a^f\big), \qquad \rho_b^f = \beta_{bc} + \tfrac{1}{2}\alpha_b^f - s_b\big(\theta_{bc} + \tfrac{1}{2}\alpha_b^f\big),$$

and by (26.1), with $\varphi = s\theta$, $\rho_{ab} = s_b\theta_{ba} + \pi - s_a\theta_{ab}$. Subtract, and replace β_{bc} and θ_{ba}:

$$\begin{aligned}(\rho_a^f - \rho_b^f) - \rho_{ab} &= \beta_{ab} - \beta_{bc} - \pi + \tfrac{1}{2}(1 - s_a)\alpha_a^f - \tfrac{1}{2}\alpha_b^f + s_b\theta_{bc} + \tfrac{1}{2}s_b\alpha_b^f - s_b\theta_{ba} \\ &= (\alpha_b^f - 2\pi) + \tfrac{1}{2}(1 - s_a)\alpha_a^f - \tfrac{1}{2}\alpha_b^f - s_b\alpha_b^f + \tfrac{1}{2}s_b\alpha_b^f \\ &= -\tfrac{1}{2}(s_a - 1)\alpha_a^f - \tfrac{1}{2}(s_b - 1)\alpha_b^f - 2\pi,\end{aligned}$$

which is (40.2) up to a whole turn. For g, apply this to the half edge from b to a, which has g to its left, and use $\rho_{ba} \equiv -\rho_{ab}$. □

The proposition is checked by `ga_fem`: on every half edge of the cactus the two sides of (40.2) agree to $1.4 \cdot 10^{-13}$ degrees, while the corrections themselves reach 13.5°. The check divides the corner transport through the face by the edge transport of the same half edge, and compares the angle of the quotient with the prediction:

```
849 /// Section 40: the corner transport from v[a] to v[a+1] through the face of h[a],
        divided by the edge transport of the
850 /// same half edge. Returns its angle and the prediction of Proposition corner_edge,
        -[(s_a - 1) alpha_a + (s_b - 1) alpha_b]/2.
851 static void cornerOverEdge(const intrinsic::Geometry& G, const edgeconn::Geometry& E,
        const Edge* e, const Complex rho[3],
852                            double& measured, double& predicted) {
853     Vertex* v[3];
854     Edge* h[3];
855     intrinsic::faceCorners(*e->face, v, h);
856     int a = 0;
857     while (h[a] != e) ++a;
858     const int b = (a + 1) % 3;
859     const Complex corner = rho[a] * std::conj(rho[b]); // from v[a] into the face, then
            out to v[b]
860     measured = std::arg(corner * std::polar(1.0, -E.transport.at(e)));
861     const intrinsic::FaceGeometry& fg = G.faces[e->face->ID];
862     predicted = -0.5 * ((G.vertices[v[a]->ID].scale - 1) * fg.angle[a] + (G.vertices[v
          [b]->ID].scale - 1) * fg.angle[b]);
863 }
```

On the upright pyramid every correction along an edge from the apex is $\mp 6°$, half of the stretching $(1.2 - 1) \cdot 60°$ of the apex corner (the base corners have $s = 1$).

Reading the proposition. In words: carrying a vector from a into f and out to b, Part III keeps its angle to the edge ab, as the edge transport does, except that each corner is stretched, and the edge sits at the side of the stretched corner, displaced by half the stretching (Figure 25). So:

1. On a flat vertex, $s = 1$, there is no correction: corner transport and edge transport are the same rule, the unfolding.
2. Summed around the face f, every corner appears twice, at the start of one edge and at the end of another, and the corrections add up to $-\sum_v (s_v - 1)\alpha_v^f = -\Omega_f$: exactly what is needed to cancel the holonomy

of the edge transport, so that corner transport closes around every face, as it must.

3. The element (23.1) of Part III and the DEC element of Part V have the same moduli, and their phases differ on every entry by the correction (40.2). DEC is Part III with the corrections removed; Part IV removes them too, and puts the curvature into the trial functions instead.

40.2. The edge loop: where Part III keeps its curvature

Go from a to b through f and come back from b to a through g (Figure 36, left). By the proposition, the vector comes back turned by

$$H_{ab} = \tfrac{1}{2}\big[(s_a - 1)(\alpha_a^f + \alpha_a^g) + (s_b - 1)(\alpha_b^f + \alpha_b^g)\big], \tag{40.3}$$

the transport through g minus the transport through f. This is the curvature of corner transport: it is not on the faces, where corner transport closes, but on the loops that cross an edge from one face to the other. Each vertex contributes, to each of its edges, half the stretching of the two corners on either side of that edge. Summing over the edges of a closed mesh, every corner is counted once for each of its two sides, so

$$\sum_{\text{edges}} H_{ab} = \sum_v (s_v - 1)\,\Theta_v = \sum_v \kappa_v = 2\pi\chi, \tag{40.4}$$

which the program finds on the cactus, 4π to all digits. On the leaning pyramid the five edges from the apex have $H = 16.85°$, $14.57°$, $11.74°$, $11.74°$, $14.57°$ (each the mean of the shares of its two faces), adding up to the defect $69.47°$. The same loop with the corner transport of Part II turns by $23.66°$, $15.18°$, $7.72°$, $7.72°$, $15.18°$: exactly the gaps between the rolled corners on either side of each edge. (At the base vertices, which have two faces each, the vertex normal lies in the plane of the two face normals, so both faces are rolled about the common edge, rolling is unfolding there, and there is no gap.)

Remark 40.2. *Corner angles are not a 1-form of the mesh, but they are a 1-form of another graph: the graph whose nodes are the vertices and the faces of the mesh, with a link from each vertex to each face around it. A change of the frame of vertex i by θ_i and of face f by θ_f changes ρ_i^f by $\theta_i - \theta_f$, a discrete gradient on that graph (Section 41). Its smallest cycles are $a \to f \to b \to g \to a$, one for each edge of the mesh, and their holonomies are the H_{ab}. The faces of the mesh are not cycles of that graph, which is why corner transport has no holonomy on them and why the identities of discrete exterior calculus, $d^\nabla d^\nabla = 1 - e^{\mathsf{T}\Omega_f}$ on the faces (Section 33), have no counterpart for it.*

40.3. What the choice means for a new problem

If a problem is naturally stated on edges, with prescribed holonomies, singularities of a direction field placed in faces, a discrete gauge to be fixed, or a 1-form to be integrated, build it on edge transport (Parts IV, V), where

TABLE 5. The connection form in each part of the text: what represents it, where it lives, and where the curvature, $d\omega$, ends up.

Part	what represents ω	where it lives	curvature $d\omega$
smooth	$\omega(X)$, (15.1), (3.3)	a 1-form on the surface	$d\omega = -K\,dA$, Remark 15.1
I, II	0 inside each face; finite rotation R_i^f, $e^{\mathsf{T}\rho_i^f}$ by rolling	corners	gaps of the rolled corners, at the vertices
III	0 inside each face; $e^{\mathsf{T}\rho_i^f}$ by bisectors, (22.1)	corners (a 1-form of the vertex–face graph, Remark 40.2)	H_{ab} on the edge loops, from the vertex defects
IV	ρ_{ab} on the edges, (26.1); inside each face A with curl $A = \kappa$, (27.2)	edges, and inside faces	$\Omega_f = d\rho$, spread with density Ω_f/A_f
V (DEC)	ρ_{ab} on the edges; 0 inside each face	edges	$\Omega_f = d\rho$, on the face boundary only

those objects exist. If a problem is naturally stated as an energy of a field, with new terms to be added and integrated face by face, build it on corner transport (Part III), where every face is flat and every integral is the scalar one with a phase. The proposition is the bridge: either set of angles can be computed from the other with (40.2).

41. The Connection Form, Part by Part

Summary. The connection form ω is the infinitesimal version of the transport: how fast a frame turns, measured against parallel transport. It appears in every part, but discretized differently: as zero inside the flat faces (Parts I to III, V), as a finite rotation per corner (Parts I to III), as a discrete 1-form on the edges, $\rho_{ab} = -\int_a^b \omega$ (Parts IV, V), and as a smooth 1-form A inside each face (Part IV). Table 5 lists them.

The smooth definition, again. In a frame field t, a tangent field is $U = t\psi$, and the covariant derivative becomes $D_X\psi = X \cdot \nabla\psi + \omega(X)\,\mathsf{T}\psi$, (15.2): the change of the coordinate, corrected by the turning $\omega(X)$ of the frame. It appeared first in Part I as the skew 2×2 matrix of (3.3). A field is parallel along a curve γ from p to q when $D_{\dot\gamma}\psi = 0$, that is $\dot\psi = -\omega(\dot\gamma)\,\mathsf{T}\psi$, so

$$\psi(q) = e^{-\mathsf{T}\int_\gamma \omega}\,\psi(p)\,: \tag{41.1}$$

the transport along a curve, in coordinates, is the exponential of minus the integral of the connection form along it. This is the bridge between the infinitesimal object ω and the finite angles ρ of the discrete methods.

Edge transport is a discrete connection form. By (41.1) along the straight edge from a to b, with the frames of the two vertex planes at its ends, $r_{ab} = e^{\mathsf{T}\rho_{ab}}$ with $\rho_{ab} \approx -\int_a^b \omega$: one number per oriented edge, changing sign with

the orientation, which is what a discrete 1-form is (Section 30). In Part IV, inside a face, $\omega(X) = -A \cdot X$, so the transport along a segment is $e^{\mathsf{T}\int A\cdot dx}$, (27.3), with the plus sign. (The sign is a convention: ω measures how the frame turns, A how a parallel vector turns in the frame, and the two are opposite.) The holonomy of a face is then $\Omega_f = \rho_{ab} + \rho_{bc} + \rho_{ca} = (d_1\rho)_f$, the discrete exterior derivative, and by Stokes's theorem it approximates $-\oint \omega = -\int_f d\omega = \int_f K\, dA$, the curvature of the face, by the structure equation $d\omega = -K\, dA$ (Remark 15.1).

Discrete changes of frame. The frame of every plane is arbitrary, and a change of frame must change the angles in a way that leaves everything geometric alone, as (15.3) does in the smooth case. Rotating the frame of vertex i by θ_i, $t_i \to t_i e^{\mathsf{T}\theta_i}$, changes its coordinates to $\psi_i' = e^{-\mathsf{T}\theta_i}\psi_i$. Rotating the frame of face f by θ_f does the same to the coordinates of the face. For the vector of the face, $e^{\mathsf{T}\rho_i^f}\psi_i$, and the transported value, $r_{ab}\psi_a = \psi_b$, to stay the same geometric vectors, the angles must change as

$$\rho_i^f \to \rho_i^f + \theta_i - \theta_f, \qquad \rho_{ab} \to \rho_{ab} - (\theta_b - \theta_a) = \rho_{ab} - (d_0\theta)_{ab}. \tag{41.2}$$

The second is the discrete form of $\omega \to \omega + d\theta$: the edge angles change by a discrete gradient. Every quantity that is a sum of angles around a closed loop is unchanged: the face holonomies Ω_f, because $d_1 d_0 = 0$; the edge loops H_{ab}, because each frame enters once with each sign; and the transport between two vertices through a face, $\rho_b^f - \rho_a^f$, loses its dependence on θ_f, which is the cancellation of the face frame noticed in Section 3. The element matrices change by $\mathrm{L}_{ab}^f \to e^{-\mathsf{T}\theta_a}\mathrm{L}_{ab}^f e^{\mathsf{T}\theta_b}$, a diagonal unitary change of basis of the whole system, and the solution by $\psi_i \to e^{-\mathsf{T}\theta_i}\psi_i$: the vectors drawn are the same.

This has a practical consequence that is easy to forget. The *phases* of the matrix entries depend on the frames, so the phases of two methods can be compared only if they use the same frames. Parts III to V all measure angles from the reference edge in the same stretched cone, so their phases can be compared entry by entry (Section 46). Part II measures from the reference edge *projected* into the plane of the vertex normal, which at a vertex whose reference edge is not in that plane is a different frame; its phases differ from those of Part III by a constant per vertex that means nothing. The quantities that can be compared across all methods are the gauge-invariant ones: the holonomies, the loops H_{ab}, the energies, and the vectors that the methods put into the faces (Section 24).

42. Cartan's Moving Frames: One Language for Parts I to IV

Summary. In Cartan's method of moving frames one does not differentiate vectors directly. One chooses an orthonormal frame on each piece of the surface, writes every field by its coordinates in that frame, and keeps track of how the frame turns with one 1-form, the connection form ω. Two equations, the structure equations, compute ω from the metric and the curvature from

ω. Every method of this text is a choice made inside this language: which frame each element and each vertex gets, what ω is inside an element, and how the frames of neighbors are related (Table 6). The same language gives a way to go beyond flat triangles: a curved element of degree two whose frames turn inside it, sketched at the end of this section.

42.1. Frames, coframes and the structure equations

On a piece of the surface, choose a positively oriented orthonormal frame field (e_1, e_2), with $e_2 = n \times e_1 = e_1\mathsf{T}$; in Part II it was (t, b). Its *coframe* is the pair of 1-forms that read the coordinates of a tangent vector in the frame,

$$\theta^1(X) = e_1 \cdot X, \qquad \theta^2(X) = e_2 \cdot X, \qquad X = \theta^1(X)\, e_1 + \theta^2(X)\, e_2. \quad (42.1)$$

The coframe carries the metric, $|X|^2 = \theta^1(X)^2 + \theta^2(X)^2$, and the area form, $dA = \theta^1 \wedge \theta^2$. The frame turns as one moves along the surface, and since it stays orthonormal, the derivative of e_1 is a multiple of e_2 and conversely:

$$\nabla_X e_1 = \omega(X)\, e_2, \qquad \nabla_X e_2 = -\,\omega(X)\, e_1. \quad (42.2)$$

This ω is the connection form of (15.1) and (3.3).

Proposition 42.1 (Structure equations). *For an orthonormal frame on a surface with Gaussian curvature K,*

$$d\theta^1 = \omega \wedge \theta^2, \qquad d\theta^2 = -\,\omega \wedge \theta^1, \qquad d\omega = -K\, \theta^1 \wedge \theta^2. \quad (42.3)$$

Proof. For a 1-form θ and two vector fields, $d\theta(X, Y) = X(\theta(Y)) - Y(\theta(X)) - \theta([X, Y])$. With $\theta^1(Y) = e_1 \cdot Y$ and the metric compatibility (3.2), $X(e_1 \cdot Y) = \nabla_X e_1 \cdot Y + e_1 \cdot \nabla_X Y = \omega(X)\, \theta^2(Y) + e_1 \cdot \nabla_X Y$. Subtracting the same with X and Y exchanged, and using that the connection has no torsion, $\nabla_X Y - \nabla_Y X = [X, Y]$ (Section 14), the last terms give $\theta^1([X, Y])$ and cancel: $d\theta^1(X, Y) = \omega(X)\theta^2(Y) - \omega(Y)\theta^2(X) = (\omega \wedge \theta^2)(X, Y)$. The same computation with $\nabla_X e_2 = -\omega(X) e_1$ gives the second equation. For the third, compute the curvature on e_1, $R(X, Y)e_1 = \nabla_X \nabla_Y e_1 - \nabla_Y \nabla_X e_1 - \nabla_{[X,Y]} e_1$. By (42.2), $\nabla_X(\omega(Y) e_2) = X(\omega(Y))\, e_2 - \omega(Y)\omega(X)\, e_1$, and the terms in e_1 cancel in the antisymmetric combination, leaving $R(X, Y)e_1 = d\omega(X, Y)\, e_2$. The Gaussian curvature is $K = R(e_1, e_2)e_2 \cdot e_1 = -R(e_1, e_2)e_1 \cdot e_2 = -d\omega(e_1, e_2)$ (the curvature operator is antisymmetric), which is the third equation. □

What the equations say. The first two determine ω. Writing $d\theta^1 = a\, \theta^1 \wedge \theta^2$ and $d\theta^2 = b\, \theta^1 \wedge \theta^2$, they give

$$\omega = a\, \theta^1 + b\, \theta^2. \quad (42.4)$$

The coframe depends only on the metric and on the choice of e_1, so ω does too: no normal, no ambient space. This is the theorem that the Levi-Civita connection is intrinsic (Section 14), in a form that can be computed. The third equation says that the curvature is the exterior derivative of the connection form, so that, by Stokes's theorem, the holonomy around a loop, $-\oint \omega$, is the curvature enclosed, $\int K\, dA$: the fact used in every part, now with its sign.

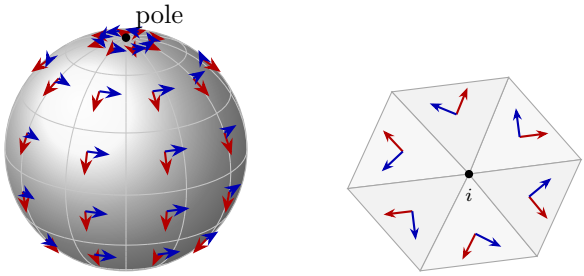


FIGURE 37. Moving frames. Left: the frame of colatitude and longitude on the sphere, $e_1 = \partial_\vartheta$ (red), $e_2 = \frac{1}{\sin\vartheta}\partial_\varphi$ (blue), with connection form $\omega = \cos\vartheta\, d\varphi$; it turns faster and faster near the pole, where it is singular. Every frame field on a sphere has singularities (Poincaré–Hopf). Right: the piecewise-constant frames of Parts I to III, one per face (along the first edge, red), with no singularity anywhere; the frames jump across the edges, and the corner angles ρ_i^f record the jumps.

Worked example 42.2. *In the plane with polar coordinates and the frame* $e_1 = \partial_r$, $e_2 = \frac{1}{r}\partial_\varphi$: $\theta^1 = dr$, $\theta^2 = r\, d\varphi$, $d\theta^1 = 0$, $d\theta^2 = dr \wedge d\varphi = \frac{1}{r}\theta^1 \wedge \theta^2$, *so* $a = 0$, $b = \frac{1}{r}$, $\omega = d\varphi$ *(the radial frame turns by the polar angle) and* $d\omega = 0$*: the plane is flat. On the unit sphere with colatitude* ϑ *and the frame* $e_1 = \partial_\vartheta$, $e_2 = \frac{1}{\sin\vartheta}\partial_\varphi$: $\theta^1 = d\vartheta$, $\theta^2 = \sin\vartheta\, d\varphi$, $d\theta^2 = \cos\vartheta\, d\vartheta \wedge d\varphi = \cot\vartheta\ \theta^1 \wedge \theta^2$, *so* $\omega = \cot\vartheta\ \theta^2 = \cos\vartheta\, d\varphi$, *and* $d\omega = -\sin\vartheta\, d\vartheta \wedge d\varphi = -\theta^1 \wedge \theta^2$: $K = 1$. *Around the circle of colatitude* ϑ *the holonomy is* $-\oint \omega = -2\pi\cos\vartheta \equiv 2\pi(1 - \cos\vartheta)$, *the area of the cap it bounds, as in Example 3.5. And the frame is singular at the poles, where* $d\varphi$ *is not defined (Figure 37).*

Changing the frame. Turning the frame by an angle field α, $e_1 \to e_1\cos\alpha + e_2\sin\alpha$, changes the connection form to $\omega + d\alpha$ and the coordinates of every vector by $e^{-\mathsf{T}\alpha}$: the change of gauge (15.3). The curvature $d\omega$ does not change, because $dd\alpha = 0$. Every frame-independent quantity of the text, the energy, the holonomies, the solution as a field of vectors, is invariant for this reason.

42.2. The energy in a moving frame, and every part as a choice

With $U = e_1\psi$, the covariant derivative is $\nabla_X U = e_1\,(X \cdot \nabla\psi + \omega(X)\,\mathsf{T}\psi)$, (15.2), and the Dirichlet energy is $\int |d\psi + \omega\mathsf{T}\psi|^2\, dA$. A finite element method in this language needs three things on every element: a frame, its connection form inside the element, and the transitions between that frame and the frames in which the unknowns are stored. Table 6 lists the choices of this text.

42.3. Three ways to build a finite element method on moving frames

1. *One constant frame per flat face.* Then $\omega = 0$ inside every face, the energy of a face is the scalar one, and all the geometry goes into the

TABLE 6. Every method of the text as a choice of moving frames: the frame inside an element, its connection form there, the frame of each unknown, and the transition between them.

method	frame inside an element	ω inside	frame of the unknowns	transition
I, II	face frame t_f, first edge, constant	0	vertex frame from n_i	ρ_i^f, rolling
III	x axis of the layout, constant	0	reference edge of the stretched cone	ρ_i^f, bisectors
IV	layout frame with prescribed curvature	$-A \cdot X$, $d\omega = -\kappa\, dA$	stretched cone	σ_a, and $\theta_a(x)$ inside
V (DEC)	none: the element is not a field	dropped	stretched cone	ρ_{ab} on edges
P2 (below)	frame of the curved element, $e_1 = \partial_1 F/\lvert\partial_1 F\rvert$	from (42.4), varies	stretched cone at vertices, edge tangent at edge nodes	per node, and parallel transport inside

transitions. This is Parts I to III; Cartan's language adds the explanation of why the element is so simple, and Part IV is the variant in which the frame of the face is allowed to turn with a prescribed curvature.

2. *One frame field for the whole surface.* Store the two components u^1, u^2 of the field in a global frame, interpolate each with scalar hat functions, and keep ω in the energy. No transitions are needed. But a frame field on a closed surface other than a torus must have singularities, by the Poincaré–Hopf theorem (the indices of a vector field add up to χ), and ω blows up at them, as $\cos\vartheta\, d\varphi$ does at the poles. The approach works on a plane domain, a torus or a single parametrized patch; on a sphere it needs several patches with transition rotations between them, which brings back the discrete transports of option 1.
3. *One frame per curved element.* Let the elements themselves be curved, give each the frame of its own parametrization, compute ω inside it from (42.4), and relate the frames of neighbors by transitions at the nodes. This is the natural higher-order extension of the text, and the one sketched next. Chun's method of moving frames solves conservation laws on curved surfaces in this way, with a frame per element of high order [1].

42.4. A curved element of degree two in moving frames: a sketch

This element has not been implemented; the steps below say how it would be built, and where the choices are. It extends Part IV: there, a flat face was given a connection with a prescribed curvature; here, a curved face carries its own frame, and its connection form and curvature are those of its geometry.

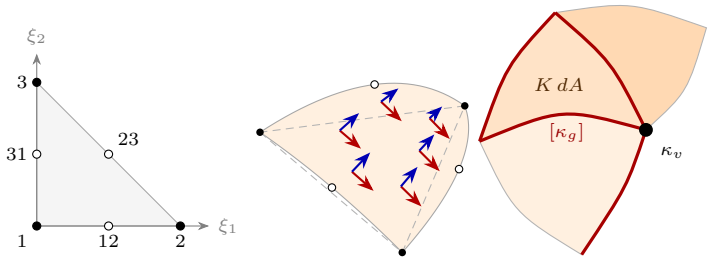


FIGURE 38. A curved element of degree two. Left: the reference triangle with its six nodes, three vertices and three edge nodes. Middle: its image on the unit sphere under the quadratic map F through six points of the sphere, with the moving frame of the element, $e_1 = \partial_1 F/|\partial_1 F|$ (red) and $e_2 = n \times e_1$ (blue), at a few points: the frame turns inside the element, and its connection form ω measures how. Right: where the curvature of a mesh of such elements lives: inside the elements ($K\,dA$, shaded), on the edges (the jump $[\kappa_g]$ of the geodesic curvature of the shared edge curve, red), and at the vertices (the angle defect κ_v of the curved corners, dot), (42.7).

1. Geometry. Each element has six nodes, its three vertices and one node on each edge (on a smooth surface, for instance, the midpoint of the edge projected onto the surface). With the barycentric coordinates $\lambda_1 = 1-\xi_1-\xi_2$, $\lambda_2 = \xi_1$, $\lambda_3 = \xi_2$ of the reference triangle, the six quadratic shape functions are

$$N_a = \lambda_a(2\lambda_a - 1) \quad (a = 1, 2, 3), \qquad N_{ab} = 4\lambda_a\lambda_b \quad (ab = 12, 23, 31), \tag{42.5}$$

each equal to 1 at its node and 0 at the other five. The element is the image of the map $F(\xi) = \sum_n N_n(\xi)\, x_n$, a curved triangle in space. Its Jacobian $J(\xi) = [\,\partial_1 F \;\; \partial_2 F\,]$ is a 3×2 matrix that varies over the element, its metric is $G(\xi) = J^T J$ (the Gram matrix of Part III, now a function of ξ), its area element $\sqrt{\det G}\, d\xi$, and its normal $n = \partial_1 F \times \partial_2 F/|\partial_1 F \times \partial_2 F|$. Two neighbors share the three nodes of their common edge, so they share the edge curve exactly: the surface is continuous, but its tangent plane jumps across the edges.

2. Frame and connection form. The frame of the element is $e_1 = \partial_1 F/|\partial_1 F|$, $e_2 = n \times e_1$: Gram–Schmidt applied to J, which depends only on G. Its connection form, by (42.2), has the components

$$\omega_k = \omega(\partial_k F) = e_2 \cdot \partial_k e_1 \qquad (k = 1, 2), \tag{42.6}$$

computed from the second derivatives of F; alternatively, (42.4) computes it from G and its first derivatives alone, which makes the element intrinsic for a given metric of the curved element. As a vector of the tangent plane, $\omega(X) = w \cdot X$ with $w = J\,G^{-1}(\omega_1, \omega_2)^T$, the same formula as for the gradients

of the hat functions, $\nabla N_n = J\,G^{-1}\hat{\nabla} N_n$ (Proposition 7.1, with B replaced by $J(\xi)$). The curvature inside the element is $K = -(\partial_1\omega_2 - \partial_2\omega_1)/\sqrt{\det G}$, by (42.3).

3. Where the curvature lives. On a mesh of flat triangles all the curvature is at the vertices. On a mesh of curved elements it lives in three places. Inside each element there is the smooth curvature K. On each edge, the edge curve has a geodesic curvature κ_g^K seen from one element K and κ_g^L seen from the other, each measured towards the inside of its own element; on a smooth surface they cancel, here their sum $[\kappa_g] = \kappa_g^K + \kappa_g^L$ is a curvature concentrated on the edge. At each vertex there is the angle defect $\kappa_v = 2\pi - \Theta_v$ of the curved corners, measured between the tangents of the edge curves. The Gauss–Bonnet theorem of each element, $\int_K K\,dA + \oint_{\partial K} \kappa_g\,ds + \sum(\pi - \alpha) = 2\pi$, summed over a closed mesh with $3F = 2E$, gives

$$\sum_K \int_K K\,dA + \sum_e \int_e [\kappa_g]\,ds + \sum_v \kappa_v = 2\pi\chi : \tag{42.7}$$

the same total as always, now spread over the three dimensions of the mesh. For flat triangles the first two sums vanish and only the defects remain, which is the setting of Parts I to V. As the curved mesh is refined on a smooth surface, the edge jumps and the vertex defects shrink, and the curvature moves into the elements, where the frame computes it exactly.

4. Nodes and their planes. The unknowns are complex numbers ψ_n at the vertices and at the edge nodes, each in a plane of its own. At a vertex, use the stretched cone of Part III, with the angles of the curved corners. At an edge node, the two elements each see half a turn, so the cone is flat ($s = 1$) and needs no stretching; take its reference direction along the tangent of the edge curve. The transition ρ_n^K into the frame of element K is the bisector rule (22.1) at a vertex, and at an edge node the angle of the edge tangent in the frame of K.

5. Trial functions: transport, then interpolate. Following Part IV, carry each nodal value from its node to the point ξ by parallel transport inside the element, and then interpolate:

$$u_K(\xi) = \sum_n N_n(\xi)\; e^{\mathsf{T}(\rho_n^K + \vartheta_n(\xi))}\,\psi_n, \qquad \vartheta_n(\xi) = -\int_{\xi_n}^{\xi} \omega, \tag{42.8}$$

with the integral along the straight segment from the node ξ_n to ξ in the reference triangle, as in (41.1); the field is $U = e_1\,u_K$. On an edge, the segment between two of its points runs along the edge, so the transport there is parallel transport along the edge curve, which the two elements agree on up to the jump $[\kappa_g]$. The traces of neighbors therefore agree up to a term proportional to $[\kappa_g]$, which vanishes under refinement on a smooth surface: a variational crime of the kind DEC commits on the faces, here confined to the edges. The cheaper choice $\vartheta_n = 0$, interpolating in the frame of the element without transport, makes the traces depend on how each element's frame turns along the edge, and is not recommended.

6. The covariant derivative and the element matrices. By the product rule, as in Part IV,

$$D\big(N_n\, e^{\mathsf{T}(\rho_n+\vartheta_n)}\big) = e^{\mathsf{T}(\rho_n+\vartheta_n)}\,\big(\nabla N_n + \mathsf{T}\, N_n\,(\nabla\vartheta_n + w)\big), \tag{42.9}$$

where $\nabla\vartheta_n + w$ plays the role of $w_a = \nabla\theta_a - A$ of (27.6) and vanishes at the node n. The element matrices are 6×6 and Hermitian,

$$\mathrm{L}^K_{ab} = \int_K e^{\mathsf{T}(\rho_b+\vartheta_b-\rho_a-\vartheta_a)}\,\overline{(\nabla N_a + \mathsf{T}N_a(\nabla\vartheta_a + w))}\cdot(\nabla N_b + \mathsf{T}N_b(\nabla\vartheta_b + w))\,dA, \tag{42.10}$$

$$\mathrm{M}^K_{ab} = \int_K e^{\mathsf{T}(\rho_b+\vartheta_b-\rho_a-\vartheta_a)}\,N_aN_b\,dA,$$

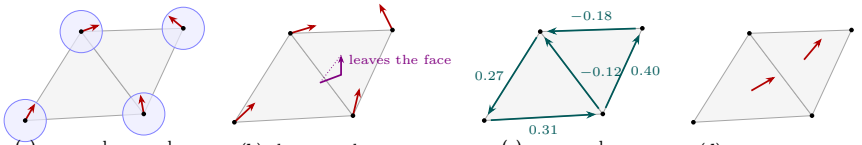
integrated with a Gauss rule on the reference triangle, $dA = \sqrt{\det G}\,d\xi$, with a one-dimensional rule for each ϑ_n at each Gauss point. Two cautions. The integrands are not polynomials (they contain G^{-1}, $\sqrt{\det G}$ and the phases), so the rule must be chosen by a convergence test, as in Part IV. And the row-sum lumping of (8.3) fails for quadratic elements, because the vertex functions N_a integrate to zero over a flat triangle: use the consistent mass, or a lumping designed for quadratic elements.

7. Assembly and solution. Unchanged: the unknowns are the vertex and edge nodes, about four times as many as vertices on a closed mesh ($V + E \approx 4V$), the matrices are assembled element by element into a Hermitian system, and the backward Euler step and the Dirichlet row are those of Section 10.

What it would buy, and what it would cost. For a smooth solution, finite elements of degree two approximate the field to one order higher than linear ones [15], provided the geometry is approximated to the same order, which the curved elements do, and the edge crime of step 5 is small enough; this has not been verified here. The gain matters most where the answer is a smooth field or an energy, as for the rotation field of Section 43, where even the flat curved element of Part IV was seven to ten times more accurate. For the vector heat method itself, the time step and the normalization near the cut locus may limit the accuracy before the element does. The costs are a mesh with edge nodes, quadrature with nested integrals for the transports, the consistent mass, and four times as many unknowns.

43. Vector-Valued Forms, Scalar 1-Forms, and Two Laplacians

Summary. Two different things go by the name of "1-form" in this subject. A *tangent-valued* 1-form takes a direction and returns a tangent vector; the covariant derivative ∇U is one, and every energy of this text is its squared norm. A *scalar* 1-form takes a direction and returns a number; a tangent field U becomes one, $U^\flat = \langle U, \cdot\rangle$, and can then be stored by numbers on the edges and differentiated with d, without any connection. The natural Laplacian of scalar 1-forms, the Hodge Laplacian, is not the connection Laplacian: on a surface they differ by the Gaussian curvature, $\Delta_{\mathrm{H}} = \nabla^*\nabla + K$, the



(a) a complex number ψ_i per vertex, in its own plane; Parts I–V

(b) three numbers $U_i \in \mathbb{R}^3$ per vertex; extrinsic surface FEM

(c) one number $\int_e U \cdot dx$ per edge: a 1-form; DEC, Whitney

(d) one vector per face, in the face plane; face-based fields

FIGURE 39. Four ways to store a tangent field on a mesh. (a) One complex number per vertex, in a plane invented for that vertex: this text. (b) Three numbers per vertex, a vector of space, which in general leaves the plane of the faces between the vertices: the extrinsic surface finite elements of Section 44. (c) One number per edge, the integral of the field along it, a scalar 1-form: DEC and Whitney forms. (d) One vector per face, in the plane of the face: face-based methods.

Weitzenböck identity (43.3). The program checks it on a rotation field of the sphere.

43.1. Four ways to store a field

Figure 39 shows the four discretizations of a tangent field in common use. This text uses (a): its unknowns are the vertex values ψ_i, and the connection is what makes them comparable. The other three avoid inventing vertex planes, each in its own way. (b) stores the field in the common space $\mathbb{R}^3$, where vectors at different points can be added, and pays with a field that is not tangent (Section 44). (c) stores, instead of a vector, how much of it points along each edge: the edge is the same edge for both of its faces, so no transport is needed, and the exterior derivative is exact. (d) stores one vector per face, in the plane of the face, which exists; comparing two neighboring faces needs only the unfolding across their common edge, which is exact and intrinsic. The constraint of this text, unknowns at the vertices, rules out (c) and (d); they are the natural choice for problems built on divergence and curl, and for direction fields with singularities, respectively [3, 18].

43.2. Tangent-valued 1-forms

The covariant derivative of a tangent field is the map $X \mapsto \nabla_X U$ from tangent vectors to tangent vectors: a 1-form with values in the tangent plane, in a frame the 2×2 matrix of Section 3. The Dirichlet energy (4.1) is its squared norm. In the discrete methods it appears as:

- inside a face of Parts I to III, the constant matrix $\sum_a (e^{\mathsf{T}\rho_a^f}\psi_a) \otimes \nabla\phi_a$ of (6.3): a tangent-valued 1-form that is constant on the face;
- inside a face of Part IV, $\sum_a e^{\mathsf{T}(\sigma_a+\theta_a)}\psi_a \otimes (\nabla\phi_a + \mathsf{T}\phi_a w_a)$, which varies across the face;

- in DEC, the covariant exterior derivative $(d^{\nabla}\psi)_{ab} = \psi_b - r_{ab}\psi_a$, (33.1), "a discrete 1-form with values in the plane of the end vertex": it integrates ∇U along the edge.

There is one more tangent-valued 1-form behind every formula, although it is rarely named: the identity map $X \mapsto X$, called dx or the *solder form*. Its discrete version is the list of edge vectors $x_b - x_a$, and in the intrinsic parts the Gram matrix G_f is what remains of it once the embedding is forgotten. Any energy of a new problem in this family is a quadratic form in these objects, which is why they are the right interface for extending the method (Section 49).

43.3. Scalar 1-forms and the Hodge Laplacian

The metric turns a tangent field U into the scalar 1-form $U^\flat$, $U^\flat(X) = U \cdot X$. On a mesh, $U^\flat$ is stored by its integrals along the edges, $\eta_{ab} = \int_a^b U \cdot dx$ (Figure 39c). The exterior derivative then gives two derivatives without any connection: $d\eta$, on the faces, the integral of $\operatorname{curl} U$ (Green's theorem), and the codifferential $\delta\eta = -\star_0^{-1} d_0^T \star_1 \eta$, at the vertices, minus the divergence of U averaged over the dual cell (Section 31). The Laplacian of 1-forms is the *Hodge Laplacian* $\Delta_{\mathrm{H}} = d\delta + \delta d$, whose energy is

$$\int_\Omega \big((\operatorname{div} U)^2 + (\operatorname{curl} U)^2\big)\, dA \;\approx\; \sum_f \frac{(d_1\eta)_f^2}{A_f} + \sum_i \frac{\big((d_0^T \star_1 \eta)_i\big)^2}{(\star_0)_{ii}}, \qquad (43.1)$$

where $\operatorname{div} U = \operatorname{tr} \nabla U$ and $\operatorname{curl} U$ is the antisymmetric part, $e_2 \cdot \nabla_{e_1} U - e_1 \cdot \nabla_{e_2} U$ in an orthonormal frame. This energy is not the Dirichlet energy $\int |\nabla U|^2$: it measures only the trace and the antisymmetric part of ∇U, not its symmetric traceless part. How the two energies differ is a classical identity.

43.4. The Weitzenböck identity on a surface

Proposition 43.1. *For a tangent field U on a surface with Gaussian curvature K,*

$$|\nabla U|^2 + K|U|^2 = (\operatorname{div} U)^2 + (\operatorname{curl} U)^2 - \operatorname{div}\big((\operatorname{div} U)\, U - \nabla_U U\big), \qquad (43.2)$$

so that on a closed surface

$$\int_\Omega |\nabla U|^2\, dA + \int_\Omega K|U|^2\, dA = \int_\Omega \big((\operatorname{div} U)^2 + (\operatorname{curl} U)^2\big)\, dA, \qquad \Delta_{\mathrm{H}} = \nabla^*\nabla + K. \qquad (43.3)$$

Proof. Work in an orthonormal frame at a point, and write $a_{jk} = e_j \cdot \nabla_{e_k} U$ for the entries of the matrix of ∇U. Then $|\nabla U|^2 = \sum_{jk} a_{jk}^2$, $\operatorname{div} U = a_{11} + a_{22}$, $\operatorname{curl} U = a_{21} - a_{12}$, and expanding the squares,

$$(\operatorname{div} U)^2 + (\operatorname{curl} U)^2 = |\nabla U|^2 + 2\,(a_{11}a_{22} - a_{12}a_{21}) = |\nabla U|^2 + 2 \det \nabla U.$$

It remains to show that $2 \det \nabla U = K|U|^2 + \operatorname{div}\big((\operatorname{div} U)U - \nabla_U U\big)$. Use index notation with summation over repeated indices, U^j the components of U and

$\nabla_k U^j$ those of ∇U. The divergence of $(\operatorname{div} U)U$ is $(\nabla_k U^k)^2 + U^k \nabla_k (\operatorname{div} U)$. The divergence of $\nabla_U U = U^k \nabla_k U^j$ is

$$\nabla_j (U^k \nabla_k U^j) = (\nabla_j U^k)(\nabla_k U^j) + U^k \nabla_j \nabla_k U^j .$$

Covariant derivatives do not commute, and their commutator is the curvature: $\nabla_j \nabla_k U^j - \nabla_k \nabla_j U^j = \mathrm{Ric}_{kl} U^l$, the Ricci identity [4, 14], and on a surface the Ricci tensor is K times the metric. So $U^k \nabla_j \nabla_k U^j = U^k \nabla_k (\operatorname{div} U) + K|U|^2$, and subtracting,

$$\operatorname{div}\big((\operatorname{div} U)U - \nabla_U U\big) = (\operatorname{tr} \nabla U)^2 - \operatorname{tr}\big((\nabla U)^2\big) - K|U|^2 = 2 \det \nabla U - K|U|^2 ,$$

using $(\operatorname{tr} A)^2 - \operatorname{tr}(A^2) = 2 \det A$ for 2×2 matrices. This proves (43.2). On a closed surface the divergence integrates to zero, which gives the first part of (43.3); the second is the same statement for the operators, since the left side is $\int (\nabla^* \nabla U + KU) \cdot U$ by Green's identity (4.2) and the right side is $\int \Delta_{\mathrm{H}} U^\flat \cdot U^\flat$. □

When does the difference matter? The two operators have the same leading part, $\nabla^* \nabla$, and differ by a term of order zero, the multiplication by K. For the short times of the vector heat method the leading part dominates, so both flows carry a vector along the shortest paths to leading order; the term K changes how fast the magnitude decays, by a factor of about $e^{-K \Delta t}$, and the corrections of higher order. For an energy, an eigenvalue or a smoothest field it changes the answer at first order: where $K > 0$ the Hodge energy charges $|U|^2$ more than the Dirichlet energy, where $K < 0$ less, and the minimizers of the two differ wherever the surface is curved. So $\nabla^* \nabla$ is the operator to discretize when the problem is about parallel transport, as here, and Δ_{H} when it is about divergence and curl. (The extra energy of a curved face in Part IV, $\frac{\kappa^2}{4} \int \phi^2 |\bar{x}_a - x|^2$, is a relative of the term $K|U|^2$: both are the energy that curvature adds to a field that is as parallel as it can be.)

Worked example 43.2. *On the unit sphere, $K = 1$, take the rotation about the z axis, $U(x) = e_z \times x$ (Figure 40). At the point of colatitude ϑ, $|U| = \sin \vartheta$. For a tangent direction X, $D_X U = e_z \times X$; writing $e_z = (e_z \cdot n)\, n +$ (a tangent vector), the tangent part is $\nabla_X U = (e_z \cdot n)\, n \times X = \cos \vartheta \; JX$, a quarter turn scaled by $\cos \vartheta$. So $\operatorname{div} U = 0$, $\operatorname{curl} U = 2 \cos \vartheta$, $|\nabla U|^2 = 2 \cos^2 \vartheta$, and with $\int_{S^2} \cos^2 \vartheta \, dA = \frac{4\pi}{3}$,*

$$\int |\nabla U|^2 = \frac{8\pi}{3}, \qquad \int K|U|^2 = \int \sin^2 \vartheta = 4\pi - \frac{4\pi}{3} = \frac{8\pi}{3}, \qquad \int (\operatorname{curl} U)^2 = \frac{16\pi}{3} :$$

the Dirichlet energy is half of the Hodge energy, and the curvature term is the other half, as (43.3) requires.

The check. `ga_fem` computes the three integrals on the geodesic spheres of Section 24. For the Dirichlet energy it puts the exact field into each method's vertex planes (for Part II by the vertex frame; for Parts III to V by carrying the direction of U in each face back to the vertex with the method's own rule, averaged over the corners) and evaluates $\psi^H \mathrm{L}\, \psi$. For the Hodge energy it integrates $U^\flat$ along the edges, which is exact here, $\eta_{ab} = \int_a^b (e_z \times x) \cdot dx =$

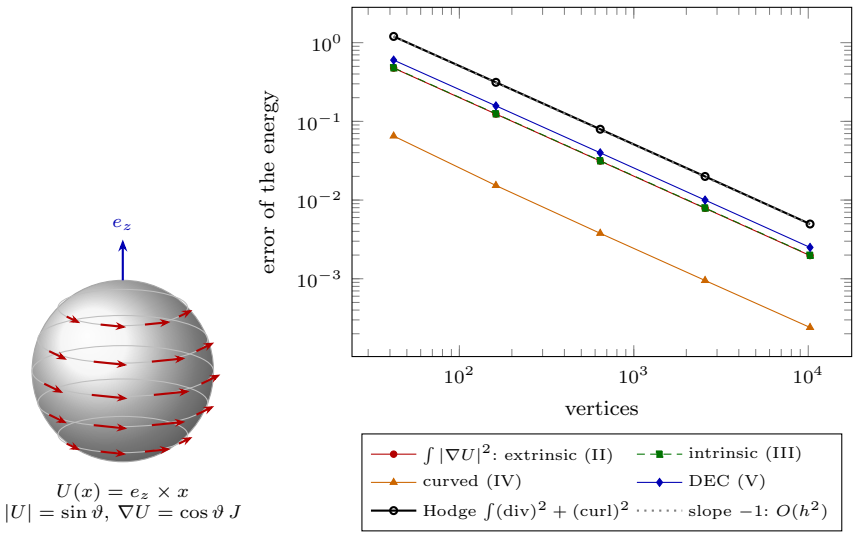


FIGURE 40. Left: the rotation field of Example 43.2. Right: the errors of its Dirichlet energy, $\psi^H \mathrm{L}\psi$ with the four connection Laplacians, and of its Hodge energy (43.1), on the geodesic spheres of 42 to 10242 vertices. All decrease like h^2. The curved element of Part IV is the most accurate by a factor of seven to ten.

$e_z \cdot (x_a \times x_b)$ along the straight edge, and evaluates (43.1) with the matrices of Part V. For the curvature term it uses the angle defects, $\sum_i \kappa_i |U(x_i)|^2$. The Hodge energy and the curvature term, in code:

```
1072        // the Hodge energy of the 1-form U^flat: eta_ab = int_ab U . dx = z . (x_a x
                x_b), exact along the chord
1073        std::map<const Edge*, double> eta;
1074        for (const Edge* e : mesh.getEdges()) eta[e] = z.dot(e->vertex->p.cross(e->pair
                ->vertex->p));
1075        double curl = 0, divergence = 0, curvature = 0;
1076        for (Face& f : mesh.getFaces()) {
1077            Vertex* v[3];
1078            Edge* h[3];
1079            intrinsic::faceCorners(f, v, h);
1080            const double circulation = eta[h[0]] + eta[h[1]] + eta[h[2]]; // d1 eta,
                    the integral of curl U over the face
1081            curl += circulation * circulation / G.faces[f.ID].area; // *2 = 1 / A_f
1082        }
1083        std::vector<double> codiff(n, 0.0); // d0^T *1 eta: minus the integral of div U
                over the dual cell
1084        auto cotOpposite = [&](const Edge* e) {
1085            Vertex* v[3];
1086            Edge* h[3];
1087            intrinsic::faceCorners(*e->face, v, h);
1088            int k = 0;
1089            while (h[k] != e) ++k;
1090            return 1.0 / std::tan(G.faces[e->face->ID].angle[(k + 2) % 3]);
1091        };
1092        for (const Edge* e : mesh.getEdges()) {
1093            if (e->vertex->ID > e->pair->vertex->ID) continue;
```

```
            const double w = 0.5 * (cotOpposite(e) + cotOpposite(e->pair)) * eta[e];
            codiff[e->pair->vertex->ID] += w;
            codiff[e->vertex->ID] -= w;
        }
        for (int i = 0; i < n; ++i) {
            divergence += codiff[i] * codiff[i] / star0[i];
            curvature += (2.0 * M_PI - G.vertices[i].totalAngle) * U(i).squaredNorm();
        }
```

vertices	$\int \lvert\nabla U\rvert^2 \to 8\pi/3 = 8.37758$ II	III	IV	V	Hodge $\to 16\pi/3$ $= 16.75516$	$\sum \kappa_i \lvert U_i\rvert^2$
42	7.90207	7.89790	8.31244	7.77699	15.55458	8.37758
162	8.25370	8.25269	8.36226	8.22018	16.44170	8.37758
642	8.34625	8.34597	8.37380	8.33773	16.67571	8.37758
2562	8.36972	8.36964	8.37663	8.36758	16.73520	8.37758
10242	8.37561	8.37559	8.37734	8.37508	16.75016	8.37758

Four observations. First, every column converges to its limit, and Hodge minus Dirichlet tends to $8\pi/3$, the curvature term: the identity holds in the limit for all four methods. It does not hold exactly on a coarse mesh (on 42 vertices, Hodge minus DEC is 7.78), because the discrete Hodge energy and the discrete connection energies are built from different ingredients. Second, the curvature column is exact at every resolution; that is a property of these meshes, not of the method: the angle defects of a mesh with icosahedral symmetry have a second moment $\sum_i \kappa_i x_i x_i^T$ proportional to the identity, so $\sum_i \kappa_i(1 - z_i^2) = 4\pi - \frac{1}{3} \cdot 4\pi$ exactly. Third, the divergence part of (43.1) is 10^{-13} on the coarsest sphere and $8 \cdot 10^{-6}$ on the finest, against an exact value of zero. Fourth, and most interesting: on the energy of a smooth field, the curved element of Part IV is seven to ten times more accurate than the others (Figure 40), while on the direction of the vector heat method it was not better than Part III (Section 29). Its trial functions contain the curvature of the face, and the energy of a smooth field on a curved surface is, in part, curvature energy. For problems whose answer is an energy, or an eigenvalue, that difference matters.

44. The Standard Method of Numerical Analysis Needs No Discrete Connection

Summary. In the numerical analysis of partial differential equations on surfaces, the usual finite element method for vector fields stores at every vertex the three components of a vector of $\mathbb{R}^3$, interpolates each with the scalar hat functions, and recovers the covariant derivative by projection, $\nabla_X U = P(D_X U)$ [7, 8]. Vectors at different vertices are then added directly, and no transport is discretized. The price is that the discrete field is not tangent, which must be corrected by a penalty or a Lagrange multiplier, and that the normals used in the projection must be accurate.

The idea. The Gauss formula (14.1) says that the covariant derivative of a tangent field is the tangent part of its derivative in space. In space, the connection is trivial: a vector at x and a vector at y are both arrows of $\mathbb{R}^3$

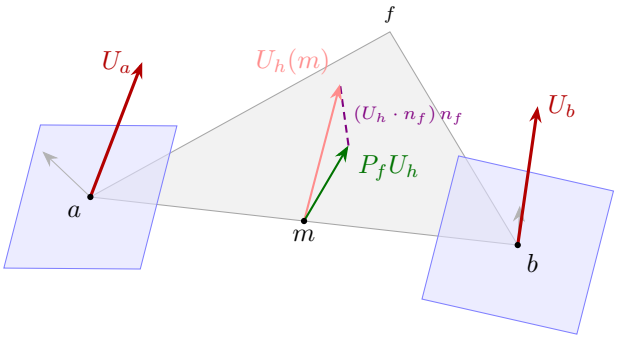


FIGURE 41. Extrinsic surface finite elements. The vectors U_a and U_b are tangent to the planes of their vertices (blue), which are tilted with respect to the face f. Interpolated in $\mathbb{R}^3$, their average at the midpoint m has a component along the face normal (dashed, exaggerated here): the discrete field leaves the surface, and the method must penalize or remove that component.

and can be subtracted. So if the field is stored as an $\mathbb{R}^3$-valued function, $U_h = \sum_a \phi_a U_a$ with $U_a \in \mathbb{R}^3$, its derivative is the ordinary derivative of three scalar finite element functions, and the covariant derivative is obtained by projecting with $P = I - nn^T$. The connection is still the Levi-Civita connection of the surface, but it is the one induced from $\mathbb{R}^3$, and it needs no discretization because the connection of $\mathbb{R}^3$ needs none. In this text, Section 6 rejected exactly this interpolation, because the interpolated field is not tangent (Figure 8); this method accepts it and corrects for it.

The discrete problem. A backward Euler step of the vector heat equation becomes: minimize, over the vectors $U_a \in \mathbb{R}^3$,

$$\frac{1}{2}\int_\Omega |U_h - U^0|^2\, dA + \frac{\Delta t}{2}\int_\Omega \left|P\,(DU_h)\,P\right|^2 dA + \frac{\tau}{2}\int_\Omega (U_h \cdot n)^2\, dA, \quad (44.1)$$

where DU_h is the 3×3 matrix of derivatives of the three components along the face, P removes its normal row and column, and τ is a penalty parameter. Three unknowns per vertex instead of one complex number; the stiffness is the scalar stiffness for each component, mixed by P (Figure 39b).

Where the connection went. Two subtleties replace the discrete connection, and both are where this method can go wrong.

1. *The field is not tangent.* The penalty τ is a tuning parameter: too small and the field leaves the surface, too large and the system becomes ill conditioned, or the method locks. A Lagrange multiplier for $U_h \cdot n = 0$ avoids the parameter at the cost of a saddle point system.
2. *Differentiate first or project first?* For a field that is not exactly tangent, $P\,D(PU_h)\,P = P\,(DU_h)\,P - (U_h \cdot n)\,\mathcal{H}$, with $\mathcal{H} = Dn$ the shape operator (Weingarten map), as one sees by differentiating $PU_h = U_h - (U_h \cdot$

n) n. The two formulations differ by a curvature term times the normal component, and each formulation must say which one it means.

The analysis of Hansbo, Larson and Larsson [8] shows that, for the optimal rate of convergence, the normal used in the penalty term must be more accurate than the piecewise constant normals of the faces. So the extrinsic information that this text removed in Part III is, in this method, essential: on a bent flat mesh such as the Miura-ori it would inherit errors from its normals, while Parts III to V are exact there.

Where it fits. It is not Part I. Part I uses $\mathbb{R}^3$ only to compute frames and rotations; its unknowns are two numbers per vertex, in a vertex plane, compared through a discrete connection. It is a connection method with extrinsic vertex planes. The extrinsic surface finite element method is the natural choice when the surface is known as a smooth surface (or approximated to higher order), when the problem couples tangential and normal quantities (fluid membranes, shells, moving interfaces), when error estimates are needed, and when general finite element libraries are to be used; the connection methods of this text are the natural choice when the mesh is the only data, when the answer should not depend on bending, when the field must be tangent by construction, and when the structure (holonomy, Gauss–Bonnet, index of singularities) must be exact.

	discrete connection (II–V)	extrinsic surface FEM
unknowns per vertex	one complex number	three real numbers
tangent by construction	yes	no: penalty or multiplier
geometry it needs	angles (III–V) or normals (II)	accurate normals
intrinsic	III–V yes	no
exact on bent flat meshes	III–V yes	no
general FEM libraries	needs custom phases	out of the box
error analysis	little	mature [7, 8]

45. Matrices, Geometric Algebra, Complex Numbers

Summary. The three notations compute the same numbers (Section 38). Geometric algebra is indispensable in Part II, where planes of space are tilted against each other; once the method is intrinsic, every plane is an abstract plane, the even part of the algebra of a plane is the complex numbers, and Parts III to V are written with `std::complex`. Geometric algebra becomes necessary again in three dimensions, where rotations do not commute.

What each notation does. Matrices (Part I) are the most explicit: every number is visible, which is why the code `FEM.cpp` was written that way, but a rotation is four numbers and a frame six, and nothing distinguishes the geometric content from the choice of basis. Geometric algebra (Part II) names the objects themselves: the tangent plane is the bivector $\mathsf{T} = In$, a rotation of the plane is $e^{\mathsf{T}\theta}$, a rotation of space taking one plane to another is a rotor,

the gradient of a hat function is a vector of the reciprocal frame. The frame-independent content is then visible, and the formulas are shorter. Complex numbers are what remains of geometric algebra in a single plane: with T playing the role of i, the even elements $\alpha + \beta\mathsf{T}$ multiply exactly as complex numbers do, (12.4).

Why the intrinsic parts need only complex numbers. In Parts III to V there is no space: a vertex plane is a stretched cone, a face is its layout in $\mathbb{R}^2$. Every rotation is a rotation of a plane, every transport is a multiplication by a unit complex number, and every bivector of the problem is a multiple of the single bivector of the plane. Geometric algebra still gives the right *reading*: the connection form is bivector-valued (it generates rotations), the curvature Ω_f is a bivector-valued 2-form, and the identity $d^\nabla d^\nabla = 1 - e^{\mathsf{T}\Omega_f}$ says that the curvature acts on the field by the rotation it generates. But every bivector being a multiple of T, the code is complex arithmetic. (The remark at the end of Part III relates this reading to the gauge theory of gravity; nothing in the text depends on it.)

When geometric algebra is needed again. Three situations bring back what Part II had:

- *Three-dimensional frames.* Frame fields in volumes, or the transport of a full frame of space along a surface, are rotations of $\mathbb{R}^3$, rotors of $\mathbb{G}_3$ (or unit quaternions). They do not commute, so the holonomy around a loop is no longer an angle that can be added, and the shares of curvature of Section 39 must be replaced by products of rotors.
- *Coupling with extrinsic quantities.* Projections, normals, the shape operator of Section 44: everything of Part II.
- *Fields of higher grade.* Bivector fields (a vorticity, an infinitesimal rotation at every point) and operators that mix grades, such as the vector derivative $\nabla = \nabla\cdot + \nabla\wedge$, whose outer part is the metric-free exterior derivative of DEC.

A practical design follows: write the core of a method with complex numbers, but behind an interface "transport" whose implementation could later be a rotor, so that the same assembly code serves the three-dimensional case.

46. One Mesh Through Every Step

Summary. The leaning pyramid of Example 21.2, traced through the steps of Figure 33 with the four methods. The program prints every number (Worked examples 9 and 10 of `ga_fem`).

Worked example 46.1. *The mesh: an apex over a regular pentagon of radius* 1 *at height* 0.6180*, moved by* 0.4 *along x; five faces* $f = 0, \dots, 4$*, face* f *with the vertices (apex,* $f+1$*,* $f+2$*), the base indices taken modulo* 5 *in* $1, \dots, 5$*. The apex is the only interior vertex.*

Step 1, geometry. *The corner angles at the apex are* 70.488°*,* 51.373°*,* 46.814°*,* 51.373°*,* 70.488° *for* $f = 0, \dots, 4$*, with* $\Theta = 290.534°$*, defect* $\kappa =$ 69.466°*.*

Step 2, vertex planes. *Part II: the plane of the vertex normal, which is vertical (Example 21.2). Parts III–V: the cone stretched by* $s = 1.239096$. *The base vertices are boundary vertices,* $s = 1$.

Step 3, transport. *The corner connections of the apex into its five faces, and the edge transports from the apex to the first base vertex of each face:*

face	α	ρ^f *(II)*	ρ^f *(III)*	*edge*	ρ_{ab} *(IV, V)*
0	$70.488°$	$-107.296°$	$-95.767°$	$0 \to 1$	$26.828°$
1	$51.373°$	$167.036°$	$179.177°$	$0 \to 2$	$-38.363°$
2	$46.814°$	$107.943°$	$116.066°$	$0 \to 3$	$-107.465°$
3	$51.373°$	$53.408°$	$57.514°$	$0 \to 4$	$177.063°$
4	$70.488°$	$-13.146°$	$-8.427°$	$0 \to 5$	$100.500°$

The numbers of II and III are close but not equal; on the upright pyramid they are equal $(-78°, -150°, 138°, 66°, -6°)$. *Along the edge* $0 \to 1$, *the corner transport of Part III through face* 0 *is the edge transport turned by* $-8.427°$, *and through face* 4 *turned by* $+8.427°$: *half of the stretching* $(s-1)\cdot 70.488° = 16.853°$ *of the apex corner of each face, as Proposition 40.1 says. Along* $0 \to 2$, *through faces* 1 *and* 0: $-6.142°$ *and* $+8.427°$.

Step 4, curvature. *Part IV and Part V: the face holonomies* $\Omega_f = (s-1)\alpha$, $16.853°$, $12.283°$, $11.193°$, $12.283°$, $16.853°$, *adding up to* $69.466°$. *Part III: the edge loops (40.3) of the edges* $0 \to 1, \ldots, 0 \to 5$, $16.853°$, $14.568°$, $11.738°$, $11.738°$, $14.568°$, *adding up to* $69.466°$. *Part II: the same loops with its own corner transport,* $23.662°$, $15.181°$, $7.720°$, $7.720°$, $15.181°$, *the gaps of Figure 35, adding up to* $69.466°$.

Steps 5 and 6, element. *The entry (apex, vertex* 1*) of the element matrix of face* 0. *Parts III and V have the same modulus,* $A_f\,|\nabla\phi_0 \cdot \nabla\phi_1| = 0.52356$, *and phases* $161.599°$ *and* $153.172°$: *they differ by* $8.427°$, *the correction of Proposition 40.1. The curved element of Part IV has the modulus* 0.52427 *and the phase* $163.738°$, *and its diagonal entry is* 0.74917 *against* 0.74794 *for the flat element: the curvature costs energy. (On the upright pyramid the same three phases are* $138°$, $132°$ *and* $141.969°$, *and the diagonals* 0.57805 *against* 0.57735, *Example 27.1.) The phase of Part II is not comparable, because its frames at the base vertices differ (Section 41).*

Steps 7 and 8, solution. *With the source at the apex,* $\Delta t = h^2$, *the four solutions put into the faces and compared corner by corner at the base: Part IV is within* $0.09°$ *of Part III, DEC within* $0.97°$, *and Part II differs by up to* $10.89°$.

The example shows in miniature everything that the experiments of Parts III to V show at scale: the same curvature in four places; Parts III and IV close together, because they distribute it in the same proportions; DEC a little further, because it drops the curvature inside the faces; and Part II furthest, because it distributes the curvature by the tilt of the faces, which this leaning pyramid makes uneven.

47. The Concept Map

Figure 42 draws every concept of the text as a node, and every arrow as a statement that the text proves, with the equation where it is proved. The upper band holds the smooth concepts, the lower band their discrete counterparts; the dashed arrows are discretizations. The colors say which part a node belongs to. Three paths through the map are worth following once.

- *From metric to curvature.* Edge lengths give, by the law of cosines, the angles; the angles around a vertex give its defect; the defect, shared in proportion to the angles, gives either the edge loops of Part III or the face holonomies of Part IV; and in both cases the total is $2\pi\chi$, the discrete counterpart of Gauss–Bonnet in the upper band.
- *From connection to operator.* The connection form, discretized as corner or edge transports, enters the element as phases; the elements, assembled, give the Hermitian connection Laplacian; the Galerkin method links it to the smooth connection Laplacian, and the Weitzenböck identity to the Hodge Laplacian.
- *From Part III to DEC.* Corner transport and edge transport differ by Proposition 40.1; the edge transport gives the curved element; dropping the curvature inside the integral gives DEC.

48. The Master Table and the Symbols

Table 7 lists every concept against every method, with the code that computes it, and Table 8 every symbol, with where it is defined. They replace, for reference, Tables 1, 2, 3 and 4, which remain where they are, each next to its own part.

49. Choosing a Method, Extending It, and What Can Go Wrong

Summary. For transport and direction problems on arbitrary meshes, use the intrinsic vertex planes; build energies with corner transport (Part III), and switch the element to the curved one of Part IV when the answer is an energy or when a conforming method is needed; keep DEC (Part V) as the reference against the literature. For physics on a known smooth surface, consider the extrinsic surface finite element method instead. Then follow the recipe and the checklist below.

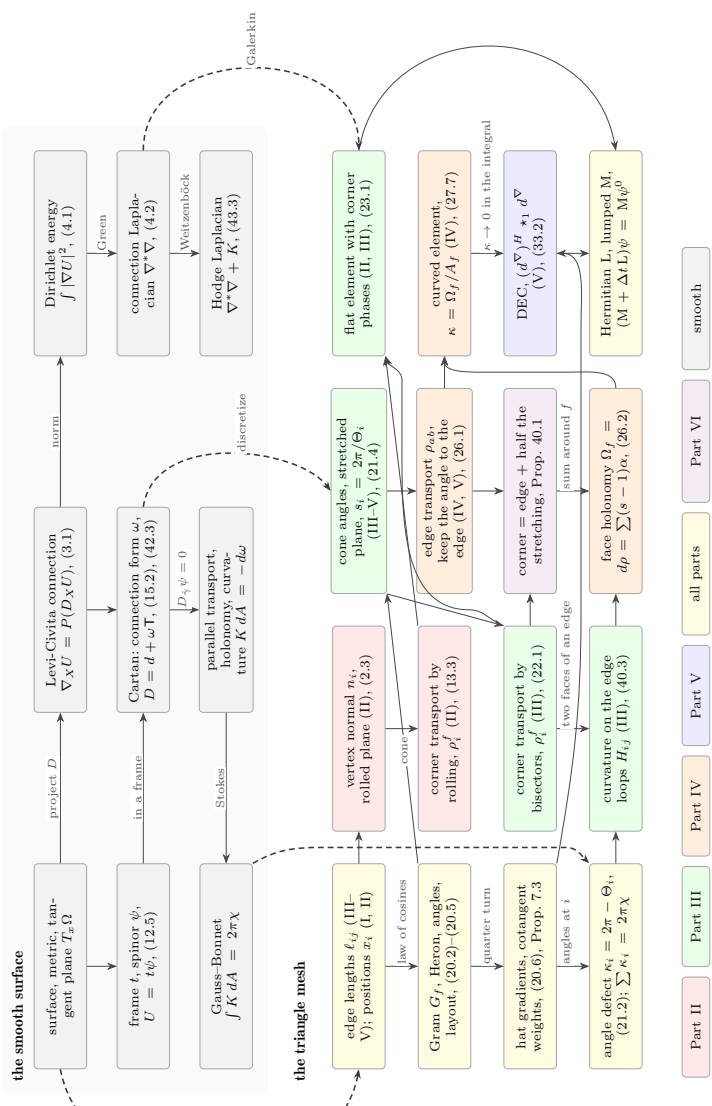


FIGURE 42. The concept map. Upper band: the smooth surface. Lower band: the mesh. Each arrow is a statement proved in the text, labeled with how it is obtained; each node gives the equation where it is defined. Dashed arrows go from a smooth concept to its discretization. Colors: the part to which a node belongs.

TABLE 7. The master table: every concept of the text, in the smooth setting and in each discrete method, with the function that computes it.

concept	smooth	I, II (extrinsic)	III (intrinsic)	IV (curved)	V (DEC)	code
input	surface in $\mathbb{R}^3$	positions x_i	lengths ℓ_{ij}	lengths	lengths	`intrinsic::compute`
face metric	first fundamental form	B, B^TB	G_f, (20.2)	G_f	G_f	`layOut`
area, gradients	dA, $\nabla\phi$	$\frac{1}{2}\lvert e_1 \times e_2\rvert$, $B(B^TB)^{-1}$	Heron, layout, (20.3), (20.6)	same as III	cotangents, (31.2)	`layOut`, `hatGradients`
tangent plane at a vertex	$T_x\Omega$	$\perp n_i$, (2.3)	cone $\times\, s_i$, (21.4)	same as III	same as III	`computeNormals`, `scale`
frame, gauge	t, ψ, (12.5)	projected reference edge	reference edge, $\varphi = 0$	same as III	same as III	`vertexT`, `outAngle`
connection	$\nabla_X U = P(D_X U)$	rolling rotor $Q_{i\to f}$, ρ_i^f, (13.3)	bisectors, ρ_i^f, (22.1)	edges, ρ_{ab}, (26.1)	edges, ρ_{ab}	`connection`, `rho`, `transport`
connection form	ω, (15.2)	0 in faces	0 in faces	ρ_{ab}; A in faces, (27.2)	ρ_{ab}; 0 in faces	—
moving frame	frame field, coframe θ^i, Rem. 15.1	constant per face; vertex frames from n_i	constant per face; vertex frames from the reference edge	layout frame with $d\omega = -\kappa\, dA$	vertex frames only	—
curvature	$K\, dA$	rolled gaps	edge loops H_{ab}, (40.3)	Ω_f, density Ω_f/A_f	Ω_f on ∂f	`holonomy`
Gauss–Bonnet	$\int K = 2\pi\chi$	gaps sum to defects	(40.4)	$\sum_f \Omega_f = 2\pi\chi$	same as IV	`totalCurvature`
trial field	—	$\sum \phi_a e^{\mathsf{T}\rho_a^f}\psi_a$, (16.1)	same	$\sum \phi_a e^{\mathsf{T}\theta_a} u_a$, (27.5)	none (crime)	—
covariant derivative	$D\psi$, ∇U	$\sum \nabla\phi_a\, e^{\mathsf{T}\rho_a}\psi_a$	same	$\nabla\phi_a + \mathsf{T}\phi_a w_a$, (27.6)	$d^\nabla\psi$, (33.1)	—
element	$\int_f \lvert D\psi\rvert^2$	$A\nabla\phi_a \cdot \nabla\phi_b e^{\mathsf{T}(\rho_b - \rho_a)}$	same, (23.1)	(27.7), quadrature	$e^{\mathsf{T}\rho_{ba}} A\nabla\phi_a \cdot \nabla\phi_b$	`gaform`, `intrinsicform`, `curvedElement`
global operator	$\nabla^*\nabla$, (4.2)	Hermitian L	same	same	$(d^\nabla)^H \star_1 d^\nabla$, (33.2)	`*form::solve`, `decform::laplacian`
mass	$\int \lvert U\rvert^2$	$A_f/3$ lumped	same	same	$\star_0$ barycentric	—
time step	(4.4)	$(\mathrm{M} + \Delta t\mathrm{L})\psi = \mathrm{M}\psi^0$	same	same	same	`solveScaled`
exact on bent flat meshes	—	parallel folds only	yes	yes	yes	`foldExperiment`
sphere, 42 vertices, mean error	—	0.28°	0.33°	0.33°	0.41°	`sphereAll`
energy of a rotation field, 42 vertices	$8\pi/3 = 8.378$	7.902	7.898	8.312	7.777	`killingEnergies`

49.1. Which method for which problem

if the problem ...	then ...
is posed on a mesh that is the only data (scans, designs, folds)	intrinsic planes (III–V); the answer does not change under bending
adds new energy terms, integrated face by face	corner transport, Part III: every face flat, every integral the scalar one with a phase
asks for an energy, an eigenvalue, or a convergence proof	the curved element, Part IV: conforming, and seven to ten times more accurate on the energy of Section 43
prescribes holonomies, places singularities in faces, fixes a gauge	edge transport, Parts IV, V: the connection is a 1-form there
must match published geometry processing code	DEC, Part V: it is the connection Laplacian of [10, 17]
lives on a known smooth surface, couples to normal quantities, needs error estimates	extrinsic surface FEM, Section 44
is built on divergence and curl (flows, Hodge decomposition)	scalar 1-forms on edges, Section 43; this needs edge unknowns
needs frames of space or volumes	rotors instead of complex numbers, Section 45
needs curved geometry or higher order	the curved element of degree two in moving frames, Section 42 (a sketch, not implemented)

What is used where. As far as we know, the vertex-based connection Laplacian of geometry processing is, almost always, the DEC operator of Part V on the intrinsic vertex planes [10, 17]. Knöppel et al. also derive a finite element version with a constant curvature in every face, which is close to the curved element of Part IV, as far as we can tell from the formulation. Intrinsic triangulations [16], which improve a bad mesh without moving it, work only with operators built from edge lengths, which is one more reason for the intrinsic planes. Face-based fields and edge-based 1-forms are as common for direction field design [3, 18]. In numerical analysis, the extrinsic surface finite element method of Section 44 is the standard [7]. The construction of Part I, a discrete connection built from vertex normals, is not the standard of either community: it was the natural first step of this text, and the following parts explain how the other methods improve on it.

49.2. Extending the method to a new problem: a recipe

A new problem of the same family, a field that should be as smooth as possible subject to some conditions, is added in five steps.

1. *Write the continuous energy* as a sum of squares of tangent-valued and scalar quantities: $|\nabla U|^2$, $|U - U^0|^2$, $(\operatorname{div} U)^2$, $|U \cdot X|^2$ for a given direction X, and so on. If an operator is wanted instead, derive it from the energy with Green's identity, as in Section 4; the weak form is then the derivative of the energy.
2. *Choose the vertex planes and the transport* with the table above. Do not mix the frames of different methods.

3. *Write each term on one face*, with the trial field $\sum_a \phi_a e^{\mathsf{T}\rho_a^f}\psi_a$ of the chosen method. With corner transport this is always the scalar integral of the hat functions times the phase $e^{\mathsf{T}(\rho_b^f-\rho_a^f)}$, so the scalar finite element literature applies directly (for example the consistent mass $\frac{A_f}{12}(1+\delta_{ab})$, (8.2)).
4. *Impose the conditions*: a prescribed value by row replacement or symmetric elimination (Section 10); a soft condition by a penalty term in the energy; an exact linear condition by a Lagrange multiplier. Fields with n-fold symmetry (crosses, $n = 4$; lines, $n = 2$) are obtained by replacing every transport $e^{\mathsf{T}\rho}$ by $e^{\mathsf{T}n\rho}$ and every ψ by ψ^n [10].
5. *Test* on the three meshes of Section 24: a bent flat mesh (Miura-ori), where the exact answer is that of the flat sheet; a sphere under refinement, where the answer is known in closed form; and a coarse, sharp mesh (the cactus), where the methods should agree to a few degrees.

49.3. What can go wrong: a checklist

These are the mistakes that were made, or nearly made, while writing this text.

- *Comparing vectors at vertices.* A vertex has no tangent plane, so comparing a computed vector at a vertex with an exact one needs a choice of plane, and every choice favors one method; a first version of the experiments of Section 24 produced a spurious error of 1.9° in this way. Compare inside the faces, corner by corner, where every method's vector lies in a plane that exists.
- *Comparing phases of different gauges.* Matrix entries and solution coordinates depend on the vertex frames; compare across methods only gauge-invariant quantities, or methods that share their frames (Section 41).
- *Angles that wrap.* Every angle is defined up to whole turns: $\rho_{ba} = -\rho_{ab} + 2\pi$ in (26.1), the holonomy (26.2) up to 4π. Store and multiply unit complex numbers, $e^{\mathsf{T}\rho}$, rather than adding angles, and wrap angles only for printing.
- *The orientation of half edges.* Which face is to the left of a half edge, the order in which the half-edge iterator walks around a vertex (clockwise in `HalfEdge/`), and whether a transport goes from a to b or back: one sign error here conjugates a transport and still produces a plausible looking field. Check on the upright pyramid, where every number is known.
- *The scale at boundary vertices.* This text uses $s_i = 1$ at the boundary, Sharp et al. $s_i = \pi/\Theta_i$ (Section 21); the two give different fields near the boundary.
- *The two κ.* The angle defect κ_i of a vertex and the curvature density $\kappa = \Omega_f/A_f$ of a face share a letter (Table 8).
- *Symmetry lost by the Dirichlet row.* Row replacement makes the matrix non-Hermitian; use LU, or eliminate the constrained unknowns to keep the Hermitian positive definite form (Section 10).

- *Line ranges in this document.* The listings read fixed line ranges of `GAConnection.h`, `IntrinsicConnection.h`, `EdgeConnection.h` and `ga_fem.cpp`; adding even one line near the top of a file shifts every listing below it. After editing those files, check the listings again (new code is best appended at the end).

49.4. The five parts in five sentences

Part I: the connection Laplacian is the scalar cotangent Laplacian with a rotation on every edge, and a vertex value is rotated into each face before it is interpolated. Part II: that rotation is one complex number of the face plane, and everything is intrinsic except the plane of a vertex. Part III: the plane of a vertex can be made from its angles, and then the method is exact on every bent flat mesh. Part IV: transporting along the edges moves the curvature into the faces, and a face with curvature needs a curved element. Part V: discrete exterior calculus is that element with the curvature inside the face dropped. And this part: the methods differ only in where they keep the curvature, Proposition 40.1 converts one into the other, Cartan's moving frames are the language in which all of them are a choice of frames and of a connection form, and the Weitzenböck identity separates them all from the Laplacian of 1-forms.

References

[1] S. Chun. Method of moving frames to solve conservation laws on curved surfaces. *Journal of Scientific Computing*, 53(2):268–294, 2012.

[2] J. N. Clelland. *From Frenet to Cartan: The Method of Moving Frames*, volume 178 of *Graduate Studies in Mathematics*. American Mathematical Society, 2017.

[3] F. de Goes, M. Desbrun, and Y. Tong. Vector field processing on triangle meshes. In *SIGGRAPH Asia 2015 Courses*. ACM, 2015.

[4] M. P. do Carmo. *Differential Geometry of Curves and Surfaces*. Prentice-Hall, 1976.

[5] M. P. do Carmo. *Differential Forms and Applications*. Universitext. Springer, 1994.

[6] C. Doran and A. Lasenby. *Geometric Algebra for Physicists*. Cambridge University Press, 2003.

[7] G. Dziuk and C. M. Elliott. Finite element methods for surface PDEs. *Acta Numerica*, 22:289–396, 2013.

[8] P. Hansbo, M. G. Larson, and K. Larsson. Analysis of finite element methods for vector Laplacians on surfaces. *IMA Journal of Numerical Analysis*, 40(3):1652–1701, 2020.

[9] D. Hestenes and G. Sobczyk. *Clifford Algebra to Geometric Calculus: A Unified Language for Mathematics and Physics*. D. Reidel, 1984.

[10] F. Knöppel, K. Crane, U. Pinkall, and P. Schröder. Globally optimal direction fields. *ACM Transactions on Graphics*, 32(4):59:1–59:10, 2013.

[11] H. P. Langtangen. Introduction to finite element methods. 2014.

[12] B. Liu, Y. Tong, F. De Goes, and M. Desbrun. Discrete connection and covariant derivative for vector field analysis and design. *ACM Transactions on Graphics*, 35(3):23:1–23:17, 2016.

[13] M. C. Lopez Belon. Project on github. Available at https://github.com/mauriciocele/basic-fem-connection-laplacian-degoes, 2024.

[14] P. Petersen. *Riemannian Geometry*, volume 171 of *Graduate Texts in Mathematics*. Springer, third edition, 2016.

[15] F.-J. Sayas. A gentle introduction to the finite element method. 2015.

[16] N. Sharp, Y. Soliman, and K. Crane. Navigating intrinsic triangulations. *ACM Transactions on Graphics*, 38(4):55:1–55:16, 2019.

[17] N. Sharp, Y. Soliman, and K. Crane. The vector heat method. *ACM Transactions on Graphics*, 38(3):24:1–24:19, 2019.

[18] A. Vaxman, M. Campen, O. Diamanti, D. Panozzo, D. Bommes, K. Hildebrandt, and M. Ben-Chen. Directional field synthesis, design, and processing. *Computer Graphics Forum*, 35(2):545–572, 2016.

Mauricio Cele Lopez Belon
Madrid, Spain
e-mail: mclopez@outlook.com

TABLE 8. The symbols of the text, where they are defined, and the traps among them.

symbol	meaning	defined in
T, T_i, T_f	3×2 frame matrix $[t \;\; n \times t]$	(2.1), (2.4)
T	unit tangent bivector In; the imaginary unit of a plane	(12.1)
ψ, u	spinor (complex) coordinate of a tangent vector, $U = t\psi$	(12.5), (2.5)
R_i^f, ρ_i^f	corner transport from vertex i into face f: 2×2 rotation, its angle	(3.5), (13.3), (22.1)
$Q_{i \to f}$	smallest rotation (rotor) taking n_i to n_f	(3.4), (13.2)
ω	connection form of a frame field	(3.3), (15.1)
G_f	Gram matrix, metric of a face	(20.1), (20.2)
α, α_a^f	corner angle (of face f at vertex a)	(20.4)
Θ_i	total angle around vertex i	(21.1)
κ_i	**angle defect** $2\pi - \Theta_i$ of a vertex (a total curvature)	(21.2)
κ	**curvature density** Ω_f / A_f inside a face of Part IV; not the same as κ_i	(27.1)
s_i	scale $2\pi/\Theta_i$ of a vertex; 1 on the boundary	(21.4)
θ, φ	cone angle at a vertex; stretched angle $\varphi = s\theta$	(21.3), (21.4)
$\theta_a(x)$	in Part IV: the angle of transport from $\bar{x}_a$ to x in a curved face; unrelated to the cone angle	(27.5)
β_e	direction of an edge in the face frame (layout)	Section 20
ρ_{ab}, r_{ab}	edge transport from a to b: angle, unit complex number; $r_{ba} = \overline{r_{ab}}$	(26.1)
Ω_f	holonomy of face f, $d\rho$ around it	(26.2)
H_{ab}	holonomy of the edge loop of corner transport	(40.3)
σ_a	angle from the plane of vertex a into the frame of a curved face	(27.4)
A	smooth connection 1-form inside a curved face, $\omega(X) = -A \cdot X$	(27.2)
w_a	$\nabla\theta_a - A$, the curvature term of the curved element	(27.6)
$\mathcal{A}(p, q, r)$	signed area of a triangle	(27.3)
d_0, d_1, d^∇	incidence matrices; covariant exterior derivative	(30.1), (33.1)
$\star_0$, $\star_1$, $\star_2$	Hodge stars: dual area, cotangent weight, inverse area	(31.1)
L, M	stiffness (connection Laplacian), lumped mass	(6.5), (8.3)
K	Gaussian curvature of a smooth surface	Section 14
Δ_{H}	Hodge Laplacian of 1-forms, $\nabla^*\nabla + K$	(43.3)
e_1, e_2; θ^1, θ^2	a moving frame and its coframe, $\theta^i(X) = e_i \cdot X$	Remark 15.1
F, J, $G(\xi)$	map, Jacobian and metric of a curved element of degree two	Section 42
κ_g, $[\kappa_g]$	geodesic curvature of an edge curve; its jump across the edge	(42.7)